TECHNISCHE UNIVERSITÄT
CHEMNITZ

Privacy-Preserving Source Code Vulnerability Detection and Repair using Retrieval-Augmented LLMs for Visual Studio Code

Master Thesis

Submitted in Fulfilment of the
Requirements for the Academic Degree
M.Sc. Web Engineering

Dept. of Computer Science
Chair of Computer Engineering

Submitted by: Md Hafizur Rahman
Student ID: 810641
Date: 11.05.2026

Internal Supervisor: Abubaker Gaber M.Sc.
Internal Examiner: Dr.-Ing. Sebastian Heil

Abstract

This thesis presents Code Guardian, a Visual Studio Code extension that detects and repairs JavaScript and TypeScript vulnerabilities entirely on the developer's machine. The system is intended for organizations that cannot use cloud-based code assistants for confidentiality, regulatory, or air-gap reasons.

Detection and repair are implemented as a two-stage pipeline backed by a local Ollama runtime, optional retrieval-augmented generation (RAG) over a signed CWE / OWASP / CVE corpus, JSON-mode decoding, multi-pass consensus filtering with an inter-run confidence gate, and an audit-mode AST + LLM hybrid for whole-project scans. A synchronous egress gate, an Ed25519-signed corpus manifest, and a containerized harness anchor the privacy and reproducibility claims. The system is evaluated on 101 curated cases (71 vulnerable, 30 secure) across five local models in LLM-only and LLM+RAG modes, three runs per sample, with three SAST baselines (Semgrep, CodeQL, ESLint) and a 15-case external corpus drawn from OWASP NodeGoat, OWASP Juice Shop, and three named CVEs.

The best configuration (`qwen3:8b` with RAG) reaches F1 71.43 %, precision 72.46 %, recall 70.42 %, FPR 10.00 %, and stage-1 detection median 2 216 ms under canonical-taxonomy matching un-gated. The inter-run confidence gate at $\geq 2/3$ agreement lifts precision to 78.13 % and F1 to 74.07 % at unchanged recall and FPR; on a held-out 30-case test split the same threshold yields F1 61.11 %, precision 73.33 %, recall 52.38 %, and FPR 0 %, confirming the threshold was not overfit to the evaluation set. Repair runs as a stage-2 call per finding under a structured `{code, language}` schema; the auto-applicable rate is 90.32 % (252 / 279), with the residual concentrated in mid-statement truncations and unterminated regex literals rather than prose. RAG is sharply model-dependent: it lifts `qwen3:8b` but degrades `codellama+RAG` by approximately 30 F1 points (the only effect surviving Bonferroni correction across the five tested models), so retrieval augmentation must be calibrated per model.

These results show that local privacy-preserving LLM workflows can support useful vulnerability detection and structured repair generation at interactive latency on consumer hardware. The held-out 71 / 30 threshold validation, the automated egress-and-leak harness, and signed-corpus integrity verification together substantiate the privacy and reproducibility claims.

Keywords: source code security, vulnerability detection, secure code repair, LLMs, RAG, privacy-preserving systems, VS Code

Acknowledgment

I am grateful to everyone who supported me during this Master's thesis.

My sincere thanks go to my internal supervisor, *Abubaker Gaber, M.Sc.*, for his continuous guidance, constructive feedback, and thoughtful discussions throughout the project. His support was essential in shaping both the technical direction and the academic quality of this work.

I also thank the academic staff of the Master's program for providing a strong foundation and a stimulating learning environment.

I am equally thankful to my colleagues and peers for their encouragement and for creating a supportive atmosphere during the study period.

Most of all, I thank my family for their patience, trust, and constant encouragement. Their support made it possible for me to complete this thesis.

Task Description

Traditional static application security testing (SAST) tools, such as Semgrep and CodeQL, are effective for detecting predefined vulnerability patterns but cannot reason about cross-file or context-dependent weaknesses. Cloud-based language model assistants provide stronger semantic analysis yet compromise data confidentiality and reproducibility because proprietary source code must leave the local environment. This thesis aims to design and evaluate a fully local, privacy-preserving secure coding assistant for Visual Studio Code that integrates large language models (LLMs) with Retrieval-Augmented Generation (RAG). All processing occurs on the developer's machine under a strict zero-egress policy to ensure that source code and intermediate data remain confidential.

The proposed assistant combines two complementary modes: real-time inline diagnostics that deliver immediate vulnerability alerts, and asynchronous audit and question–answering modes that use offline retrieval of vulnerability information such as CWE, CVE, and OWASP entries. A modular architecture separates the LLM inference layer, the local retrieval pipeline, and the Visual Studio Code extension interface. The system enforces privacy through local embeddings, signed corpora, and network isolation, while reproducibility is achieved through deterministic decoding, version-pinned dependencies, and containerized execution. The threat model assumes local adversaries capable of attempting code exfiltration, retrieval poisoning, or prompt injection, mitigated through corpus signing, network isolation, and provenance verification.

Evaluation will be conducted on benchmark and real-world JavaScript/TypeScript code. The comparison will be conducted across benchmark datasets (Juliet and OWASP Benchmark, approximately 40–50 test cases mapped to CWE identifiers) and one actively maintained real-world JavaScript/TypeScript project with documented vulnerabilities and verified patches. If the optional SAST baseline is executed, Semgrep and a scoped CodeQL subset will be included for contextual comparison. Evaluation metrics include precision, recall, F1-score, latency, resource usage, privacy validation, and reproducibility, all measured under deterministic local execution.

Contents

List of Figures

List of Tables

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
AST	Abstract Syntax Tree
BLEU	Bilingual Evaluation Understudy
BPMN	Business Process Model and Notation
CI	Confidence Interval
CI/CD	Continuous Integration / Continuous Delivery
CPU	Central Processing Unit
CVE	Common Vulnerabilities and Exposures
CWE	Common Weakness Enumeration
DPR	Dense Passage Retrieval
F1	F1 Score
FPR	False Positive Rate
GDPR	General Data Protection Regulation
GPU	Graphics Processing Unit
HCI	Human-Computer Interaction
HNSW	Hierarchical Navigable Small World
HTTP	Hypertext Transfer Protocol
IDE	Integrated Development Environment
JS	JavaScript
JSON	JavaScript Object Notation
JWT	JSON Web Token
LDAP	Lightweight Directory Access Protocol
LLM	Large Language Model
LRU	Least Recently Used
NVD	National Vulnerability Database
OWASP	Open Worldwide Application Security Project
RAG	Retrieval-Augmented Generation
RAM	Random Access Memory
RSS	Resident Set Size
SAST	Static Application Security Testing
SHA	Secure Hash Algorithm
SQL	Structured Query Language
SSRF	Server-Side Request Forgery
TS	TypeScript
TTL	Time To Live
URL	Uniform Resource Locator
VS Code	Visual Studio Code

XSS Cross-Site Scripting

1. Introduction

Injection flaws, cross-site scripting, and insecure data handling remain among the most frequently reported weakness classes in industry surveys [1, 2]. Two families of tools currently attempt to catch them before production. Static Application Security Testing (SAST) tools such as Semgrep and CodeQL scan source code for known patterns and integrate naturally into CI/CD pipelines [3, 4]. Large Language Models (LLMs) have more recently been applied to detection and repair suggestion [5, 6]. Neither family delivers all three properties developers need at once.

1.1. Problem Description

SAST tools are deterministic and privacy-preserving but shallow: they match syntactic patterns without understanding context, cannot explain *why* a finding matters, and rarely suggest how to fix it. The false-positive rate is high enough that developers routinely ignore the output under delivery pressure [7, 8]. LLM-based assistants close both gaps — contextual reasoning and concrete repair — but the dominant offerings transmit source code to cloud servers, which is unacceptable for organizations bound by confidentiality requirements, regulations such as GDPR, or air-gapped environments [9, 10]. The result is a three-way trade-off no current tool resolves: SAST gives determinism and privacy but not reasoning or repair; cloud LLM assistants give reasoning and repair but not privacy. Hybrid commercial offerings such as Snyk Code [11] partly close the gap by running detection in a proprietary engine that can be deployed locally. The engine is not a contextual LLM, however, and key features remain bound to vendor cloud services in the default configuration. A developer who needs contextual LLM reasoning, concrete repair suggestions, and a strict on-device privacy guarantee at once has no off-the-shelf option. This thesis explores whether a local, modular design — keeping inference on the developer's machine, grounding model reasoning in curated security knowledge, and integrating into the editor with developer-controlled remediation — can close that gap.

1.2. Motivating Scenario

To illustrate these challenges in a realistic development setting, consider a typical workflow at a fintech company building a Node.js backend.

A developer is implementing an Express.js endpoint that retrieves user account details. The handler reads a user ID from the request query string and constructs a SQL query to look up the account. The code is straightforward, the tests pass, and the pull request is approved by a colleague:

Listing 1.1: Vulnerable Express.js endpoint with SQL injection

```
app.get('/api/account', (req, res) => {
  const userId = req.query.id;
  const query = "SELECT * FROM accounts WHERE id = '" + userId + "'";
  db.query(query, (err, results) => {
    res.json(results);
  });
});
```

The query is built by string concatenation. An attacker who sends `id=' OR '1'='1` retrieves every account in the database. No one notices the vulnerability until a penetration test three months later.

The scenario surfaces every arm of the three-way trade-off in a single endpoint. A linter (ESLint) stays silent because the concatenation is syntactically valid. A SAST scanner (Semgrep) at best produces a generic "potential SQL injection" warning without grounding in the specific `req.query` → `db.query` data flow. A cloud LLM assistant (GitHub Copilot) explains and repairs the issue but requires transmitting the source to an external server. No tool provides context-aware detection, actionable repair, and privacy together. A local AI assistant integrated into VS Code closes this gap. As the developer writes the endpoint, the assistant analyzes the enclosing function, identifies the unsanitized `req.query.id` flowing into a SQL string, and produces:

> **SQL Injection (CWE-89):** User-controlled input `req.query.id` is concatenated into a SQL query without sanitization. Replace with a parameterized query: `db.query("SELECT * FROM accounts WHERE id = ?", [userId])`.

The finding appears as an inline diagnostic in the editor. The developer hovers to read the explanation, clicks the quick-fix action to preview the parameterized query, and applies the patch — all without the code leaving the machine. This is what Code Guardian provides: contextual detection, grounded explanation, and developer-controlled repair, running entirely on localhost.

1.3. Scope of the Thesis

This thesis focuses on **JavaScript/TypeScript** projects in **Visual Studio Code**. The primary privacy goal is **no source-code exfiltration**: all code and IDE context are processed exclusively by local services running on the developer's machine. The

system is designed as a VS Code extension that connects to a local Ollama instance for LLM inference, with an optional retrieval-augmented generation (RAG) module backed by a local HNSW vector store for security knowledge grounding.

The technical scope includes local LLM inference via Ollama, function-level code scoping, structured JSON output enforcement, and retrieval-augmented prompting with curated security knowledge bases (CWE descriptions, OWASP guidance). No model fine-tuning or training is performed; the focus is on system-level orchestration — prompt design, output parsing, caching, and IDE integration — using off-the-shelf local models served through Ollama [12].

The evaluation is limited to the accuracy, consistency, and latency of the structured output produced from code snippets in a controlled test harness. This thesis does not cover areas such as multi-language support beyond JavaScript/TypeScript, real-time collaborative editing, endpoint hardening, hardware-level attacks, or enterprise identity and network controls. Optional knowledge-base refresh may fetch *public* vulnerability metadata (for example CVE/CWE descriptions) and can be disabled for fully offline operation.

1.3.1. Research Questions

The thesis addresses three research questions, referenced throughout as RQ1–RQ3. **RQ1 (Feasibility):** To what extent can a locally deployed LLM integrated into VS Code provide useful vulnerability detection and repair suggestions without transmitting source code to external services? **RQ2 (Grounding):** How does retrieval-augmented prompting influence detection quality compared to an LLM-only configuration, and to what extent is that effect uniform across model families? **RQ3 (Practicality):** What performance mechanisms (scoping, caching, debouncing) are necessary to make LLM-based security feedback usable within interactive IDE workflows?

1.3.2. Threat Model

The main threat addressed is unintended disclosure of proprietary code through external analysis services. Local inference reduces this risk surface but does not remove all security risks. Table 1.1 summarizes the threat classes considered, the attack path each assumes, and the design response carried into the chapters that follow.

Table 1.1.: Threat model summary for Code Guardian.

Threat class	Assumption / attack path	Design response in this thesis
Source-code exfiltration	External API calls could expose proprietary code or derived context.	Local inference only; a synchronous egress gate blocks every non-loopback connection by default.
Prompt injection	Attacker-controlled code/-comments attempt to override analysis instructions [13].	Strict JSON-mode contract for both detection and repair, defensive parsing, and user approval before applying any fix.
Retrieval poisoning	Low-quality or malicious knowledge entries reduce grounding quality.	Local curated knowledge corpus verified at load time against an Ed25519-signed SHA-256 manifest with bound provenance.
Local host compromise	Malware or untrusted local users access local code, prompts, or caches.	Out of scope; assumes trusted host and OS-level hardening.

1.4. Objectives

The main objective of this thesis is to design and evaluate an IDE-integrated assistant — called Code Guardian — that provides contextual vulnerability detection and repair suggestions while preserving source-code confidentiality. The system aims to overcome the limitations of existing approaches by combining local LLM inference with retrieval-augmented generation in a privacy-preserving architecture that runs entirely on the developer's machine.

To achieve this goal, the following three specific objectives are defined:

1. **Design a privacy-preserving VS Code extension for local vulnerability analysis** that separates the process into a two-stage pipeline: detection followed by developer-controlled repair suggestion. The extension enforces structured JSON output for machine-checkable diagnostics, integrates an optional RAG module backed by curated security knowledge (CWE descriptions, OWASP guidance), and implements performance mechanisms — function-level scoping, LRU caching, and debounced triggering — for practical IDE interaction latency.
2. **Evaluate system performance across local model configurations** using a documented evaluation harness with 101 test cases spanning 14 CWE categories. Five local models are tested in both LLM-only and LLM+RAG modes, reporting precision, recall, F1, false-positive rate, JSON parse success, and end-to-end latency. Results from all configurations are compared to identify the most effective trade-off between detection quality, output robustness, and response time.

3. **Assess the practical impact of retrieval-augmented prompting** by comparing LLM-only and LLM+RAG configurations to determine whether grounding model reasoning in external security knowledge improves detection quality and explanation grounding, or whether the added retrieval context introduces noise that degrades specific model configurations.

These objectives guide the development and validation of a system that is not only technically functional but also practical for interactive developer workflows.

1.5. Thesis Outline

The thesis begins with the conceptual foundation. Chapter 2 reviews related work on static analysis tools, LLM-based vulnerability detection, and retrieval-augmented generation for secure coding, and derives the R1–R5 requirements that frame the rest of the thesis. From these requirements, Chapter 3 presents the overall system concept — the two-stage detection-then-repair pipeline, structured JSON output enforcement, the optional RAG module, and the architectural decisions that follow from the privacy goal.

Building on the concept, Chapter 4 describes the VS Code extension itself, including the technology choices and the core components (analyzer, RAG manager, analysis cache, diagnostic rendering) with an emphasis on modularity and local execution. Chapter 5 then examines system performance across five local models in both LLM-only and LLM+RAG configurations, reporting precision, recall, F1, false-positive rate, JSON parse success, latency, and the auto-applicable repair rate against the R1–R5 requirements.

Chapter 6 consolidates the headline findings, names the limitations and threats to validity, and states the deployment implications. Chapter 7 sketches the directions follow-on work should pursue. The appendices provide supplementary material, including detailed implementation results (privacy verification evidence, configuration tables, prompt contracts) and detailed experimental results (full per-configuration accuracy tables, reproducibility measurement, qualitative case studies).

2. Analysis

This chapter defines the requirements for automated vulnerability detection and repair assistance in an IDE and positions the thesis approach against related work. Following the style of the reference theses, the chapter first derives the requirements baseline, then reviews existing approaches, and finally summarizes the implications for the proposed concept.

2.1. Requirements Analysis

The thesis targets local IDE assistance for JavaScript/TypeScript development: detect vulnerabilities, explain them, and propose repairs without code exfiltration. Compared with rule-based SAST, the approach adds LLM reasoning and optional retrieval grounding, but must still satisfy strict operational constraints.

Based on the problem statement and related work, five requirements define the evaluation baseline. Table 2.1 summarizes them before they are discussed in detail.

Table 2.1.: Requirements overview for Code Guardian.

ID	Requirement	Operational intent
R1	Accuracy	Find real vulnerabilities with useful precision, recall, and controlled false positives, using context-aware reasoning.
R2	Consistency	Produce stable findings and structured output under fixed settings.
R3	Repair quality	Suggest concrete, security-improving fixes while keeping the developer in control.
R4	Usability	Deliver feedback at latencies that fit normal IDE workflows.
R5	Privacy	Keep all analysis local with no source-code transmission.

The following subsections define each requirement in detail and map it to measurable criteria. Table 2.2 traces the concrete harness measures for each requirement; threshold scales are introduced in the corresponding subsections.

Table 2.2.: Traceability from requirements to harness measures.

ID	Requirement	Harness measures
R1	Accuracy	Precision, recall, F1, sample-level secure FPR; canonical-taxonomy type matching with per-canonical-category P / R / F1; inter-run confidence and confidence-gated metrics.
R2	Consistency	JSON parse success rate; inter-run detection agreement (%); label and severity agreement (%) over 3-run subsamples.
R3	Repair quality	Manual review of repair fix rate, semantic correctness, strategy alignment, executable-code share on a 25-sample subset; auto-applicable rate (`@babel/parser` syntax check on every `suggestedFix`) plus `byReason` breakdown over the full set.
R4	Usability	End-to-end latency (mean and median, ms) per configuration; stage-split components `inferenceTimeMs` and `parseTimeMs` recorded per call.
R5	Privacy	Five-item architectural verification checklist.

Every measure is computed by the harness from per-case logs; only the R3 manual-review portion is explicitly hand-aggregated. The reporting of each measure against its requirement appears in Chapter 5.

2.1.1. R1: Accuracy

R1 addresses accuracy. Code Guardian should find real vulnerabilities while keeping false alarms low. In an IDE workflow, accuracy means not just finding issues, but producing results that developers can trust and act on. This includes context-aware reasoning: the system should tell apart code that is actually vulnerable from code that only looks suspicious, by considering surrounding logic like input validation and sanitization.

This requirement comes from a key trade-off in security tools. Traditional tools often produce too many false warnings because they match surface patterns without understanding context [7, 4]. LLM-based systems can both over-warn and miss real issues, depending on the model and prompt [14, 15]. Without enough context, they may give confident but wrong explanations. A useful tool must be judged by how correct its findings are, not just by whether it can explain them.

For this thesis, R1 means detection quality must be good enough to deserve developer attention under local deployment. This includes recognizing when mitigation is already present and avoiding false conclusions when context is missing. The design uses scoped code extraction, structured prompts, optional retrieval grounding, comparison with rule-based tools, and clear reporting of precision, recall, F1, and false-positive rate (FPR).

Accuracy has four key parts. Precision measures how many reported issues are real, since too many false alarms cause developers to ignore the tool. Recall measures how many real vulnerabilities the system finds, including those that depend on data flow and surrounding context. F1 balances both into a single score. The secure-sample FPR tracks how often the system flags safe code as vulnerable, which is the best predictor of whether developers will accept the tool [7].

To allow comparison with other tools and across Code Guardian's own settings, the thesis defines a four-level scoring scale. The thresholds follow standard conventions from information retrieval and software engineering [7, 14]. Table 2.3 shows this scale.

Table 2.3.: Evaluation scale for R1: Detection Accuracy (example thresholds).

Visual Score	**Interpretation (with thresholds)**
	**High accuracy.** F1 ≥ 0.75, precision ≥ 0.70, recall ≥ 0.70, and FPR ≤ 0.20. The system finds vulnerabilities well with few false alarms. Suitable for always-on IDE use.
	**Medium accuracy.** F1 ≥ 0.60, precision ≥ 0.55, recall ≥ 0.55, FPR ≤ 0.40, and not in the High band. Results are worth reviewing with some manual checking needed. Suitable for on-demand or audit-mode use.
	**Low accuracy.** F1 ≥ 0.45, FPR ≤ 0.70, and not in the Medium or High band. Findings need heavy manual checking. Only useful alongside rule-based tools.
	**Insufficient accuracy.** F1 < 0.45 or FPR > 0.70. Detection is too weak or false alarms too frequent. The system is not useful in practice.

2.1.2. R2: Consistency

R2 addresses detection consistency. Code Guardian should produce stable results when the same code is analyzed multiple times with the same settings. In security analysis, unstable output quickly reduces trust and makes it harder for developers to act on findings.

This requirement matters because LLMs are nondeterministic. They can produce different outputs, schema errors, or changed labels across runs [14, 6]. Even if detection quality is good, inconsistent output makes a tool unsuitable for IDE use. Developers need to trust that warnings will not change randomly between runs [7].

Consistency has four key parts. First, structured output: the system must return valid JSON that follows the expected schema, since broken output is useless for IDE automation. Second, detection agreement: running the same code twice should produce the same findings. Third, labeling: vulnerability types and severity ratings should stay the same across runs. Fourth, localization: reported line ranges should not shift between runs.

For this thesis, R2 means stable JSON output, stable findings across repeated runs, and repeatable labels and locations. The design uses a strict JSON contract, defensive parsing, scoped prompts, caching, and optional retrieval grounding to reduce random variation.

A four-level scale is defined for R2. Output stability is measured by JSON parse success rate. Detection agreement is measured by inter-run agreement rate (the fraction of vulnerable samples for which all N repeated runs agree on the binary detection outcome). Label and severity stability are measured as raw agreement percentages over samples detected in all runs. Cohen's κ is not computed because it requires a baseline-rate adjustment that varies across the unbalanced per-category sub-populations in the dataset; the raw agreement percentages reported here are interpretable directly and align with how the harness computes them. Table 2.4 shows this scale.

Table 2.4.: Evaluation scale for R2: Detection Consistency (example thresholds).

Visual Score	**Interpretation (with thresholds)**
	**High consistency.** JSON parse rate $\geq 99\%$, detection agreement $\geq 90\%$, label / severity agreement $\geq 90\%$. Output is stable and reliable. Suitable for always-on IDE use.
	**Medium consistency.** JSON parse rate in $[95\%, 99\%)$, detection agreement in $[75\%, 90\%)$, or label / severity agreement in $[75\%, 90\%)$. Mostly stable with occasional changes. Suitable for on-demand or audit-mode use.
	**Low consistency.** JSON parse rate in $[85\%, 95\%)$, detection agreement in $[60\%, 75\%)$, or label / severity agreement in $[60\%, 75\%)$. Frequent variation. Findings need repeated confirmation before acting.
	**No consistency.** JSON parse rate $< 85\%$, detection agreement $< 60\%$, or label / severity agreement $< 60\%$. Output is unreliable. The system cannot produce repeatable findings.

2.1.3. R3: Repair Quality

R3 addresses repair quality. Code Guardian should not only detect vulnerabilities but also suggest concrete fixes. A good repair is specific to the affected code, follows recognized secure-coding practices, and is presented as a suggestion that the developer can review and accept or reject.

This matters because developers often receive warnings without usable guidance on how to fix them, which increases effort and reduces follow-through [7, 8]. At the same time, research on automated program repair shows that generated patches are only useful when they fix the root cause without introducing new problems [16].

For Code Guardian, R3 means repairs should target the affected code region, use established mitigations like parameterization, input validation, and safer API choices, and be delivered as optional diffs rather than automatic edits. The developer always keeps control over whether to apply a fix.

Repair quality is measured along two complementary axes. The first is a qualitative manual review on a sampled subset, scoring each repair on whether it addresses the reported vulnerability type, follows recognized secure-coding patterns, and is presented as syntactically applicable code rather than prose guidance. The second is the deterministic, fleet-wide *auto-applicable rate*: the fraction of generated `suggestedFix` strings that syntactically parse as JavaScript or TypeScript without errors. The manual review captures *decision-support quality* (would a developer benefit from reading this?); the auto-applicable rate captures *automation quality* (could the IDE quick-fix surface insert this without further editing?). Both are reported in Chapter 5.

Table 2.5.: Evaluation scale for R3: Repair Quality (example thresholds).

Visual Score	Interpretation (with thresholds)
●	**High.** $\geq 75\%$ of repairs are syntactically valid, address the correct vulnerability type, and follow established secure-coding patterns. Fixes are specific and ready for developer review.
◑	**Medium.** $[50\%, 75\%)$ of repairs are valid and relevant. Most fixes target the right issue but may use generic patterns or need minor edits before applying.
○	**Low.** $< 50\%$ of repairs are valid and relevant. Fixes are mostly unusable, too generic, or miss the root cause.

The threshold scale for R3 (Table 2.5) combines them: the rate qualifier targets the manual-review proportion, with the auto-applicable rate reported as a secondary

signal in the corresponding evaluation section. Unlike R1 and R2, which use four levels, a coarser three-level scale is appropriate here because repair quality is partly qualitative and fine-grained distinctions are not reliable on small reviewer samples.

2.1.4. R4: Usability

R4 addresses usability. Security feedback is only useful if it arrives fast enough to fit normal editing workflows. A tool that takes too long will be ignored, no matter how accurate it is.

This is especially important for local LLM-based systems because model inference, retrieval, and prompt processing can make analysis noticeably slower than traditional diagnostics [17, 18]. Growing delay breaks the developer's flow and reduces willingness to use the tool.

For Code Guardian, R4 means that both inline and on-demand analysis must stay practical on real hardware. The design uses function-level scoping, debouncing, caching, and separate modes for lightweight inline checks versus heavier full-file scans.

Usability is measured on a three-level scale based on end-to-end latency. End-to-end latency is the wall-clock time from request submission to parsed result. The harness additionally records this latency in two components — `inferenceTimeMs` (the duration of the local model call) and `parseTimeMs` (the duration of the response parser) — so that the source of any observed latency change can be attributed unambiguously when comparing configurations. The threshold scale itself uses end-to-end median latency, since that is what the developer actually experiences. The thresholds reflect established response-time guidelines from HCI research [17, 18]. Table 2.6 shows this scale.

Table 2.6.: Evaluation scale for R4: Usability (example thresholds).

Visual Score	Interpretation (with thresholds)
●	**High.** Median latency $\leq$ 5s for inline analysis, $\leq$ 15s for full-file scans. Feedback feels responsive and fits normal editing flow.
◑	**Medium.** Median latency in $(5\text{s}, 15\text{s}]$ for inline, $(15\text{s}, 30\text{s}]$ for full-file. Noticeable delay but still practical for on-demand use.
○	**Low.** Median latency $>$ 15s for inline or $>$ 30s for full-file. Too slow for regular IDE use. Only practical for planned audit scans.

2.1.5. R5: Privacy

R5 addresses privacy. Code Guardian must detect and repair vulnerabilities without sending source code to external services. All prompts, findings, retrieved context, and generated fixes should stay on the developer's machine.

This matters because source code often contains proprietary logic or regulated information. Many organizations cannot use cloud-based analysis tools for this reason. Partial redaction does not solve the problem, since security-relevant context is spread across the code and may be lost when stripped.

For Code Guardian, R5 means local model inference, local retrieval, and clear separation between code-bearing workflows and optional public-metadata refresh paths. The design uses Ollama for local inference, local vector storage for RAG, and disabled-by-default background data updates.

Unlike the other requirements, privacy is a design constraint rather than a spectrum. Table 2.7 defines the verification checklist.

Table 2.7.: Verification checklist for R5: Privacy.

	Privacy criterion
☐	LLM inference runs locally via Ollama. No source code is sent to cloud APIs.
☐	RAG vector store and embeddings are stored locally. No code-derived data leaves the machine.
☐	Analysis prompts and results stay on localhost. No telemetry or external transmission.
☐	Optional vulnerability data refresh uses only public metadata (NVD, OWASP, CWE) and can be disabled for fully offline operation.
☐	No source code or code fragments are included in any outbound network request.

2.2. Related Work

This section reviews prior work on SAST, LLM-based security assistance, retrieval-augmented prompting, and privacy-preserving IDE integration, then positions the thesis approach against these paradigms.

2.2.1. Traditional Static Application Security Testing

SAST tools remain the default workhorse for vulnerability detection in many development workflows. Rule-based analyzers and taint-analysis systems such as Semgrep and CodeQL statically inspect source code for predefined patterns and data-flow queries [19, 20]. Their main advantage is operational: results are deterministic, reproducible, and easy to integrate into CI/CD pipelines and compliance-driven environments.

From a research perspective, modern static analyzers build on foundational ideas such as abstract interpretation [21] and scalable bug-finding techniques [22]. In practice, large-scale deployments demonstrate that static analysis can find real defects in industrial codebases at massive scale [3, 23]. Semantic approaches based on code property graphs unify abstract syntax, control flow, and data flow into a single queryable representation, supporting expressive vulnerability detection beyond surface-pattern matching [24]. Taint-tracking analyses such as FlowDroid further demonstrate that flow-, context-, and field-sensitive reasoning can be made tractable on real software [25]. Security-oriented static analysis has also been studied explicitly, including work that frames static analysis as an effective security engineering control when combined with secure development processes [4, 26].

A second advantage of SAST is *auditability*. Findings are tied to explicit rules or queries, which enables traceable reasoning, stable regression behavior, and predictable security gates. This is particularly relevant in regulated settings, where organizations need evidence of controls rather than only aggregate accuracy claims. Determinism also simplifies governance: when a rule is updated, teams can review the diff in findings directly instead of inferring changes from opaque model behavior.

These strengths come with well-known limitations. SAST tools can struggle with contextual reasoning and generalization, producing false positives when a risky-looking pattern is guarded elsewhere (e.g., validation in a different function, framework-enforced invariants) and false negatives when vulnerabilities depend on semantic relationships or framework-specific behavior. Language and framework abstractions further complicate exhaustive rule coverage and often push teams toward conservative, noisy rulesets.

There is also a structural precision/recall tension in static analysis deployments. Aggressive rule sets can improve vulnerable-case coverage, but often increase triage burden through noisy alerts; stricter rules reduce noise but risk under-detection. The trade-off is not only technical but organizational: teams with limited security-review capacity may prefer conservative rule sets, while high-assurance environments may tolerate larger alert queues to avoid misses.

Finally, many SAST tools provide limited remediation guidance, requiring developers to interpret findings and design fixes manually. Empirical studies show that warning overload and poor actionability reduce adoption and remediation rates [7, 8]. This

gap between *detection* and *remediation support* is one reason why SAST outputs are often strongest as gatekeeping signals, but weaker as day-to-day developer coaching tools.

2.2.1.1. Evaluation Against Requirements

Table 2.8 evaluates the three SAST tools discussed above against requirements R1–R5 using the visual scale defined in Section 2.1.

Table 2.8.: Evaluation of traditional SAST solutions against requirements R1–R5. Empty circles for the research-paper systems indicate that the requirement is not addressed by the work, not that the work fails to satisfy it.

Solution	**R1**	**R2**	**R3**	**R4**	**R5**
Semgrep [19]	◑	●	○	●	●
CodeQL [20]	◕	●	○	●	●
ESLint security	◔	●	○	●	●
Coverity [3]	◕	●	○	○	○
Code Property Graph [24]	◕	●	○	○	○
FlowDroid [25]	◕	●	○	○	○

The three deployable tools (Semgrep, CodeQL, ESLint security) score high on consistency (R2), usability (R4), and privacy (R5), reflecting deterministic execution, low latency, and on-device operation. Accuracy (R1) varies with rule sophistication: CodeQL's data-flow queries reach medium, Semgrep's pattern matching sits at medium-low, and ESLint security plugins remain low because they target style and obvious-sink rules rather than security-flow analysis. The three research-grade systems (Coverity, Code Property Graph, FlowDroid) demonstrate medium R1 accuracy through richer data- and taint-flow representations, but neither IDE integration nor deployment privacy is part of their published evaluation, so R4 and R5 are not addressed. None of the six provide actionable repair guidance (R3), leaving developers to design fixes manually. The full cross-paradigm comparison is in Appendix B.3. These limitations motivate approaches that keep deterministic checks as guardrails while adding more flexible reasoning and repair generation closer to the developer workflow.

2.2.2. LLM-Based Vulnerability Detection and Repair

Recent advances in LLMs have renewed interest in using learned models for code understanding, vulnerability detection, and repair suggestions. Contemporary systems build on transformer architectures [27]. Pretraining objectives in the style of BERT

and T5 then established the encoder and encoder-decoder backbones that downstream code models inherit [28, 29]. Both general-purpose LLMs and code-specialized models can generate syntactically plausible code and perform transformations such as refactoring and patch drafting [5, 6]. For IDE-integrated security assistance, these capabilities are attractive because they enable explanations and candidate fixes at the point of code.

However, empirical evaluations show that model-generated code can be *functionally correct yet insecure*, and that probabilistic generation can introduce hallucinated assumptions or omit critical security checks [15, 14]. Independent replications confirm this finding across model families [30], and developer studies report that AI-assisted coding can increase the rate of insecure code in user-written programs [31, 32]. This matters disproportionately in security, where small omissions (e.g., missing validation, unsafe sinks) can create exploitable weaknesses.

Recent survey work shows both the breadth and the fragmentation of this area. Reviews by Zhang et al. and Gholami cover broad cybersecurity use cases, but both report recurring problems in reproducibility, evaluation quality, false positives, and operational trust [33, 34]. Focusing specifically on code security, Basic and Giaretta show that the literature spans not only detection and repair but also security risks introduced by LLM-generated code itself [35].

A key difference from classical static analysis is that LLM behavior is strongly prompt- and format-dependent. Small changes in instructions, output schema, or contextual examples can materially change both finding rates and false-positive behavior. As a result, measured model quality is not only a property of model weights, but also of engineering choices such as scope selection, schema strictness, retrieval context, and failure handling.

For IDE integration, this dependence has an additional implication: free-form natural language answers are often insufficient for automation. Practical tools typically require machine-checkable outputs (e.g., JSON findings with line spans) and conservative handling of malformed responses. In this thesis, structured-output robustness is therefore treated as a deployment gate, not merely a presentation detail.

Before LLMs, machine-learning-based vulnerability detection explored learning semantic representations of code for classification. Early systems used deep learning over code fragments and patterns [36], while representation-learning work explored embeddings for code structure and semantics [37]. More recent approaches leveraged graph representations to capture richer program semantics [38, 39]. These methods improved over purely lexical baselines, but often required task-specific training data and did not naturally provide developer-facing explanations or repair suggestions. Subsequent empirical replications question how far these gains generalize beyond their training distributions [40, 41]. This motivates larger and more carefully curated benchmarks such as Big-Vul and DiverseVul [42, 43]. PrimeVul continues the same

curation effort [44]. Dedicated security-evaluation suites for code language models address the same gap [45, 46].

Finally, security-related failure modes of LLMs extend beyond simple false positives/negatives. The broader LLM risk literature emphasizes both ethical/operational risks [47] and adversarial behaviors such as jailbreaks and prompt-based attacks [48]. For IDE-integrated security tools, these risks motivate strict output contracts, careful prompt design, and conservative integration of model suggestions into developer workflows.

Another practical challenge is *calibration*. Many LLM-based assistants can explain a vulnerability convincingly even when the underlying label is wrong or weakly grounded [49, 50]. Without explicit confidence controls and abstention behavior, this can increase developer over-trust. For security tooling, persuasive explanations are not sufficient; the system must also provide stable structure, traceable evidence, and clear boundaries between high-confidence findings and uncertain hypotheses.

Recent empirical studies sharpen these concerns. Li et al. show that context-poor evaluation can underestimate LLM capability and distort rationale assessment [51]. IRIS shows that LLMs are stronger when coupled with static analysis than when used alone [52]. CWEVAL shows that static-rule scoring can miss or misjudge functionally correct yet insecure outputs, motivating outcome-driven security oracles [53]. SecRepair similarly frames secure repair as a full-pipeline problem spanning data, model adaptation, and evaluation [54].

In response, contemporary approaches increasingly combine learned models with auxiliary signals such as retrieval, tools, or deterministic checks. Retrieval-augmented generation provides a mechanism to ground model reasoning in external security knowledge without retraining [55, 56]. Tool-oriented prompting (e.g., using dedicated retrieval or analysis tools) further motivates modular designs where different components provide complementary evidence [57]. In Code Guardian, these ideas are reflected in the optional RAG module and the strict JSON-output contract used for IDE diagnostics and evaluation.

Repair suggestion quality is also connected to the broader literature on automated program repair. Classic systems such as GenProg and subsequent patch-learning approaches show both the promise of automation and the difficulty of evaluating patch correctness beyond passing tests [58, 16]. Modern neural repair work moves the field from heuristic search to sequence-to-sequence and edit-decoder formulations [59, 60], and ensemble or zero-shot approaches further demonstrate that pretrained code models can generate plausible patches without task-specific fine-tuning [61, 62]. For IDE-integrated security assistance, these findings motivate presenting repairs as *optional* quick fixes, requiring explicit developer review and preserving responsibility for functional correctness.

2.2.2.1. Evaluation Against Requirements

Table 2.9 evaluates the LLM-based detection and repair systems discussed above against requirements R1–R5.

Table 2.9.: Evaluation of LLM-based vulnerability detection and repair solutions against requirements R1–R5. Empty circles for the research-paper systems indicate that the requirement is not addressed by the work, not that the work fails to satisfy it.

Solution	**R1**	**R2**	**R3**	**R4**	**R5**
GitHub Copilot [5]	◕	◑	◑	●	○
IRIS [52]	◕	◑	◑	○	○
SecRepair [54]	◑	◑	◑	○	○
CWEval [53]	◕	●	○	○	○
VulDeePecker [36]	◑	◕	○	○	○
Devign [38]	◕	◕	○	○	○

LLM-based approaches and their pre-LLM deep-learning predecessors (VulDeePecker, Devign) lift accuracy (R1) into the medium band through contextual reasoning, and IRIS demonstrates that combining a static analyzer with the LLM can extend that lift further. Repair quality (R3) reaches medium-low for Copilot, IRIS, and SecRepair — repairs are produced but quality is not consistent — while CWEval, VulDeePecker, and Devign are detection-only and do not generate repairs at all. Consistency (R2) is held back by sampling variance and free-form output for the LLM systems; the deep-learning predecessors reach medium because deterministic inference removes sampling variance but free-form output is replaced by classifier scores rather than structured findings. Usability (R4) is high only for Copilot, which is IDE-integrated by construction; the research prototypes are batch-evaluated and do not report IDE latency. Privacy (R5) fails for the cloud-hosted Copilot and is not addressed by any of the research-paper systems, which assume cloud-hosted models or unspecified deployment. The full cross-paradigm comparison is in Appendix B.3. The gap across all approaches is the combination of local privacy, contextual reasoning, and repair generation in a single IDE-integrated tool.

2.2.3. Retrieval-Augmented Generation for Secure Coding

RAG is a general paradigm for grounding language model outputs in external knowledge without retraining. In RAG, a retriever selects relevant documents for a given query and the generator conditions on those documents when producing an answer [55]. Retrieval can be implemented with lexical scoring functions such as

BM25 [63] or with dense retrieval based on semantic embeddings. Dense retrieval approaches such as DPR and retrieval-augmented pretraining (REALM) motivate this design by showing that semantic retrieval can provide effective context for generation [56, 64]. Survey work further highlights RAG as a practical way to reduce hallucination and improve factuality [65, 14].

In secure coding settings, the retrieved context typically corresponds to vulnerability taxonomies (e.g., CWE), secure coding guidelines (e.g., OWASP cheat sheets), and historical vulnerability examples. This fits vulnerability detection and repair well because many issues are best explained by mapping code patterns to known weakness classes and established mitigations [2, 1]. By grounding the model in such sources, a system can improve detection consistency (R2) and explanation quality (R3), while reducing unsupported guesses.

Recent secure-code generation research suggests that *what* is retrieved matters as much as retrieval itself. RESCUE argues that raw security documents are often too noisy; it instead distills vulnerability-fix data into hierarchical guidance and concise code examples, then retrieves across security facets such as API pattern, vulnerability cause, and code structure [66]. The same lesson is relevant for vulnerability detection: curated security-specific retrieval units are likely to be more useful than generic prose.

Nevertheless, RAG introduces its own failure modes. If retrieval returns weakly related snippets, the generator can become distracted and produce confident but misaligned explanations. If retrieval returns overly generic security text, outputs may drift toward checklist-style warnings that increase false positives. In security tooling, retrieval quality is therefore as important as model quality: grounding helps only when retrieved context is both relevant and specific to the analyzed code pattern.

Implementation-wise, RAG depends on embedding models and efficient nearest-neighbor search. Sentence-level embedding methods such as Sentence-BERT are commonly used to map text into vector space [67]. Approximate nearest-neighbor indices such as HNSW provide high recall with practical latency [68], and libraries such as FAISS popularized large-scale similarity search in practice [69]. In Code Guardian, these ideas are realized through local embeddings and a persistent HNSW vector store to keep retrieval on-device and compatible with IDE latency constraints.

From an engineering perspective, RAG design involves a three-way trade-off among latency, grounding depth, and noise:

- increasing top-k retrieved chunks can improve recall of relevant guidance but expands prompt size and latency;
- larger chunks preserve context but may dilute precision in similarity search;
- aggressive knowledge refresh increases topicality but can introduce quality variance and reduce reproducibility if source curation is weak.

These trade-offs motivate model-specific calibration rather than one global RAG configuration. The same retrieval setup can improve one model while degrading another, depending on how each model integrates external context under strict output constraints. This observation is central to the ablation design in this thesis.

2.2.3.1. Evaluation Against Requirements

Table 2.10 evaluates the RAG approaches discussed above against requirements R1–R5. Vanilla RAG denotes the generic Lewis-style pipeline with off-the-shelf retrievers (BM25 or DPR) over uncurated security text; RESCUE is its curated, security-tuned successor.

Table 2.10.: Evaluation of RAG approaches for secure coding against requirements R1–R5.

Solution	**R1**	**R2**	**R3**	**R4**	**R5**
Vanilla RAG [55]	◑	◕	◔	○	○
RESCUE [66]	◕	◕	●	○	○

Both approaches reach medium consistency (R2), reflecting that retrieval grounding reduces but does not eliminate sampling variance. Accuracy (R1) and repair quality (R3) depend sharply on what is retrieved: vanilla RAG over generic security prose lifts both modestly, while RESCUE's curated hierarchical guidance reaches high repair quality by mapping vulnerability classes to concrete fix patterns. Neither approach addresses IDE integration (R4) or runtime privacy (R5) — both are evaluated as offline batch pipelines on cloud infrastructure. The full cross-paradigm comparison is in Appendix B.3. The key insight is that RAG is a useful augmentation, not a standalone solution.

2.2.4. Privacy-Preserving and IDE-Integrated Approaches

Privacy concerns pose a significant barrier to adopting LLM-based security tools in real-world settings. Cloud-hosted assistants require transmitting proprietary source code to external servers, which is unacceptable in many regulated or security-sensitive environments [10]. Beyond policy constraints, research has demonstrated concrete privacy risks in large language models, including memorization and extraction of training data [9] and inference attacks that raise questions about model confidentiality and data exposure [70]. For security tooling, these risks motivate designs that keep source code and analysis artifacts local by default.

The IDE context introduces additional security considerations. Because the assistant processes attacker-controlled inputs (e.g., comments, strings, dependency code),

prompt-injection and tool-manipulation attacks become relevant; OWASP explicitly catalogs such risks for LLM applications [13]. These concerns motivate designs that treat code as data, constrain outputs to machine-checkable schemas, and isolate retrieval sources to curated security knowledge.

IDE-integrated security tools can improve developer engagement and remediation rates by providing feedback during active development rather than post hoc analysis [7, 8]. However, many IDE assistants prioritize productivity features such as code completion and refactoring, with limited focus on security guarantees, reproducibility, or privacy boundaries. In practice, local deployment and deterministic interaction contracts become critical: users must understand what data is processed, where it is processed, and how results may vary across models and runs.

Local deployment, however, is not a complete security solution. Moving inference on-device shifts the trust boundary from cloud providers to local runtime integrity, workstation hardening, and extension-level data handling. Attackers who gain local access can still exfiltrate prompts, caches, or generated repairs unless operational controls (OS hardening, access control, and secure storage defaults) are in place.

A second practical boundary concerns *mixed connectivity*. Many systems operate with local inference but optional online knowledge refresh. This hybrid pattern can preserve source-code privacy while still introducing supply-chain and provenance risks if external knowledge sources are not validated. Privacy-preserving design should therefore separate code-bearing data paths from public-metadata update paths and make this separation visible to users.

2.2.4.1. Evaluation Against Requirements

Table 2.11 evaluates the IDE-integrated and locally-deployable security tools discussed above against requirements R1–R5.

Table 2.11.: Evaluation of privacy-preserving and IDE-integrated approaches against requirements R1–R5.

Solution	**R1**	**R2**	**R3**	**R4**	**R5**
GitHub Copilot [5]	◕	◑	◑	●	○
Snyk Code (local) [11]	◕	●	○	●	◕
Semgrep [19]	◑	●	○	●	●

Copilot leads on usability (R4) but sends code to the cloud, fully failing R5. Snyk Code's local mode reaches high consistency and usability but offers no repair generation (R3). Semgrep is fully local with the strongest privacy posture but its rule-based detection sits at medium-low accuracy (R1) and offers no contextual reasoning. The

combination of local privacy, contextual LLM reasoning, repair generation, and IDE integration in a single tool remains the unmet gap, as shown in the unified comparison in Appendix B.3.

2.2.5. Overall Evaluation and Gaps

The four paradigm-level evaluations in Sections 2.2.1.1–2.2.4.1 reveal a consistent pattern: each paradigm covers a different subset of R1–R5, but no paradigm covers the full set on its own.

What scores high across paradigms. Three properties are reliably achieved by at least one mature approach:

- **Consistency (R2)** is high for every deterministic tool (Semgrep, CodeQL, ESLint, Snyk) and reaches medium for the LLM and RAG approaches once retrieval grounding stabilizes outputs.
- **Usability (R4)** is high for every IDE-integrated tool (Copilot, Snyk, Semgrep), and absent for every batch research prototype (IRIS, SecRepair, RESCUE, CWEval) regardless of detection quality.
- **Privacy (R5)** is high for fully local SAST tools (Semgrep, CodeQL, ESLint), medium for Snyk Code's local mode, and absent for every cloud-hosted system (Copilot, IRIS, SecRepair, RESCUE, CWEval).

Where every paradigm falls short. Two properties are not reached by any reviewed approach in combination with the others:

- **Accuracy (R1) above the medium band** is reached by no reviewed system on its own. SAST tools sit at low or medium-low because pattern matching cannot reason about context; LLM systems reach medium because contextual reasoning helps but introduces hallucinations; RAG lifts medium-low to medium when the retrieved context is curated, but no paradigm reaches the high band.
- **Repair guidance (R3)** is either absent (SAST: none; CWEval: not in scope) or reaches at most medium-low for the LLM and RAG paradigms, with the partial exception of RESCUE's curated repair-pattern retrieval which reaches high but only inside an offline batch pipeline.

The unmet combination. Specifically, no reviewed system simultaneously achieves all four of the following: (1) contextual LLM reasoning, (2) repair generation, (3) IDE integration, and (4) on-device privacy. SAST tools cover (3) and (4) but not (1) or (2). Cloud LLM assistants cover (1), (2), and (3) but fail (4). Research prototypes cover (1) and (2) but neither (3) nor (4). Local-only SAST tools cover (3) and (4) but not (1) or (2). The cross-paradigm comparison in Appendix B.3 (Table B.2) makes this gap visible at a glance: the four columns most relevant to the unmet combination (R1 reasoning depth, R3 repair quality, R4 IDE latency, R5 privacy) are simultaneously high for no single reviewed approach.

Positioning of this thesis. Code Guardian targets exactly this unmet combination. It runs LLM inference and retrieval on-device through Ollama and a local HNSW vector store; integrates as a native VS Code extension that returns structured JSON findings into the diagnostic surface; generates one repair per detected finding through a separate Stage 2 call under a strict `{code, language}` schema; and evaluates the result with a local harness that uses the same dataset and scoring logic to score Semgrep, CodeQL, and ESLint as deterministic baselines. The positioning claim is not that local LLMs outperform cloud assistants on every metric, but that they enable a *different deployment envelope*: privacy-preserving IDE assistance with explicit control over model choice, retrieval strategy, and output contracts. Table 2.12 situates this envelope against the dominant alternatives on the design dimensions that distinguish them.

Table 2.12.: Positioning of Code Guardian against representative security-assistance approaches across the design dimensions that distinguish them.

Approach	Privacy	Contextual reasoning	Repair suggestions	IDE integration	Output contract
GitHub Copilot	Cloud-dependent	Strong (LLM)	Yes	Native	Unstructured
Snyk Code (local)	Local analysis	Rule-based	Limited	Plugin	Structured
Semgrep	Fully local	Rule-based	No	CLI/Plugin	Structured
IRIS / SecRepair	Varies	Strong (hybrid)	Yes	Batch-only	Research-grade
Code Guardian	Fully local	LLM + RAG	Yes	Native VS Code	Strict JSON

Concrete mechanisms relative to prior work. Relative to existing work, this thesis introduces five mechanisms not jointly applied by any reviewed approach. First, structured-output robustness is treated as a first-class requirement (strict JSON contract with format-level enforcement and defensive parsing). Second, multi-pass consensus filtering with seeded passes suppresses sampling-variance false positives.

Third, deterministic inference controls (`temperature=0`, fixed seed, JSON-mode decoding) address output instability. Fourth, RAG is treated as a model-dependent intervention measured per model rather than assumed universal. Fifth, a unified local evaluation harness compares against Semgrep, CodeQL, and ESLint under the same dataset and scoring logic. Together these mechanisms compose into five broader deliverables described in the conclusion: a privacy-preserving extension architecture, the two-stage detection-then-repair pipeline, the multi-pass consensus filter and confidence gate, the hybrid AST+LLM audit pipeline, and the reproducible evaluation harness.

2.3. Summary

This chapter derived five requirements (R1 Accuracy, R2 Consistency, R3 Repair Quality, R4 Usability, R5 Privacy) from the problem statement and reviewed four representative paradigms against them: traditional SAST, LLM-based vulnerability detection and repair, retrieval-augmented generation for secure coding, and privacy-preserving IDE-integrated tools. Each paradigm was rated against R1–R5 using a visual scoring scale, and the per-paradigm matrices were consolidated in the cross-paradigm positioning analysis. The consistent finding is that no single existing approach satisfies all five requirements simultaneously: SAST tools deliver R2, R4, and R5 but trail on R1 reasoning depth and offer no R3 repair guidance; cloud LLM assistants close R1 and R3 but fail R5 by transmitting source code; research prototypes reach R1 / R3 but lack R4 IDE integration and assume cloud deployment. The unmet combination — contextual LLM reasoning, repair generation, IDE integration, and on-device privacy in one tool — defines the design space the next chapter operates in. Chapter 3 derives the Code Guardian concept from these requirements and the gap analysis, mapping each architectural decision to the R1–R5 it primarily addresses.

3. Concept

This chapter presents the conceptual foundation of Code Guardian, grounded in the requirements and limitations identified in Chapter 2. The core objective is to detect and repair vulnerabilities in JavaScript and TypeScript code directly inside Visual Studio Code, while keeping source code on the developer's machine. Unlike cloud-hosted assistants that transmit code to external services, and unlike syntactic SAST tools that match patterns without contextual reasoning, the proposed system combines local large-language-model inference, optional retrieval-augmented grounding, and structured-output detection to bridge the three-way trade-off named in the introduction.

The design is motivated by three critical shortcomings observed in existing work. First, traditional SAST tools deliver determinism and privacy but match syntactic patterns without contextual reasoning, which produces high false-positive rates and no actionable repair guidance [7, 8]. Second, cloud-based LLM assistants close the reasoning and repair gap but transmit source code to vendor servers, which conflicts with confidentiality, regulatory, and air-gapped deployment constraints [9, 10]. Third, current research prototypes targeting LLM-based vulnerability analysis run as batch pipelines without IDE integration, so their detection quality does not translate into developer-facing feedback under interactive latency budgets [52, 54].

The proposed Code Guardian system addresses these limitations through a privacy-preserving, modular architecture that runs the entire detection-and-repair pipeline on the developer's machine. Each architectural decision maps directly to the five requirements established in Section 2.1: a local Ollama backend with a synchronous egress gate keeps source code on-device (R5); a two-stage detect-then-repair pipeline with strict JSON-mode decoding stabilizes structured output (R2) and isolates classification from remediation to lift detection precision and repair quality (R1, R3); function-level scoping, request debouncing, and an LRU analysis cache keep interactive latency inside IDE-usable bands (R4); and an optional RAG module grounds reasoning in curated CWE and OWASP guidance whose effect is calibrated per model rather than assumed universal (R1, model-dependent).

The remainder of this chapter is organised as follows. Section 3.1 derives the conceptual design from the analysis results, walking through how each architectural decision answers a specific requirement and recording the alternatives that were considered and rejected. Section 3.2 describes the components that realise this design — context extraction, knowledge retrieval, vulnerability detection, repair generation, and the IDE-integration surface — and specifies the input / output contract of each. Section 3.3 presents the workflows that orchestrate these components into

the surfaces a developer actually interacts with: real-time function diagnostics, on-demand file scans, interactive analysis, and workspace-wide audit. Section 3.4 provides the high-level system architecture using a C4-style decomposition — context view, container view, and process view — including sequence diagrams for the dominant interaction patterns. Detailed implementation and experimental validation are deferred to Chapters 4 and 5 respectively.

3.1. Concept Derivation from Analysis Results

The previous chapter identified five requirements that no existing tool satisfies simultaneously. This section walks through the main architectural decisions of Code Guardian and explains why each one was chosen, starting from the privacy boundary and moving inward through retrieval, detection strategy, and IDE integration.

3.1.1. From Cloud-Based to Privacy-Preserving Local Deployment

Local deployment is the starting design constraint, not an optimization. Cloud LLM assistants transmit source code to external services, which conflicts with GDPR [10] and the OWASP LLM Top 10 sensitive-information-disclosure risk [13]. Beyond policy, training-data leakage has been documented in code-generation models [9, 71]. Membership-inference attacks can also reveal whether specific fragments were in a model's training set [70]. Running the LLM, retrieval pipeline, and analysis components entirely on local hardware keeps source code, intermediate representations, and analysis results on-device (R5). The same setup enables fully offline operation when knowledge refresh is disabled. The privacy boundary is defined around *source code*: analyzed code and IDE-derived context are sent only to a local LLM backend; the optional knowledge-base refresh fetches public CVE/CWE/OWASP metadata only and can be disabled. The trade-off is increased latency relative to cloud accelerators, addressed by careful model selection, retrieval design, and prompt scoping (R4).

3.1.2. Retrieval-Augmented Generation for Security Knowledge Grounding

A standalone LLM can hallucinate vulnerability types, misclassify severity, and produce inconsistent results across repeated analyses [14]. Retrieval-augmented generation grounds reasoning in external knowledge that evolves independently of model parameters [55, 56]. The local knowledge base aggregates CWE-aligned vulnerability patterns and mitigations alongside OWASP guidance [2, 1]. It is supplemented by public CVE/NVD summaries [72, 73] and curated JS/TS notes

(prototype pollution, unsafe deserialization, etc.). During analysis, semantically-similar entries are retrieved and provided as context. Retrieval is not universally beneficial: empirical results in this thesis show it improves quality on some model families and degrades it on others, so it is treated as a configurable option rather than a default.

3.1.3. IDE Integration, Modular Components, and Two-Stage Pipeline

Traditional SAST tools sit downstream in CI/CD; their delayed feedback loop increases cognitive load and reduces remediation rates [7, 8]. Code Guardian integrates directly into Visual Studio Code, providing inline debounced detection during active development and on-demand audit commands for selections, files, and workspaces. IDE integration addresses R4 (interactive presentation of findings, explanations, and quick-fix repairs) and R3 (preview-and-apply remediation with minimal friction).

Internally the system decomposes into four components — context extraction (scope selection plus localization metadata), knowledge retrieval (vector search over the local corpus), vulnerability detection (LLM analysis grounded in retrieved context), and repair generation (developer-controlled patches as quick fixes). Each is independently testable and replaceable; intermediate outputs remain inspectable for debugging and reproducibility. Detection and repair are deliberately split into two stages: Stage 1 asks only "identify the vulnerabilities and return structured findings" (type, severity, location, description); Stage 2 takes each confirmed finding and runs a dedicated repair prompt with the full snippet. Coupling both tasks into a single prompt biases the model toward generating a plausible fix rather than accurately classifying the issue; separating them improves R1 detection precision and R3 repair quality, and lets repairs run in parallel.

3.1.4. Context-Aware Analysis and Structured Explainability

R1 demands semantic and structural reasoning rather than syntactic pattern matching. The system addresses this through reliable scope selection (the enclosing function via the TypeScript Abstract Syntax Tree, AST) plus prompt grounding (retrieved security guidance when RAG is enabled), and expects the model to reason about data flow, sources, sinks, and guards *within the provided scope*. Evidence outside the snippet (validation in another file, framework-level middleware) is the principal limitation and is named as future work. R3 and R4 additionally require explainable output: the diagnostics pipeline emits structured reports with code localization, a short risk-explanation message, and an optional quick-fix repair; interactive WebView modes can render longer Markdown explanations and incorporate retrieved CWE/OWASP

knowledge to ground the reasoning [2, 74]. The evaluation harness additionally requests `type` and `severity` fields to support quantitative scoring.

3.1.5. Deterministic Inference and Consensus Filtering

Output instability undermines developer trust (R2). Code Guardian fixes `temperature=0` and enables JSON-mode decoding (`format='json'`) so the sampler emits only valid JSON, eliminating parse failures. The detection prompt is then run multiple times with different seeds; only findings confirmed by a majority of passes are retained. If consensus removes everything, the system falls back to the first pass to avoid silent suppression.

3.1.6. Alternatives Considered for Each Design Decision

The subsections above present the selected design. For each major decision the team also evaluated competing options. Table 3.1 records the alternatives that were considered and the requirement-grounded reason each was rejected. The narrative paragraphs that follow expand on the three decisions where the trade-off is most consequential for the final system.

Table 3.1.: Design alternatives considered and rejected for each major concept decision, grounded in R1–R5.

Decision	**Alternatives considered**	**Reason for rejection**	**Selected**
Inference location	Cloud LLM API (GPT-4, Claude)	Transmits source code off-device; violates R5 by design.	Local Ollama
	Self-hosted private API on a remote server	Adds a network hop and an additional trust boundary; still leaves the developer's machine for analysis.	
Knowledge grounding	Pure LLM (no retrieval)	Hallucinates vulnerability types and severity (R1); no separation of model knowledge from CWE/OWASP guidance, which prevents updating the corpus without retraining.	RAG (configurable per model)
	Domain fine-tuning of the base model	Requires labeled vulnerability corpus and GPU training; couples knowledge updates to model retraining; out of scope for an IDE plugin.	
	Few-shot examples baked into the system prompt	Burns prompt budget on static examples and inflates latency (R4); cannot grow with new CVEs.	

(continued on next page)

(Table 3.1 continued from previous page)

Decision	Alternatives considered	Reason for rejection	Selected
Analysis scope	File-level (always send full file)	Files exceed the prompt budget for many real workspaces; latency becomes unbounded (R4).	Function-level + opt-in workspace scan
	Whole-project static-flow analysis	Approaches the cost of a full SAST engine and contradicts the interactive-IDE goal (R4).	
	Sliding-window scope (N lines around cursor)	Frequently splits a function across two prompts and loses syntactic boundaries; hurts R1.	
Prompt structure	Single combined detect-and-repair prompt	Biases the model toward producing a plausible fix at the cost of accurate classification; degrades both R1 and R3.	Two-stage detect-then-repair
	Single prompt with chain-of-thought reasoning trace	Inflates latency without measurable gain on the function-level scope; harder to enforce JSON contract (R2).	
Output contract	Free-text Markdown explanations	Cannot be machine-parsed reliably; breaks IDE diagnostics integration; fails R2.	Strict JSON with JSON-mode decoding + schema
	Loose JSON without format-mode decoding	Parse failures observed in pilot runs even at `temperature=0`; degrades R2.	
Vector store	Cloud-hosted (Pinecone, Weaviate)	Requires uploading embeddings derived from local data; conflicts with R5.	HNSWlib (local, on-disk)
	Flat brute-force search	Acceptable at the current corpus size but degrades as the knowledge base grows; rejected for forward compatibility.	
Consensus mechanism	Single pass	Sampling variance produces inconsistent findings on the same code; fails R2.	Multi-pass with explicit confidence + opt-in gate
	Binary majority vote (keep / drop)	Discards a useful per-finding signal; treats the precision/recall trade-off as a fixed design constant rather than a developer-configurable choice.	

(continued on next page)

(Table 3.1 continued from previous page)

Decision	Alternatives considered	Reason for rejection	Selected
Repair application	Auto-apply on accept	Risk of silent behavioral change; conflicts with R3 (developer control) and reviewability.	Preview-then-apply via Quick Fix
	Refactor-the-whole-file rewrite	Touches code unrelated to the finding; harder to review; higher chance of regressions.	

Local inference vs. remote private hosting. The closest non-cloud alternative to local Ollama is a self-hosted private LLM endpoint — a model running on a controlled remote server inside the organisation's perimeter. Both options keep source code out of vendor clouds. The local option was selected for three reasons. First, R5 is defined around the developer's machine, not the organisation's perimeter; a remote private endpoint still requires source code to cross a network boundary, which expands the privacy threat surface to network capture and remote-host compromise. Second, a local backend works air-gapped, which several target deployment scenarios require. Third, the latency profile of an IDE plugin running local LLMs is dominated by inference time on the developer's hardware; adding a network hop would push the on-demand band beyond the target 5 s ceiling on any non-LAN deployment. Remote private hosting remains a sensible option for organisations with shared GPU pools but is outside the scope this thesis defends.

Two-stage detect-then-repair vs. a single combined prompt. A combined detect-and-repair prompt is more compact and reduces total token count. It was rejected because empirical pilot runs showed that asking the model to both classify and fix biases the classification step toward findings that have a plausible fix — the model is more willing to flag a snippet when it can also propose a remediation. Splitting the two tasks isolates classification from remediation and lets each prompt focus on a single optimisation target. The cost is a second LLM call per finding; this cost is bounded because Stage 2 runs only on findings that survive the consensus filter, and it can run in parallel across findings. The two-stage choice supports a high auto-applicable rate together with the strict structural `{code, language}` schema enforced on the Stage 2 output; the empirical auto-applicable rate is reported in the evaluation chapter. A combined prompt would have to share that schema across the detection and repair fields, which the pilot runs found harder to enforce reliably under JSON-mode decoding.

Multi-pass consensus vs. binary majority vote. A binary majority vote is the simplest consensus rule: if at least $\lceil n/2 \rceil$ of n passes report a finding, keep it; otherwise drop it. The thesis instead exposes the agreement rate as a per-finding confidence value and lets an opt-in threshold gate which findings reach the diagnostic stream. This converts the precision/recall trade-off from a design constant to a developer-configurable knob: the default behaviour is no gating (all findings surface, ranked by confidence), but operators who care more about precision than recall can raise the threshold. The evaluation reports both with-gate and without-gate metrics so the contribution of each gate is attributable to the design rather than baked into the headline.

The next section describes the components that realise this design.

3.2. System Components

The Code Guardian system decomposes into five components, each motivated by a specific design pressure derived from R1–R5. This section characterises the responsibilities of each component, the requirement it primarily addresses, and the design rationale behind its inclusion. The components are presented in the order in which they touch a request: context extraction selects what to analyse, knowledge retrieval grounds the analysis, vulnerability detection produces structured findings, repair generation turns findings into developer-controlled patches, and the supporting subsystems orchestrate the data and metadata that flow around them. Implementation-level detail (parameters, defensive-parsing logic, cache keys, retry policy) is deferred to Chapter 4; this section stays at the level of conceptual contract.

3.2.1. Context Extraction

The context-extraction component selects the analysis unit from the developer's editor state. It accepts a text-document URI and a cursor position, walks the TypeScript AST, and returns the smallest enclosing function together with snippet-relative line offsets and metadata for downstream localisation.

Design considerations. Real-time feedback in the IDE requires an analysis unit that is small enough to fit interactive latency budgets (R4) and large enough to support contextual reasoning (R1). The function boundary is the natural compromise: it is a syntactically meaningful unit; it bounds prompt size predictably (~2 000 characters in the empirically chosen guard); and it preserves the local data-flow context within which most function-level vulnerabilities are decidable. A size guard skips unusually large functions to bound worst-case latency.

Design rationale. Function-level scope keeps prompt-construction time predictable and isolates the analysis from cross-file evidence the model cannot see. Cross-file flows (validation in another module, framework-level middleware) are out of scope for the inline path and are handled either by the workspace-scan workflow or named as future work. The extractor also exposes an explicit-selection mode (any user-selected range), a file-level mode (the full active document under a 20 000-character guard), and a workspace-level mode (filtered by file size and dependency-folder exclusions) so deeper inspection remains available on demand.

3.2.2. Knowledge Retrieval (RAG)

The knowledge-retrieval component grounds model reasoning in curated security guidance. It accepts a code snippet, embeds it locally, performs a vector search over a local HNSW index, and returns the top-k matching CWE, OWASP, and CVE entries as additional prompt context. The store is on-device; only optional metadata-refresh paths fetch public CVE/NVD summaries from external services.

Design considerations. A standalone language model can hallucinate vulnerability types, misclassify severity, and produce inconsistent results across repeated analyses [14]. Retrieval-augmented generation grounds reasoning in external knowledge that evolves independently of model parameters and is auditable, while preserving the privacy boundary because both the embedding model and the index run locally [55, 56].

Design rationale. Retrieval is a configurable option, not a mandatory step. Pilot runs during development showed that retrieval behaviour varies by model family — it lifts some configurations and degrades others — so the design exposes RAG as a per-model knob rather than baking it into the pipeline. The privacy contract is enforced by the component boundary itself: the embedding step and the vector search reach only loopback addresses, and the optional refresh path fetches public metadata only — it never sends a snippet, an embedding, or a finding to an external service. This separation between code-bearing data paths and public-metadata paths is the load-bearing element for R5.

3.2.3. Vulnerability Detection

The vulnerability-detection component is the load-bearing element for R1 (accuracy) and R2 (consistency). It accepts a code snippet plus optional retrieved context, prompts the local model under a strict JSON-mode contract, runs multiple seeded passes, and emits each finding as an inter-run-confidence-scored, snippet-line-relative issue record.

Processing approach. Detection runs at `temperature=0` with JSON-mode decoding (`format='json'`) so the sampler emits only schema-conformant JSON, eliminating parse failures from malformed responses. The prompt is then run multiple times with different seeds (the runtime defaults to two passes; the evaluation harness uses three). Findings that match across passes (by canonical type and line range) accumulate inter-pass agreement; an opt-in confidence gate suppresses single-pass findings before stage-2 repair generation. If the gate would empty the result set, the system falls back to the first pass to avoid silent suppression.

Design rationale. Detection is split from repair generation deliberately. Coupling both tasks into a single prompt biases the model toward producing a plausible fix rather than accurately classifying the issue (Section 3.1). Separating them lets the detection prompt focus on the single optimisation target of accurate, reproducible classification, which improves R1 directly and makes R2 measurable. Promoting agreement from a binary keep-or-drop decision to an explicit per-finding confidence value is the second design choice: it converts the precision/recall trade-off from a fixed design constant into a developer-configurable knob.

3.2.4. Repair Generation and IDE Integration

The repair-generation component takes each consensus-filtered finding and produces a single targeted patch under a strict `{code, language}` schema. Each patch is exposed through the IDE-integration surface as a VS Code Quick Fix that the developer can preview, apply, or discard; applying a patch always integrates with the editor undo stack.

Design considerations. Repair suggestions are decision support, not autonomous edits. The hard constraint is that the developer keeps control: no patch is ever applied without explicit accept (R3). The structural `{code, language}` contract (rather than free-text guidance) is what makes the patch machine-applicable; the IDE Quick Fix surface is what makes the application reviewable.

Design rationale. Stage-2 runs only on findings that survive the consensus filter, which keeps repair cost proportional to the load-bearing detections. Each Quick Fix carries the original finding metadata so the developer can see why the patch is suggested before applying it. The IDE triggers are deliberately diverse (debounced edits, manual commands, workspace scans) so the pipeline fits both flow-state coding and explicit audit work, addressing R4 across distinct usage modes.

3.2.5. Supporting Subsystems

Four supporting subsystems round out the component set: the model manager, the analysis cache, the workspace dashboard, and the vulnerability-data manager.

Roles. The *model manager* owns runtime model selection and connection management with the local Ollama backend, including health checks and graceful fallback. The *analysis cache* is an LRU+TTL store keyed by snippet hash that lets repeated analyses return instantly without re-invoking the model, which is critical for the debounced inline path (R4). The *workspace dashboard* aggregates per-file findings across a workspace scan into an issue-density view with severity heuristics, intended for prioritisation rather than as a formal risk score. The *vulnerability-data manager* caches public CVE / OWASP / CWE metadata locally with periodic refresh; it is the only component that touches the network in non-loopback mode, and the egress gate forces every fetch through an explicit allow-list.

Design rationale. Each subsystem is independently testable and replaceable, which keeps the privacy boundary checkable in isolation (R5): a regression in any single subsystem cannot silently widen the egress surface. The cache is what makes interactive latency tolerable on repeated edits; the dashboard is what makes the workspace-scan workflow useful as a triage tool; the model manager is what lets the same pipeline serve very different model profiles (small-and-fast vs. large-and-accurate) without code changes.

3.2.6. Component Interfaces and Data Flow

Table 3.2 summarises the input/output contract of each component. By making these contracts explicit, the architecture supports component-level analysis, isolated improvement, and substitution of individual stages without affecting downstream behaviour. The next section orchestrates these components into the four detection workflows that the developer actually interacts with.

3.3. Detection Workflows

While the component architecture defines what responsibilities exist in the system, IDE usability depends on how these components are orchestrated in concrete workflows. Code Guardian supports five workflows that trade off latency, context breadth, and output surface. Table 3.3 summarises them; the subsections that follow describe each in detail and relate them to requirements R1–R5. The process logic shared across the four detection workflows is illustrated by the BPMN in Section 3.4.3; differences are confined to the trigger and the output surface.

Table 3.2.: Code Guardian component interfaces and data flow. Reading top to bottom traces a request from the cursor position to the rendered diagnostic and Quick Fix.

Component	**Input**	**Output**
Context Extraction	Document URI + cursor position (or selection / file / workspace scope)	Code snippet + line-offset metadata + scope kind
Knowledge Retrieval (RAG)	Code snippet	Top-k retrieved CWE / OWASP / CVE entries with similarity scores (or empty if RAG disabled)
Vulnerability Detection	Code snippet + optional retrieved context	JSON array of findings: type, severity, snippet-relative line range, description, inter-run confidence
Repair Generation	Single confirmed finding + originating snippet	Structured `{code, language}` patch keyed to the finding
IDE Integration	Findings + patches + line-offset metadata	VS Code diagnostics, Problems-panel entries, Quick Fix actions, dashboard tiles

Table 3.3.: Workflow modes supported by Code Guardian. The four detection workflows share the same backend pipeline (extract scope → optional RAG → Stage 1 LLM → parse → render); they differ only in trigger and output surface. Stage 2 repair is invoked per finding from any of the four detection workflows.

Workflow	**Trigger**	**Scope**	**Output surface**
Real-time function diagnostics (§3.3.1)	`onDidChangeText` (debounced 800 ms)	Enclosing function (≤2 000 chars)	VS Code diagnostics, Problems panel, quick fixes
On-demand file diagnostics (§3.3.2)	Command palette	Active document (≤20 000 chars)	VS Code diagnostics, Problems panel
Interactive analysis & Q&A (§3.3.3)	Command palette, selection action	Selection / files & folders	WebView panel (streamed Markdown)
Workspace scan (§3.3.4)	Command palette	`*.js/.ts/.tsx/.jsx` (≤500 KB / file)	Dashboard WebView, Problems panel
Stage 2 repair (§3.3.5)	Per Stage 1 finding	Single finding	VS Code quick fix (developer-applied)

3.3.1. Real-Time Function-Level Diagnostics

The real-time workflow provides continuous feedback while a developer edits JavaScript/TypeScript code. The key goal is to deliver timely, IDE-native warnings without disrupting flow (R4).

Trigger. The extension listens to document change events for JavaScript and TypeScript documents. Analysis is *debounced* (default: 800 ms, following common VS Code extension practice for balancing responsiveness against unnecessary re-analysis) so the model is not invoked on every keystroke.

Scope and guards. When the debounce fires, Code Guardian extracts the innermost enclosing function at the cursor position and analyzes only that snippet. A size guard skips unusually large functions (default: 2 000 characters, chosen empirically to keep prompt size within the range where local models produce reliable structured output) to bound worst-case latency and avoid overloading the local model.

Structured output for diagnostics. The analyzer is prompted to return a strict JSON array of issues. Each issue contains a message and a line range. Repair suggestions are generated separately in Stage 2 and attached before rendering. The diagnostic adapter maps the snippet-relative, 1-based line numbers to VS Code ranges using the extractor's line offset and clamps ranges to valid document bounds. This enables stable rendering in the Problems panel, editor squiggles, and hover tooltips.

Trade-offs. Function-level analysis improves responsiveness, but it can miss vulnerabilities whose evidence lies outside the current scope (e.g., validation in a different module). This limitation motivates the on-demand and workspace workflows and is addressed further in the future-work chapter.

3.3.2. On-Demand File Diagnostics

The file workflow provides broader coverage when a developer explicitly requests a deeper scan.

Trigger. A command palette action runs analysis over the full active document.

Scope guard. To avoid generating excessively large prompts, the prototype skips very large files (default: 20,000 characters) and warns the user instead.

Output. Findings are surfaced through the same structured diagnostics pipeline as the real-time workflow. This keeps the UI consistent, and it ensures that file scans can be reviewed in the Problems panel and navigated using standard IDE tooling.

3.3.3. Interactive Analysis and Follow-up Q&A

Some security questions require richer explanations than a single diagnostic message. Code Guardian therefore includes webview-based interaction modes.

Selection analysis view. The user can analyze a selected region (or the current line) and inspect the response in a dedicated analysis panel. This mode supports follow-up

questions and streams Markdown-formatted responses for readability. Because it is user-initiated and not continuously triggered, it can tolerate higher latency than real-time diagnostics.

Contextual Q&A view. The contextual Q&A panel allows the user to select files and folders as context and ask security questions about that subset of the workspace. The extension reads the selected files locally and sends their contents only to the local LLM backend, preserving the no-exfiltration objective for source code.

RAG usage. When enabled, the RAG manager can enrich prompts for these interactive workflows by injecting retrieved security knowledge snippets (CWE/OWASP/CVE-derived guidance). Retrieval and embeddings are performed locally; optional knowledge refresh operations fetch only public metadata.

3.3.4. Workspace Scan and Security Dashboard

The workspace workflow supports periodic audits and prioritization across a repository.

File discovery and bounds. The scanner enumerates `*.js`, `*.jsx`, `*.ts`, and `*.tsx` files in the workspace while excluding dependency folders such as `node_modules`. Very large files are skipped (default: >500 KB) to keep runtime bounded.

Aggregation and scoring. Scan results are aggregated into a dashboard WebView. In addition to listing per-file issues, the dashboard computes a coarse security score based on issue density (issues per KLOC) and a keyword-based severity heuristic. This score is intended for prioritization rather than as a formal risk metric.

Developer interaction. The dashboard supports opening affected files directly from the report and rerunning scans. This workflow complements real-time diagnostics by helping developers understand which parts of the codebase concentrate the most findings.

3.3.5. Repair Suggestion Workflow

After consensus-filtered detection (Stage 1), the system generates targeted repairs for each confirmed vulnerability using a dedicated repair prompt (Stage 2). Repairs are exposed as VS Code quick fixes. Applying repairs is always user-initiated and integrates with the undo stack. This preserves developer control (R3) and reduces the risk of unintended behavioral changes from automatically applied patches.

3.3.6. Confidence Abstention and Hybrid Gating

Two design decisions complement consensus filtering to address the false-positive problem identified in the analysis chapter (R1) without re-introducing the deterministic-rule rigidity that motivated using LLMs in the first place.

Confidence as a first-class signal. Multi-pass consensus is itself an empirical signal. If two seeded passes report the same finding, the model agrees with itself; if only one pass reports it, the finding is noisier. The design promotes that signal from a binary keep-or-drop decision to an explicit per-finding confidence value, computable as the fraction of passes that produced a matching finding. A configurable threshold then decides which findings reach the diagnostic stream. The default behavior is no gating. Operators who care more about precision than recall can lift the threshold to suppress single-pass noise, which makes the precision/recall trade-off a configuration choice rather than a design constant.

Hybrid SAST agreement as a confirmation signal. A second source of corroboration is the established SAST tooling. When a deterministic baseline (Semgrep) flags a finding on overlapping lines and a matching canonical category, the LLM finding is promoted to maximum confidence and labeled as hybrid-source. The hybrid label is reported back to the IDE so the developer can see which findings have independent corroboration. This re-uses Semgrep's strengths (low false-positive rate on known patterns) without adopting its weaknesses (very narrow coverage), and it does so transparently — the LLM's own findings remain visible at their original confidence even when Semgrep is silent.

Both mechanisms are conservative: they extend the existing pipeline with optional gates rather than replacing the un-gated path. The chapter's evaluation reports both with-gate and without-gate metrics so the contribution of each gate is attributable.

3.4. System Architecture and Design

This section presents the high-level architecture of Code Guardian using the C4 model, which structures architectural documentation into hierarchical levels: *context* (the system boundary, the developer, and any external actors), *containers* (the major internal components and their interactions), and *process* (the dynamic execution flow that ties the components together). These three views provide a comprehensive understanding of how the conceptual design translates into a deployable architecture and make the privacy boundary explicit at every level: code stays on-device, local inference is performed through a localhost LLM backend, and retrieval (when enabled) is backed by a local knowledge base and vector index. A fourth subsection adds concrete sequence diagrams for the dominant interaction patterns.

3.4.1. Context View: System Boundary and External Interactions

Figure 3.1 positions Code Guardian within its operational environment. The primary actor is the developer working inside Visual Studio Code. The system executes in the VS Code extension host and communicates with a local LLM backend (Ollama) running on the same machine [12].

Local assets. The core local assets are:

- **Workspace source code** (JavaScript/TypeScript) being edited.
- **Local LLM runtime** (Ollama) used for analysis and (optionally) embeddings.
- **Local security knowledge base** and **vector index** used for retrieval when RAG is enabled [75, 68].
- **Local caches** for analysis results and vulnerability metadata.

Optional external data. Code Guardian may optionally fetch *public vulnerability metadata* to refresh the knowledge base (e.g., CVE records from the NVD API). This traffic does not include user source code and can be disabled by running in offline mode. After a refresh, cached knowledge can be reused without network access. This separation preserves the core privacy goal (no source-code exfiltration, R5) while still allowing the knowledge base to evolve over time.

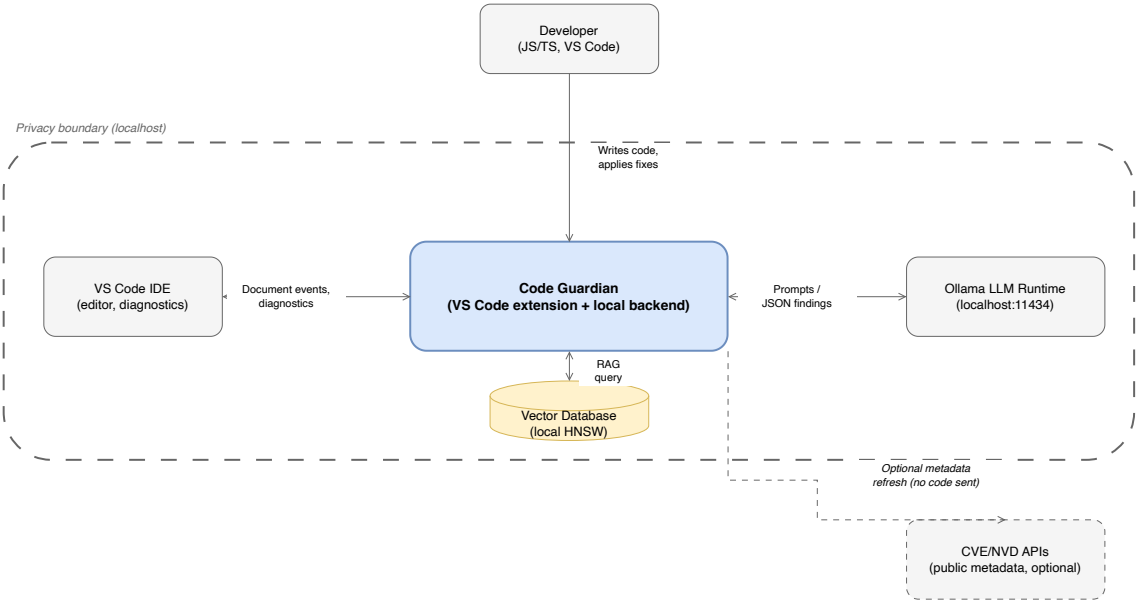

Figure 3.1.: C4 Context diagram showing Code Guardian within its operational environment. The system communicates with VS Code (IDE platform), Ollama (local LLM on localhost:11434), and a local vector database, all within the privacy boundary (green dashed). CVE/NVD APIs (gold, dashed) are the only optional external connection and do not receive source code.

3.4.2. Container View: Internal Component Structure

Figure 3.2 summarizes the main internal containers and their responsibilities.

VS Code extension host. The extension is responsible for registering editor triggers, extracting analysis scopes, and rendering results using VS Code diagnostics and WebViews [76]. It provides commands for file analysis, selection analysis, contextual Q&A, model selection, cache inspection, and workspace scanning.

Structured diagnostics pipeline. For real-time and file-level diagnostics, the extension invokes a JSON-only analyzer that returns a list of issues with line ranges. Repair suggestions are generated in a separate stage and attached before rendering. Results are mapped into VS Code diagnostics and quick fixes. A bounded analysis cache reduces redundant LLM calls for unchanged snippets.

Interactive analysis pipeline. For selection analysis and contextual Q&A, the extension opens a WebView panel and streams Markdown-formatted responses from the local model. This mode supports conversational follow-up and can incorporate retrieved knowledge when RAG is enabled.

RAG manager and data manager. When enabled, the RAG manager maintains a local knowledge base (serialized entries) and a persistent vector store. A vulnerability data manager refreshes public metadata (CWE-/OWASP-aligned curated entries and optional CVE/advisory sources) and caches results on disk. The vector store is rebuilt or updated based on the knowledge base content.

3.4.3. Process View: End-to-End Workflows

Figure 3.3 illustrates the dynamic execution flow that connects the components. The four detection workflows (real-time, on-demand file, interactive Q&A, workspace scan) share the same backend pipeline; they differ only in the trigger and the output surface. The common pipeline proceeds through seven stages.

Trigger. Inline detection listens to `onDidChangeText` document events; on-demand workflows are dispatched from the command palette or a selection action; the workspace scan iterates over filtered `*.js / *.ts / *.tsx / *.jsx` files. The trigger determines only *when* the pipeline runs, not *how*.

Debouncing. The inline workflow inserts an 800 ms debounce timer between the trigger and the rest of the pipeline so continuous typing collapses into a single analysis. Other workflows skip this stage because they are explicitly user-initiated.

Scope extraction. Context extraction selects the analysis unit: the innermost enclosing function for inline detection, an explicit user selection, the active document, or each enumerated file in the workspace. A size guard skips snippets that exceed the model's reliable prompt range.

Cache lookup. The pipeline consults the analysis cache keyed by snippet hash. A hit short-circuits straight to diagnostic rendering, which is what makes repeated edits cheap on the inline path.

Retrieval (optional). When RAG is enabled and the cache misses, the retrieval component embeds the snippet locally, queries the local HNSW index, and injects the top-k matching CWE / OWASP / CVE entries as additional prompt context. When RAG is disabled, this stage is skipped.

Detection passes and consensus. The analyzer invokes the local model under JSON-mode decoding with a fixed schema, repeats the call with different seeds, and aggregates the per-pass findings into consensus-scored issues. An opt-in confidence gate suppresses low-agreement findings before stage-2.

Repair generation (on demand per finding). Each surviving finding triggers a separate stage-2 prompt that returns one structured `{code, language}` patch. Stage-2 runs only on consensus-filtered findings, which keeps repair cost proportional to load-bearing detections.

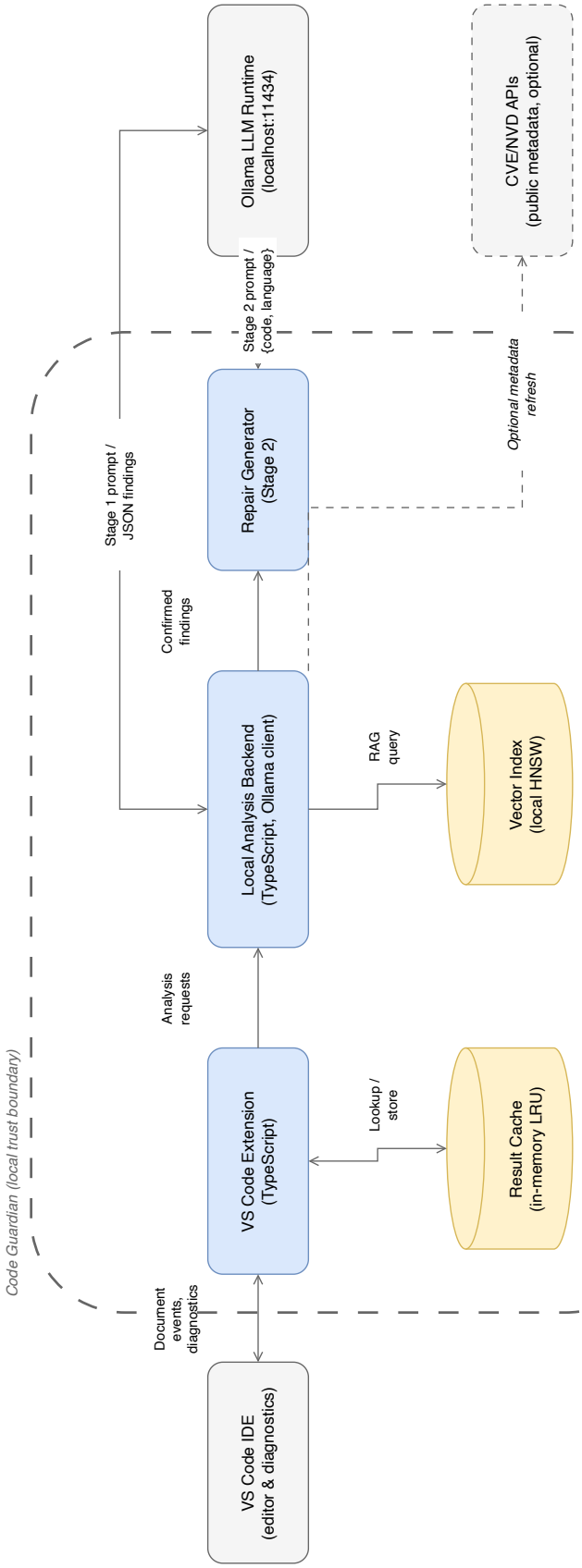

Figure 3.2.: C4 Container diagram showing internal components. The VS Code Extension handles triggers, caching, and UI rendering. The Local Analysis Backend performs LLM inference with JSON-mode and multi-pass consensus filtering, RAG orchestration, and Stage 2 repair generation. Vector Database and Result Cache are local data stores. All source code stays within the system boundary.

Diagnostic rendering. The pipeline maps snippet-relative line ranges back to document ranges, writes the result to the analysis cache, and surfaces findings through VS Code diagnostics, the Problems panel, and Quick Fix actions. The workspace-scan workflow additionally aggregates per-file findings into the dashboard view.

The audit-mode hybrid (described in Chapter 4) extends the on-demand path with a deterministic AST sink scanner that runs in parallel with the LLM stage and unions the findings. Across all workflows source code is sent only to localhost; the optional metadata-refresh path fetches public CVE / CWE / OWASP records without source content and can be disabled for fully offline operation.

3.4.4. Sequence Diagrams: Concrete Detection Flows

Three sequence diagrams capture the dominant interaction patterns. Inline mode with a cache hit (Figure 3.4) returns previously-stored findings without invoking the model, which is the cheap path that absorbs most repeated keystrokes. Audit mode with RAG (Figure 3.5) embeds the snippet, performs vector search (top-$k = 3$ at runtime, $k = 5$ in the evaluated setting), injects retrieved CWE/OWASP chunks, runs two detection passes with seeds 42 and 137 plus consensus filtering (runtime default; the headline evaluation configuration uses three passes), and finally generates repairs in a separate Stage 2 pass. Real-time detection (Figure 3.6) layers an 800 ms debounce timer over the inline path so continuous typing collapses into a single analysis.

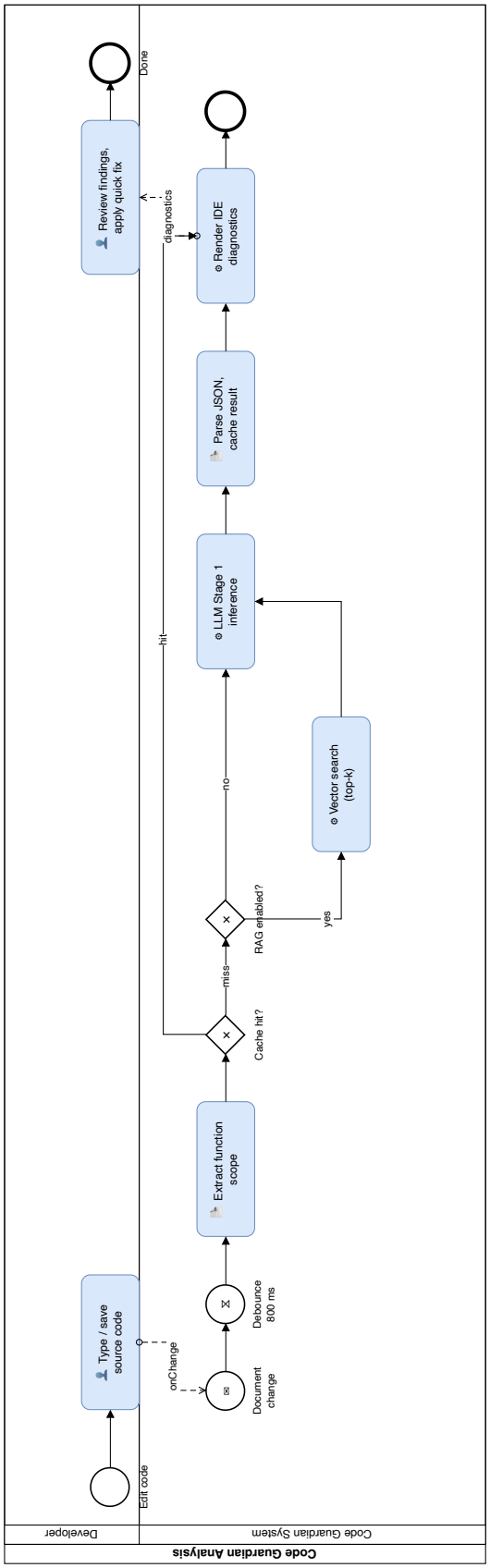

Figure 3.3.: BPMN process view of Code Guardian's inline analysis workflow with two swim lanes (Developer, Code Guardian System). The workflow starts when the developer types or saves source code, sending an `onChange` message into the system. After an 800 ms debounce timer the system extracts the function scope and reaches an exclusive cache-hit gateway: a hit short-circuits straight to rendering, a miss reaches a second gateway for the optional RAG branch (vector search and prompt augmentation when enabled). LLM Stage 1 inference, JSON parsing with cache write, and IDE diagnostic rendering follow, after which a message flow returns the findings to the developer for review and quick-fix application.

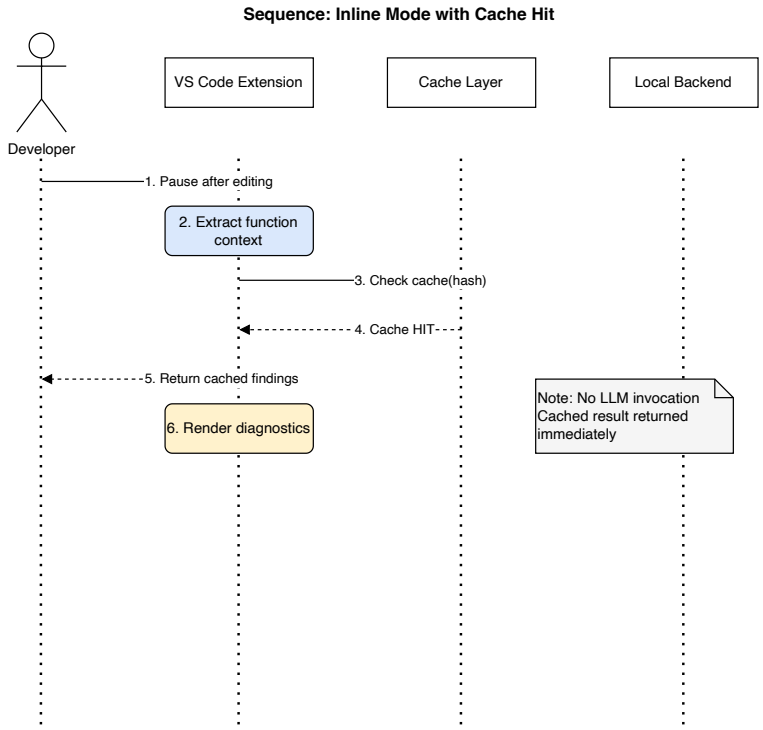

Figure 3.4.: Inline mode with cache hit: no LLM invocation when a previously analyzed function reappears.

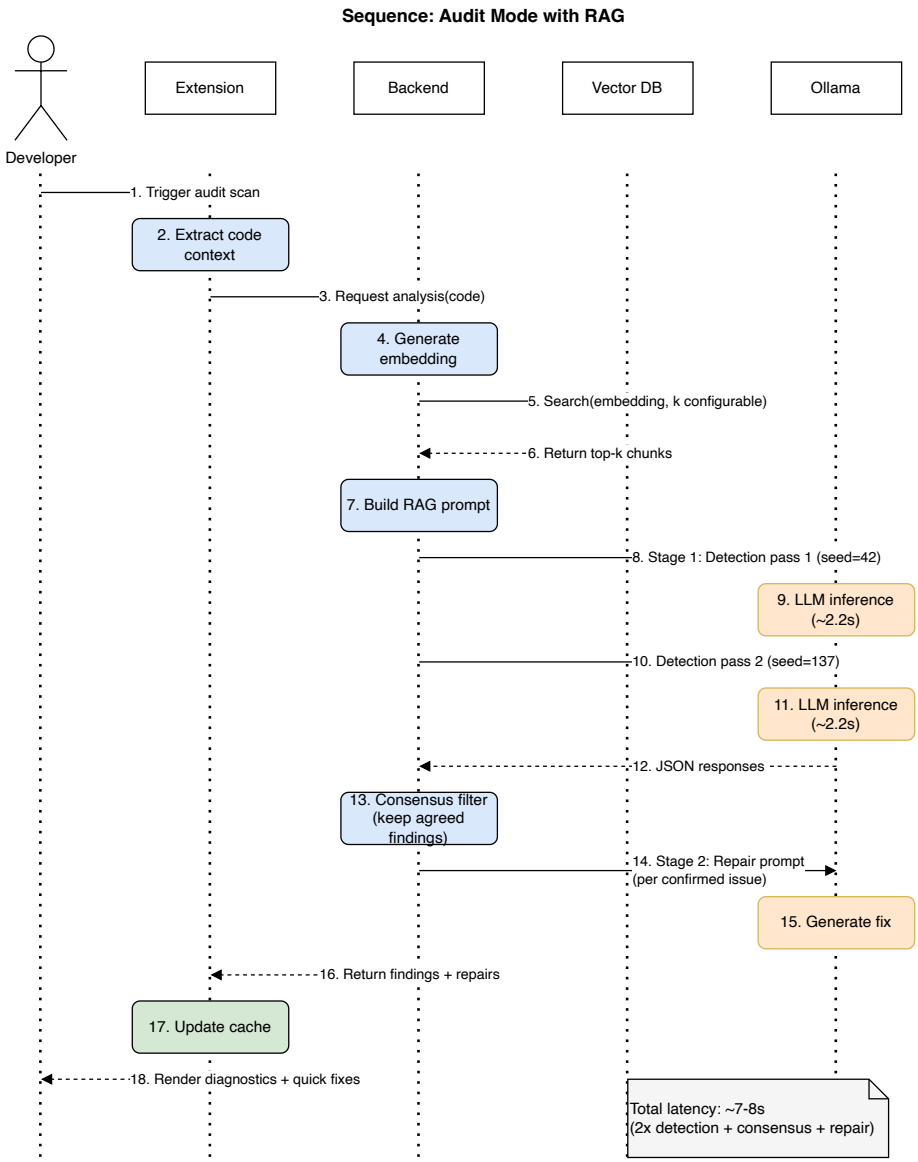

Figure 3.5.: Audit mode with RAG: vector search, two-pass consensus detection (runtime default; the eval headline uses three passes), then Stage 2 repair generation.

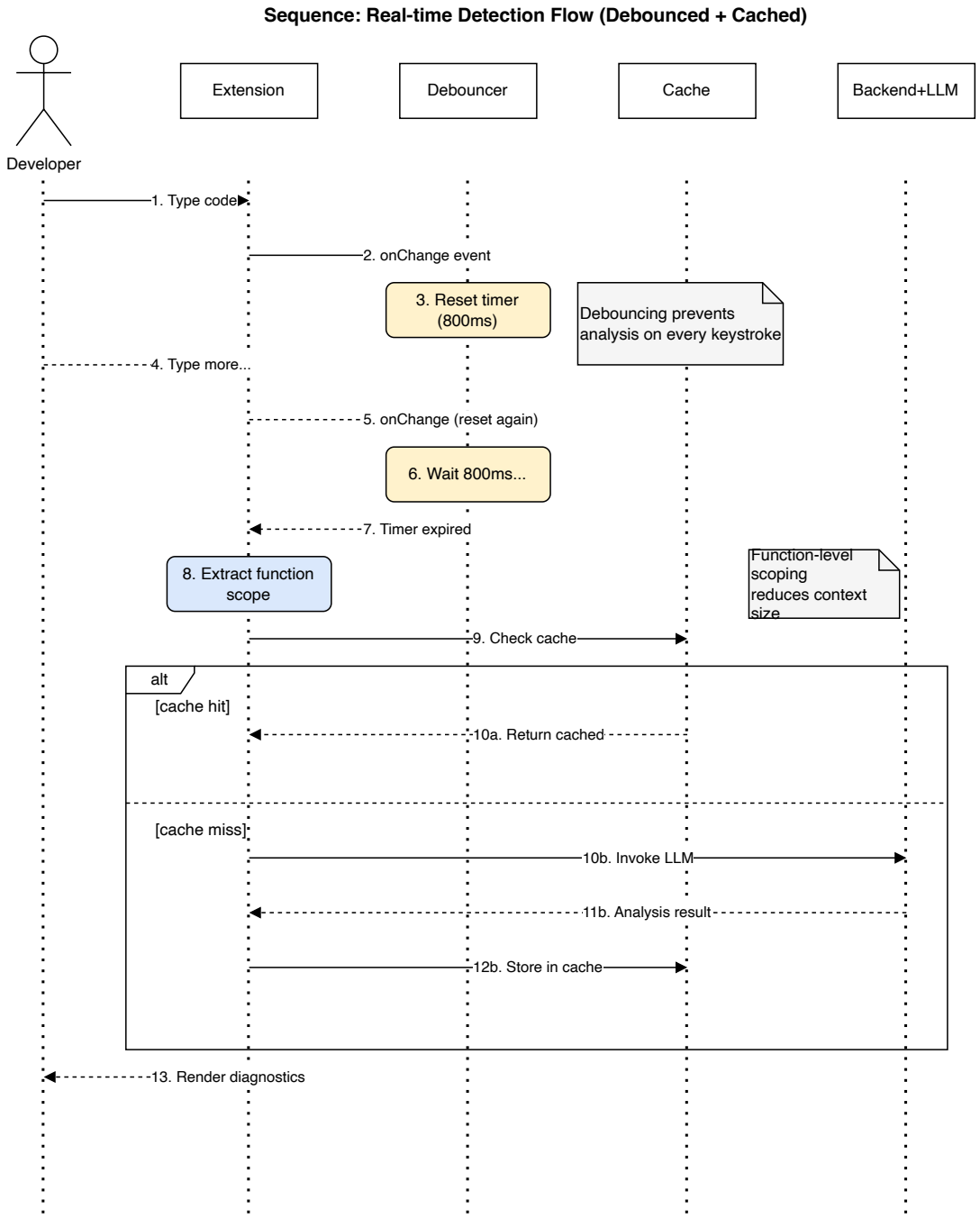

Figure 3.6.: Real-time detection with debouncing: an 800 ms timer prevents redundant LLM invocations during continuous typing.

3.5. Summary

This chapter translated the R1–R5 requirements into a concrete system concept for Code Guardian. The concept derivation walked through six architectural decisions and recorded the alternatives that were rejected for each: cloud versus local inference, RAG versus pure LLM versus fine-tuning, function versus file versus whole-project scope, single combined versus two-stage detect-then-repair prompts, free-text versus strict JSON output, and binary versus confidence-gated multi-pass consensus. The selected decisions compose into five components — context extraction, knowledge retrieval, vulnerability detection, repair generation with IDE integration, and supporting subsystems — whose input / output contracts were made explicit in a single interfaces table. Four detection workflows orchestrate these components onto distinct developer surfaces (real-time inline diagnostics, on-demand file scans, interactive Q&A, workspace audit), with stage-2 repair invoked per finding. The system architecture was specified through C4-style context, container, and process views, plus sequence diagrams for the three dominant interaction patterns. The privacy boundary — source code on-device, public metadata refresh as the only outbound channel — is enforced at every level of decomposition. Chapter 4 translates this concept into a deployable VS Code extension, and Chapter 5 measures the resulting system against the R1–R5 thresholds defined in Chapter 2.

4. Implementation

This chapter describes how the conceptual design from Chapter 3 is realised as a deployable VS Code extension. The implementation is a TypeScript codebase that runs inside the VS Code extension host, communicates with a local Ollama backend over loopback, and ships as a packaged `.vsix` installable on macOS, Linux, and Windows. The chapter describes the engineering decisions that move the design from concept to running software, with particular attention to where the design's correctness depends on specific implementation choices.

Three engineering pressures shape the implementation. First, the privacy boundary (R5) is only as strong as the weakest network call: a single misrouted request to a non-loopback endpoint silently undermines the contract, so the implementation needs a runtime mechanism that makes the boundary checkable rather than asserted. Second, interactive latency (R4) cannot be retrofitted: every component on the inline path must respect the debounce, the cache, and the size guards from the start, and any blocking operation must complete inside the budget the IDE diagnostics surface tolerates. Third, output reliability (R2) and repair safety (R3) require defensive parsing on every model response: the codebase must treat malformed JSON, partial outputs, and unparseable repairs as expected events rather than exceptions.

The chapter addresses these pressures by separating the runtime substrate from the analysis pipeline and by isolating each privacy-relevant concern in its own component. The technology stack (Section 4.1) fixes the locality envelope and the deterministic-decoding contract; the system workflow (Section 4.2) operationalises detection and repair as a two-stage flow with explicit latency gates and an audit-mode AST hybrid for whole-project scans; the core components (Section 4.3) compose that pipeline into the editor's event loop; the user-interface layer (Section 4.4) maps internal findings onto VS Code diagnostics, Quick Fix actions, the workspace dashboard, and the contextual Q&A WebView; and the safety / privacy layer (Section 4.5) ties the runtime egress gate, the signed knowledge corpus, and the syntactic auto-applicability bar back to the threat model named in Chapter 2.

The remainder of this chapter is organised as follows. Section 4.1 describes the technology stack and the rationale behind each choice (TypeScript, VS Code Extension API, Ollama, HNSWlib, LangChain, etc.). Section 4.2 presents the detection and repair pipeline as it executes at runtime, including the AST+LLM audit hybrid. Section 4.3 describes the implementation of each conceptual component named in Section 3.2: context extraction, analyzer, RAG manager, analysis cache, model manager, and workspace scanner. Section 4.4 covers the user-interface surfaces. Section 4.5 concludes with the privacy-enforcement mechanisms.

Empirical validation of these implementation choices is deferred to Chapter 5, which measures detection accuracy, consistency, repair quality, latency, and privacy guarantees against the requirements defined in Chapter 2.

4.1. Technology Stack and Rationale

Code Guardian is implemented as a Visual Studio Code extension that performs security analysis and repair suggestion *locally* on the developer machine. The design goal is to integrate vulnerability detection into the IDE workflow while maintaining a strict *no source-code exfiltration* property (R5) and practical responsiveness for interactive use (R4).

VS Code Extension (TypeScript)

The extension is implemented in **TypeScript** using the VS Code Extension API [76]. Core responsibilities include registering editor events (on-change and on-command triggers), mapping analysis findings to VS Code diagnostics, and exposing quick fixes through code actions. The extension also provides two WebView-based interfaces: an interactive analysis view for selected code and a workspace security dashboard that aggregates findings across the project. Where IDE integration benefits from standardized editor tooling, the Language Server Protocol provides a relevant reference point for diagnostics-style interactions in modern IDEs [77].

Local LLM Inference via Ollama

All model inference is performed through **Ollama** running on `localhost:11434` [12]. Code Guardian supports multiple local model families (e.g., `qwen3`, `gemma3`, `CodeLlama`) and allows switching models at runtime through settings. Requests are configured for low-temperature decoding to reduce randomness and improve schema adherence, and retry logic with exponential backoff is applied to handle transient failures (e.g., model warm-up, timeouts).

Retrieval-Augmented Generation (RAG)

Optional RAG is implemented with **LangChain** and a local vector index [75]. Security knowledge entries (CWE patterns, OWASP Top 10 guidance, selected CVE summaries, and JavaScript/TypeScript secure coding notes) are embedded using a lightweight local embedding model (`nomic-embed-text` via Ollama) and stored in an **HNSW** vector store. At analysis time, relevant knowledge snippets are retrieved

and injected into the prompt to improve grounding of the model's reasoning and support R1–R4.

The `nomic-embed-text` model was selected for three reasons: (1) native availability via Ollama, ensuring the same local deployment model as the LLM runtime; (2) strong performance on code and technical text retrieval tasks relative to its size (768-dimensional embeddings at 137M parameters); and (3) permissive licensing for local deployment. Alternative embedding models such as `all-MiniLM-L6-v2` or `bge-small-en` were considered but required separate runtime infrastructure or offered inferior code-domain performance in preliminary testing.

Dynamic Vulnerability Knowledge Updates

The knowledge base is designed to be refreshable. Code Guardian supports updating OWASP and CVE information and persists cached data on disk. Importantly, only public vulnerability metadata is fetched; source code and extracted context remain local.

Engineering Selection Criteria

Technology choices were guided by four practical criteria:

- **Local deployability:** all core components must run on a developer workstation without managed cloud dependencies.
- **Operational transparency:** outputs and failure modes must be inspectable for debugging and thesis reproducibility.
- **Incremental integration:** the solution should align with VS Code-native diagnostics and command workflows instead of requiring a separate toolchain.
- **Replaceability:** model backends and retrieval components should be swappable without redesigning the full extension.

These criteria explain the concrete stack decisions: Ollama for local model serving, TypeScript for extension maintainability and VS Code API fit, and local vector retrieval for optional grounding. Together they form a modular but operationally coherent baseline for privacy-preserving IDE security assistance.

Shared TS / JS Modules

The system uses four small modules that are kept in sync between the extension runtime (TypeScript) and the evaluation harness (Node.js) by sharing a single source-of-truth JSON catalog. Each module is covered by a deterministic snapshot test that asserts twin agreement on a fixed fixture set.

- **Canonical category taxonomy** (`src/categoryTaxonomy.json`, `src/categoryTaxonomy.ts`, `evaluation/category-taxonomy.js`). A versioned list of 26 canonical vulnerability classes plus an ordered set of regex patterns mapping raw model-emitted labels to canonical IDs. Used by both the extension and the harness for consistent type-level scoring.
- **SAST fusion logic** (`src/sastBridge.ts`, `evaluation/sast-fusion.js`). Pure function `fuseSastWithLlm` that promotes LLM findings agreed by Semgrep on line range and canonical category to confidence 1.0 and source `hybrid`. Spawning Semgrep from the extension is deferred; the harness already drives Semgrep for the SAST baseline so the fusion path reuses the same workspace.
- **Import / sink resolver** (`src/importResolver.ts`, `evaluation/import-resolver.js`). Lightweight regex-based scanner that surfaces imports, validator-package presence (`sanitize-html`, `joi`, `zod`, etc.), and category-tagged sink occurrences (e.g., `eval`, `child_process.exec`, `crypto.createHash('md5')`, `jsonwebtoken.verify`). Output is appended to the user prompt behind an opt-in flag.
- **Repair syntax validator** (`src/repairValidator.ts`, `evaluation/repair-validator.js`). Uses `@babel/parser` with TypeScript, JSX, and async plugins to check whether a generated repair string is syntactically applicable JavaScript or TypeScript. Markdown fences are stripped before parsing; obvious prose openings are rejected before the parser is invoked.

These additions keep the privacy boundary intact: every module operates on data that is already on the developer's machine. The validator and resolver depend only on `@babel/parser`, which itself runs locally and ships with the extension bundle.

4.2. System Workflow

This section explains how Code Guardian turns JavaScript/TypeScript code into vulnerability findings and repair suggestions, implementing the design from Chapter 3 against the requirements in Chapter 2 (R3, R4, R5).

4.2.1. End-to-End Flow

Two execution paths matter in the prototype. The *structured diagnostics* path emits JSON findings for editor diagnostics, and the *interactive analysis* path emits Markdown explanations in a WebView with follow-up Q&A. A run is triggered automatically while editing (debounced) or explicitly via a command (analyze selection, analyze file, scan workspace); the chosen scope decides how much context is included.

The extension extracts the relevant code fragment and its location. For real-time diagnostics, the default unit is the enclosing function found by depth-first AST traversal via the TypeScript language service, with a 2 000-character size guard. For selection analysis the unit is the selected region; for file analysis it is the entire document. If RAG is enabled and initialized, the system retrieves security-knowledge snippets through a local embedding model and HNSW index and appends the guidance to the prompt. The real-time diagnostics path is kept lightweight to preserve responsiveness; RAG is primarily used in the interactive WebView and the batch-scan workflows.

The local Ollama model is then invoked with output constraints that match the workflow. Structured diagnostics request a JSON array of issues with defensive parsing (strip Markdown fences, extract the first JSON array substring, validate fields) and degrade safely to empty findings on parse failure. Interactive analysis streams Markdown to the WebView. Findings render as VS Code diagnostics (squiggles, Problems panel, hover tooltips) with optional Quick Fix actions. Results are cached by (snippet, model, mode) under LRU + TTL so repeated edits do not re-invoke the model.

Although inference is probabilistic, the orchestration is deterministic: for a fixed input snapshot, active model, and prompt mode, scope extraction, prompt-assembly structure, parsing, localization mapping, and diagnostic emission follow a fixed sequence with explicit guards. This isolates model effects during debugging and evaluation. On each trigger the extension extracts and validates scope, computes a content-based cache key, returns immediately on hit, and on miss assembles the prompt (optionally with RAG), calls the model, validates the JSON response, maps snippet-relative line numbers to document positions, caches the result, and renders.

4.2.2. Debouncing and Workflow Modes

A debouncer suppresses analysis on every keystroke: each document-change event resets a timer, and analysis runs only after an 800 ms pause. The real-time workflow uses function-level scope plus the 2 000-character guard (∼400–500 tokens) to keep inline feedback within acceptable bounds on consumer hardware. On-demand selection or file analysis applies a 20 000-character file guard (∼4 000–5 000 tokens)

so prompts fit the effective context window of smaller local models (8K–16K) with headroom for RAG context and response generation. The workspace scanner iterates JS/TS files (excluding `node_modules`) and aggregates findings into a dashboard; files above 500 KB are skipped so memory use stays bounded across large monorepos.

Stage-2 repair generation is also gated on scope. Function-scope inline runs detect findings but skip Stage 2 so an extra Ollama call per issue does not land on every debounced re-analysis; the IDE code-action provider instead invokes `generateRepair` lazily when the developer opens the lightbulb on a specific diagnostic. The explicit scopes (selection, file, workspace scan, audit) eagerly run Stage 2 so the dashboard and quick-fix surfaces carry `suggestedFix` and `autoApplicable` ready for one-click apply.

4.2.3. Pipeline Gates

Four configurable gates sit between the consensus filter and diagnostic emission, each toggleable through extension settings.

Confidence assignment stamps every stage-1 finding with a confidence score and a `detectionSource` provenance tag. Both-pass agreement is 1.0. The single-pass fallback (which would otherwise drop findings silently) keeps them at 0.3 so they remain visible to downstream gates.

The opt-in *confidence gate* applies a threshold in $[0, 1]$ before stage-2 (default 0, no gating). At the runtime default of two passes the only reachable agreement values are 0.5 (one of two) and 1.0 (both passes), so any threshold > 0.5 collapses to requiring full (2/2) agreement. Extending the pass count to three (configurable up to five through `codeGuardian.consensusPasses`) admits intermediate values: a threshold of ≥ 0.67 then admits findings agreed by at least two of three runs — the configuration used in the evaluation chapter's gated headline.

The opt-in *hybrid SAST fusion* runs Semgrep once over the snippet. Any LLM finding overlapping a Semgrep finding on line range plus canonical category is promoted to `detectionSource: 'hybrid'` with confidence 1.0; LLM-only findings keep their inter-run confidence.

Repair syntax validation parses every `suggestedFix` after stage-2 and tags findings with an `autoApplicable` boolean (and a short rejection reason such as `prose_not_code`). The IDE quick-fix surface uses this to decide whether to offer one-click apply or a preview-first action.

4.2.4. System Prompt and Latency Telemetry

The stage-1 system prompt is a tight ∼80-token instruction; JSON-mode decoding plus the response schema enforce output shape, so a verbose response-format example block is unnecessary. Per-call latency is reported as `inferenceTimeMs` plus `parseTimeMs` so any latency change can be attributed unambiguously. Inference dominates the total by more than 99% in measured micro-evaluation; the prompt size is therefore the dominant prefill-token cost lever.

4.2.5. Audit Mode: AST + LLM Hybrid Pipeline for Whole-Project Scans

Inline real-time analysis uses function-level scope to stay inside the 1.5 s latency budget; whole-project audits run under a different scope, with no per-keystroke deadline, so the system can afford a deeper structural analysis. Audit mode is the deployment of the workspace dashboard command (`codeSecurity.workspaceDashboard`). It is controlled by the setting `codeGuardian.enableAuditMode` (default `true`); disabling it restricts the dashboard to LLM-only function-scoped analysis without the AST stage.

The pipeline runs in two stages, deterministic before semantic.

Stage 0 — AST project map. `src/projectMapBuilder.ts` walks the workspace using Node `fs` (skipping `node_modules`, `cypress`, `tests`, etc.) and parses every `.js`/`.ts`/`.jsx`/`.tsx` file with the TypeScript compiler API. For each file the visitor records: imports (`require()` and `import` declarations), top-level exports, request-input usage (`req.body`, `req.query`, `req.params`), and a curated set of direct sink shapes that map cleanly to the canonical taxonomy:

- `eval(...)`, `Function(...)` — CWE-95 code injection, with the request-input chain reported when the first argument is `req.body.X`;
- `child_process.exec / execSync / spawn` — CWE-78 command injection;
- `crypto.createHash('md5'|'sha1'|'rmd160')` — CWE-327 weak crypto;
- `require(<dynamic>)` — CWE-22 dynamic-path require;
- ObjectLiteralExpression with key `$where` and a TemplateExpression value — CWE-943 NoSQL injection via `$where` interpolation;
- RegularExpressionLiteral matching the nested-quantifier shape `(X+)+ / (X*)+ / (X+)*` — CWE-1333 ReDoS;
- ObjectBindingPattern with ≥ 3 fields whose initialiser is `req.body`, `req.query`, or `req.params` — CWE-915 mass assignment.

Each detection is a `SecurityIssue` object tagged `detectionSource: 'ast'` with confidence 1.0. The whole project map is built in well under a second on the M4 Max profile; the cost is bounded by file count and is independent of model latency.

Stage 1 — per-file LLM analysis. After the AST map is built, the workspace scanner iterates the same file list and analyzes each file with the existing `analyzeCodeWithLLM` pipeline (RAG-augmented when enabled). The whole-file scope requires a larger context window than the inline mode's 2 000-character chunks; the implementation requests `num_ctx: 16384` so a 100–150-line source file plus the system prompt and RAG snippets fit with headroom for output. LLM findings are tagged `detectionSource: 'llm'`.

Stage 2 — union and corroboration. For each file, AST detections (Stage 0) and LLM findings (Stage 1) are unioned by `workspaceScanner.mergeIssues`. Findings on the same line collapse into a single entry tagged `detectionSource: 'hybrid'`, signaling that both detectors agreed. The dashboard renders the three sources distinguishably so the developer can read the detection provenance directly from the diagnostic.

Why the AST is independent of the LLM. The two detectors run in parallel with no shared prompt context: the LLM never sees the AST findings, and the union step in Stage 2 does not require the LLM to corroborate AST detections. This isolation prevents the model from down-weighting categories it perceives as "already covered" and keeps each detector's output an independent signal that the union step can combine. A consequence is that the run-to-run deterministic rate of audit mode equals the LLM rate, since AST findings are byte-identical across runs by construction.

Performance. A targeted 6-file run on OWASP NodeGoat completes in 117.2 s under audit mode versus 1 236.5 s for the inline-scoped 26-file run. The AST stage adds approximately 30 ms per 100 files; per-file LLM cost is dominated by the larger `num_ctx` (16K tokens) which raises per-call latency from a function-chunked stage-1 baseline of 2 216 ms median to 19 s median for whole-file scope. The latency increase is acceptable because audit mode is invoked explicitly, not on every keystroke. Files above 500 KB are skipped to keep memory bounded across large monorepos.

4.3. Core Component Implementation

The subsections below walk through the extension's components in the order data flows through them: context extraction, LLM analysis, diagnostic rendering, retrieval,

caching, and workspace-level scanning.

4.3.1. Context Extraction and Scoping

Code Guardian extracts the smallest useful unit of code for interactive analysis: the enclosing function at the cursor. This reduces prompt size and improves responsiveness without requiring full-program analysis. The extractor parses the active document, locates the innermost function-like node covering the cursor, and returns both the snippet and its 0-based start-line offset within the original document; the offset is later used to map model-reported line numbers back to the full file so diagnostics are placed correctly. If parsing fails or no suitable node is found, the implementation falls back to a conservative line/selection scope, favoring availability over strict completeness in real-time mode. For explicit commands the scope expands to a selected region (or current line) or the full file — broader scopes used for deeper inspection or pre-commit review.

4.3.2. LLM Analyser (Local JSON-Only Output)

The analyzer performs local inference through Ollama and runs the detect-then-repair pipeline as two separate calls, each constrained by its own structured-output schema (Section 4.2). Stage 1 returns an `issues` array with the detection-side fields shown in Listing 4.1; Stage 2 issues a per-finding repair call that returns a structured `{code, language}` object, and the validated repair is attached to the corresponding issue under `suggestedFix` before rendering.

Listing 4.1: Stage 1 detection schema and the post-stage-2 in-IDE issue shape

```
// Stage~1 detection schema (LLM-emitted)
interface DetectionIssue {
  type: string;      // canonical category, e.g. "sql-injection"
  severity: string;  // coarse: "low" | "medium" | "high" | "critical"
  startLine: number; // 1-based, snippet-relative
  endLine: number;   // 1-based
  message: string;
}

// In-IDE issue after Stage~2 repair attachment
interface SecurityIssue extends DetectionIssue {
  confidence?: number;     // [0,1], inter-run agreement
  detectionSource?: 'llm' | 'ast' | 'hybrid';
  suggestedFix?: string;   // attached from Stage~2 {code, language}
  autoApplicable?: boolean; // result of @babel/parser syntax check
}
```

Reliability rests on three mechanisms. The system prompt enforces strict JSON-only output. Defensive parsing applies staged normalization — trim wrapper text, strip

Markdown fences, extract the first array-like substring, then parse and validate field types — and discards invalid entries rather than partially accepting them; hard parse failures produce an empty issue list with a categorized parse error so malformed outputs cannot propagate into editor state. Transient errors (timeouts, temporary unavailability) are handled by bounded retries with exponential backoff that absorbs short-lived contention from model warm-up or concurrent local inference; retries are capped to prevent cascading delays in interactive workflows. Non-recoverable failures (repeated timeouts, model-not-found) surface a user-facing message and return an empty list so the editor stays responsive while preserving explicit control over model configuration.

4.3.3. RAG Manager and Knowledge Updates

When enabled, the RAG manager maintains a local security knowledge base and a persistent HNSW vector index, embedding entries via a local Ollama model (`nomic-embed-text`). Entries are chunked using a recursive text splitter to balance semantic coherence and retrieval recall, and the index is persisted in an extension-managed directory so it can be reused across sessions without rebuilding; initialization is lazy to avoid slowing extension activation when retrieval is not used. For each analysis request, the manager retrieves the top-k snippets and augments the prompt with short vulnerability definitions and mitigation guidance. The vulnerability-data manager can refresh public sources on demand (OWASP Top 10, a curated CWE set, a bounded number of recent CVEs); retrieved metadata is cached on disk, only public information is fetched, and no source code or extracted context leaves the device (R5). A minimal baseline knowledge bundle is used when updates fail so RAG-enabled prompting remains functional offline with reduced coverage.

4.3.4. Analysis Cache

To avoid repeated inference on unchanged snippets, results are cached in a bounded LRU with time-to-live expiration. The cache key is a SHA-256 hash of the trimmed snippet plus the active model name and prompt mode (RAG on/off), so LLM-only and RAG results are stored separately. Entries expire after 30 minutes and the cache holds at most 100 entries; least-recently-used eviction handles overflow. In repetitive editing patterns (undo/redo, small refactors) this removes a substantial share of otherwise-repeated LLM calls.

4.3.5. Workspace Scanner and Dashboard

For project-level visibility, the workspace scanner analyzes JavaScript and TypeScript files in batch and aggregates results into a dashboard WebView. The scanner enumer-

ates by extension, excludes dependency folders (e.g. `node_modules`), and skips very large files to bound runtime; processing is bounded and sequential to avoid saturating local inference capacity and degrading the interactive experience, optimizing for reproducible periodic audits rather than throughput. Because the JSON-only schema does not mandate a severity field, the scanner applies a conservative keyword-based severity heuristic for coarse dashboard prioritization, treated as a presentation aid rather than a ground-truth classifier.

4.4. User Interface

Code Guardian is designed to integrate security feedback into the existing Visual Studio Code workflow rather than introducing a separate application. The interface emphasizes (i) low-friction detection during coding, (ii) actionable remediation via quick fixes, and (iii) workspace-level prioritization.

4.4.1. Inline Diagnostics and Hover Explanations

Detected issues are rendered as standard VS Code diagnostics. This provides familiar affordances: squiggles in the editor, a centralized list in the Problems panel, and hover tooltips that summarize the finding. This presentation supports R4 by making explanations available at the point of code, without requiring context switching.

Real-time behavior. For JavaScript and TypeScript documents, diagnostics can be produced during normal editing via a debounced trigger. To preserve responsiveness, the default real-time scope is the enclosing function at the cursor, and unusually large functions are skipped. Developers can therefore treat Code Guardian feedback similarly to linting warnings: it appears close to the code being written and is automatically updated as the local scope changes.

4.4.2. Quick Fixes for Repair Suggestions

When a finding includes a suggested fix, Code Guardian offers a *Quick Fix* action. Fix application is always user-initiated and confirmed, which preserves developer control and reduces the risk of unintended behavior changes (R5). Fixes integrate with the editor undo stack so developers can revert and iterate.

Developer control. The prototype does not apply patches automatically. Instead, suggested fixes are attached to diagnostics and surfaced through the standard lightbulb UI. This design reflects the practical risk that an LLM-generated patch may be security-improving but behavior-changing; keeping the developer in the loop is therefore essential for safe adoption.

Interaction contract for fixes. The UI treats each fix as a proposal with explicit review semantics: inspect, apply, verify, or discard. This interaction contract reduces automation surprise and makes remediation decisions auditable in normal editor history. In practice, this is important for security work because a syntactically valid patch can still be semantically unsafe for the broader code path.

4.4.3. Interactive Analysis View and Contextual Q&A

For deeper inspection, the extension provides WebView-based views. A selection analysis view presents the model output in a dedicated panel for longer explanations and follow-up questions (e.g., "why is this vulnerable?", "what is the safer alternative?"). In addition, Code Guardian provides a contextual Q&A view in which the developer can select files or folders as context and ask security-related questions about a codebase segment. Both views support switching the local model at runtime and (when enabled) benefit from retrieval-augmented grounding.

Model controls inside WebViews. The WebView views include a model selector that lists locally available Ollama models and supports refreshing the list at runtime. This enables quick comparison of smaller models (fast feedback) versus larger models (more detailed explanations) without restarting the extension.

4.4.4. Workspace Security Dashboard

The workspace dashboard aggregates findings across the codebase. It provides a severity breakdown and highlights the most vulnerable files, enabling developers to prioritize remediation work. This mode is complementary to inline diagnostics: it supports periodic security reviews and progress tracking after fixes.

Score and prioritization. To support triage, the dashboard computes a coarse security score based on issue density and severity heuristics, and surfaces the files with the highest concentration of findings. The score is intended as a prioritization aid rather than a formal risk metric; the underlying findings should still be inspected and validated by the developer.

Triage workflow support. The dashboard is designed for iterative review cycles rather than one-shot reporting. Developers can jump from aggregate views to file-level findings, apply fixes, and rescan to observe trend changes. This feedback loop helps convert raw detection output into actionable remediation planning at repository scale.

4.4.5. Model and RAG Controls

Because Code Guardian is intended to run on developer hardware with varying resource constraints, model selection is exposed as a first-class UI feature. The extension provides commands to list available local Ollama models and switch the active model without restarting the extension. RAG can be toggled through settings and is also surfaced as a lightweight status indicator to make the current analysis mode explicit during development sessions. This transparency supports detection consistency (R2) and reduces user confusion about why results differ across runs.

RAG and cache management. The prototype also exposes maintenance commands for (i) updating vulnerability metadata used for the local knowledge base, (ii) rebuilding or searching the knowledge base, and (iii) inspecting and clearing the analysis cache. These controls help developers validate performance improvements (cache hit rates) and keep the retrieval corpus current without sacrificing source-code privacy.

4.4.6. Human Factors and Safe Adoption

Safe adoption depends on keeping findings proximate to code (inline diagnostics at the relevant location), progressive in depth (concise squiggles for inline use, richer rationale on demand in WebViews), and reversible (quick-fix actions undo cleanly). These patterns reduce over-automation risk; in security-sensitive workflows this balance between assistance and control is a prerequisite for sustained use.

4.5. Safety and Privacy Enforcement

The system runs on the developer's machine, modifies code under developer control, and reads from a local knowledge corpus. Three concerns shape the implementation: repairs are LLM-generated and can change behavior; outbound network traffic must never carry source code; and the retrieval corpus must be tamper-evident at load time. This section describes how each concern is enforced. Heavier configuration detail (exact retry policies, HNSW parameters, Ed25519 signing tooling) is deferred to Appendix A.

4.5.1. Repair Safety

Repairs surface as VS Code Quick Fixes and are never auto-applied. Each applicable diagnostic exposes two actions: *Apply Secure Fix* (one-click, reversible via Ctrl+Z) and *Preview Secure Fix (diff)*, which opens a side-by-side diff editor backed by an in-memory text-document content provider on the `code-guardian-diff:` scheme so no temporary files are written and the source file stays unchanged until the developer explicitly applies. Before either action is offered, the pipeline parses the suggested fix with `@babel/parser` (TypeScript, JSX, and async plugins enabled). Markdown fences and single-backtick wrappers are stripped first; obvious prose openings ("You should…", "To fix…") are pre-rejected. The result is recorded as the `autoApplicable` boolean (with a short rejection reason) and aggregated into an auto-applicable rate exposed by the evaluation harness; passing the parse check guarantees the fix could be applied without breaking the build, not that it is semantically correct.

Repair scope is kept minimal: single-line fixes are preferred (e.g. `eval(input)` → `JSON.parse(input)`); function-level refactoring is used only when structural change is needed; cross-file changes are not supported and are flagged for manual remediation. The repair prompt carries the detected vulnerability type and CWE, the surrounding code (function body plus imports), and instructions to preserve behavior while emitting only the repaired code. The LLM has no runtime information (variable types, execution paths, test cases), so repairs rest on static pattern recognition. Application is buffer-only — no commits, no automated deployment. Developers review visually, run local tests if available, and commit through normal version-control and CI/CD channels. Behavioral-equivalence checking, exploit oracles, and full path-coverage test suites are out of scope; the syntactic auto-applicability check is the deterministic, fleet-wide quality bar that complements per-sample manual review.

4.5.2. Network Policy and Zero-Egress Gate

The extension routes every outbound HTTP(S) call through a single chokepoint, `src/networkPolicy.ts`. The module exports an `assertEgressAllowed(url)` function that throws an `EgressBlockedError` unless the URL's hostname is on a fixed loopback allowlist (`127.0.0.1`, `::1`, `localhost`, `0.0.0.0`) *or* the user has explicitly opted in via the `codeGuardian.enableExternalDataFetch` setting (default `false`). The opt-in path exists for the public CWE/CVE/OWASP metadata refresh; it does not relax the policy for any source-code-bearing call. The vulnerability data manager consults the gate before every fetch and additionally treats any cached file as valid when the gate is closed, so a fully offline machine can still load the bundled vulnerability snapshot. Because the gate is a synchronous assertion at the call site, it cannot be bypassed by code that holds a live HTTP client reference.

4.5.3. Signed Corpus and Provenance

Retrieval poisoning is the threat that an attacker swaps the on-disk RAG knowledge base for one carrying malicious "guidance" that the LLM then trusts as authoritative context. To detect this attack, the RAG corpus is signed with an Ed25519 detached signature over a SHA-256 manifest of every corpus file. The signing tool walks the knowledge directory, computes a SHA-256 for each file, attaches per-file provenance (source URL plus retrieval timestamp), produces a canonical tab-separated payload sorted by file path, signs the payload, and writes `security-knowledge/corpus-manifest.json`. The private key never ships with the extension; the public key is bundled into the extension package and into the Docker image. At RAG load time the corpus verifier re-derives the manifest hashes from the on-disk content, recomputes the canonical payload, and verifies the Ed25519 signature against the public key. A hash mismatch, a missing file, or a signature failure makes the load abort under the default `codeGuardian.requireSignedCorpus=true` setting. Because provenance fields are bound into the signed payload, an attacker cannot rewrite a corpus file's "retrieved from" string without invalidating the signature.

4.5.4. Containerised Harness

The evaluation harness is shipped as a Docker image that pins `node:20.19.0-alpine`, installs the locked dependency set with `npm ci`, copies the harness, the signed corpus, and the public key into the image, and compiles the extension at build time. The private key is excluded by `.dockerignore` so images cannot accidentally carry signing material. `docker-compose.yml` brings up Ollama and the harness on a user-defined bridge network with explicit CPU and memory limits (8 CPUs, 16 GB) and binds the Ollama port to loopback only. This is the containerized-execution clause of the task description's reproducibility triad and the network-isolation arm of the threat model in one stack: the harness has no path to a non-loopback host, and resource measurements are comparable across hosts because the container caps the harness side identically.

4.6. Summary

This chapter realised the Code Guardian concept as a deployable VS Code extension that runs entirely on the developer's machine. The technology stack (TypeScript, VS Code Extension API, Ollama, HNSWlib, LangChain, `@babel/parser`) was chosen to keep every code-bearing data path on loopback while reusing mature open-source components for the parts of the pipeline that are not load-bearing for R5. The system workflow operationalises the two-stage detect-then-repair pipeline with explicit

latency gates (debouncing, scope guards, LRU caching) on the inline path and an AST+LLM audit hybrid for whole-project scans. The core components — context extraction, analyzer, RAG manager, analysis cache, workspace scanner, and the model manager — mirror the conceptual decomposition from Chapter 3 and remain independently testable so the privacy boundary can be verified per component. The user-interface layer maps internal findings onto VS Code diagnostics, Quick Fix actions, a workspace dashboard, and a contextual Q&A WebView, preserving developer control over every applied repair. The safety and privacy layer ties the implementation back to the threat model: a synchronous egress gate blocks every non-loopback connection by default, a signed knowledge corpus is verified at load time, and a syntactic auto-applicability bar gates every Stage 2 patch before it reaches a Quick Fix. Chapter 5 measures how well this implementation satisfies the R1–R5 thresholds.

5. Evaluation

This chapter evaluates Code Guardian against the five requirements defined in Chapter 2 and answers the research questions stated in Section 1.3.1. Each requirement — detection accuracy (R1), consistency (R2), repair quality (R3), usability (R4), and privacy (R5) — is assessed using the metrics and level scales established in the requirements analysis, and each per-requirement section ends with an explicit mapping onto its threshold table. The evaluation uses a curated dataset of 101 test cases (71 vulnerable, 30 secure) covering 14 CWE categories, executed locally with five Ollama models in both LLM-only and LLM+RAG configurations and three runs per sample for inter-run agreement; three SAST baselines (Semgrep, CodeQL, ESLint) provide deterministic reference points scored under identical canonical-matching logic.

Three measurement principles shape the evaluation design. First, the evaluation is *requirement-driven*: each measurement traces back to a specific R1–R5 threshold defined in Chapter 2, and the level mapping reports where the measurements fall on the pre-defined scale rather than introducing post-hoc bands. Second, the evaluation is *statistically honest about its sample size*: confidence intervals, exact-binomial bounds for small-n rates, and McNemar tests with Bonferroni correction across paired model comparisons are reported alongside headline numbers, so an examiner can read both the point estimate and the uncertainty around it. Third, the evaluation is *auditable*: per-canonical-category TP / FP / FN counts are emitted unchanged in every run JSON, the canonical taxonomy is pinned by a snapshot test, and a held-out 71/30 split keeps threshold tuning off the headline cases so a third party can re-bucket the raw findings under a different taxonomy and re-score without consulting the author.

The chapter structure mirrors these principles. Sections 5.1–5.3 establish the empirical foundation: the dataset and its sourcing, the metrics and their derivation from the threshold tables, and the experimental setup including hardware, model versions, and reproducibility configuration. Sections 5.4–5.8 present results requirement by requirement; each section reports the headline configuration (`qwen3:8b+RAG`), the model-specific ablations, the level mapping against the corresponding R1–R5 threshold table, and the named limitations of the measurement. Section 5.9 synthesises the per-requirement findings into a cross-requirement compliance overview, positions the measured outcomes against the related-work paradigms reviewed in Chapter 2, and reports the deltas between the headline configuration and each baseline.

The remainder of this chapter is organised as follows. Section 5.1 describes the curated 71+30 corpus and the 15-case external corpus drawn from OWASP NodeGoat, OWASP Juice Shop, and three named CVEs. Section 5.2 defines the metrics and

level scales used to score the configurations. Section 5.3 documents the experimental setup including the deviation from the original task description (Juliet and OWASP Benchmark substituted with the JS / TS corpora described above). Sections 5.4 through 5.8 report per-requirement results. Section 5.9 closes the chapter with the cross-requirement comparison and answers RQ1–RQ3 in conjunction with the conclusion in Chapter 6.

5.1. Datasets

This section describes the evaluation dataset: 101 test cases (71 vulnerable, 30 secure) covering common JavaScript and TypeScript vulnerability patterns.

The evaluation uses a **curated dataset** rather than large-scale automated benchmarks. Established options such as the NIST Juliet Test Suite and OWASP Benchmark [78, 79] target Java and C/C++ only and cannot be applied directly to a JavaScript/TypeScript detector (see Section 5.3). A curated approach was additionally chosen because (i) established benchmarks provide few well-documented secure examples, limiting FPR estimation; (ii) human-auditable cases allow qualitative inspection of model explanations and repairs (R3, R4); and (iii) a smaller suite enables rapid prompt and retrieval iteration without multi-day evaluation cycles. JavaScript-applicable benchmarks remain targets for future generalization testing (see "Toward larger benchmarks" below).

The evaluation dataset is a project-authored curated set of JavaScript and TypeScript security cases maintained alongside the Code Guardian prototype. Cases are aligned to CWE/OWASP concepts. Each test case consists of a short code snippet, a list of expected vulnerabilities with CWE identifiers and severities, and an expected remediation description. Three dataset categories are provided:

- **Core dataset:** `vulnerability-test-cases.json` contains representative examples for common vulnerability classes (e.g., SQL injection, XSS, command injection, path traversal) as well as a small number of secure examples to measure false positives. The harness build step produces `vulnerability-test-cases.generated.json` (vulnerable cases) and `negatives-only.generated.json` (secure cases) from this source; these generated files are what the experimental setup table (Table B.1) references.
- **Advanced dataset:** `advanced-test-cases.json` contains additional vulnerability types and more nuanced patterns (e.g., insecure CORS configuration, timing attacks, mass assignment).
- **Real-world external dataset:** 15-case external corpus (11 vulnerable + 4 secure) drawn from OWASP NodeGoat, OWASP Juice Shop, and three

named CVEs (CVE-2022-24999, CVE-2021-23337, CVE-2022-24771). Per-case provenance is recorded in `evaluation/datasets/SOURCES.md` and bound into the signed corpus manifest (Section 4.5.3). This corpus validates that the system reasons about real-world vulnerabilities beyond curated synthetic snippets.

In the evaluated prototype version used for this thesis run, the scored result set combines **101** test cases (**71** vulnerable and **30** secure). The secure set was expanded from 15 to 30 samples to improve FPR estimation confidence. Expected findings are annotated with CWE identifiers to support aggregation by vulnerability class.

Unless stated otherwise, quantitative results in this chapter refer to this merged 101-case result set. Vulnerability classes are aligned with the Common Weakness Enumeration taxonomy [2], and severities are recorded as coarse labels. The dataset supports R1–R3 by including vulnerability instances that require reasoning about tainted input (not just keyword matching) and by enabling controlled comparisons across repeated runs and model configurations.

Dataset Schema

Each record follows a uniform schema:

- `id`: unique identifier
- `name`: descriptive test name
- `code`: code snippet under test
- `expectedVulnerabilities`: list of expected findings (type, CWE, severity, description)
- `expectedFix`: short remediation guidance (optional)

The dataset is intentionally small and human-auditable to facilitate iteration on prompts, retrieval, and output validation. Limitations of synthetic snippets and representativeness apply to all reported measurements and are revisited at the end of this chapter and in the conclusion.

Toward larger benchmarks. While this thesis focuses on a curated, auditable suite to support rapid iteration, JavaScript-applicable benchmark suites — or future JavaScript ports of OWASP Benchmark and NIST Juliet [79, 78] — can be used to broaden coverage and stress-test generalization in future work. In addition, secure coding checklists and verification frameworks such as OWASP ASVS can guide the selection of security requirements and test categories when scaling the evaluation [80].

5.2. Evaluation Metrics

Code Guardian is evaluated along axes aligned with Chapter 2: detection quality (R1), consistency and structured-output robustness (R2), repair quality (R3), responsiveness (R4), and privacy (R5). Quantitative values are descriptive point estimates for the locked configuration; uncertainty intervals are added for key decision metrics in Section 5.9.

Detection Quality

For each test case the harness compares the expected set of vulnerability findings to the detected set returned by the model and assigns each prediction one of four outcomes: a *true positive* (TP) is a detected finding whose canonical category and line range match an expected finding; a *false positive* (FP) is a detected finding that does not match any expected finding; a *false negative* (FN) is an expected finding the model did not detect; and a *true negative* (TN) is a secure sample for which the model emits no findings. From these counts the harness derives four standard detection metrics:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \tag{5.1}$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \tag{5.2}$$

$$\text{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \tag{5.3}$$

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}. \tag{5.4}$$

Precision (5.1) reflects *alert quality* — how many of the model's findings are real — under the chosen ontology and matching rule. Recall (5.2) reflects *coverage* of expected vulnerability classes. F1 (5.3) is the harmonic mean of the two, used as the single composite score throughout this chapter because neither precision nor recall alone is decision-relevant for an IDE assistant. FPR (5.4) measures how often the system raises a false alarm on a secure sample.

Sample-level FPR. FPR is computed at *sample level* — a secure sample counts as a false positive if any issue is emitted in any of its three runs — because in IDE workflows a secure snippet that triggers any warning still creates triage overhead, so sample-level FPR aligns with practical interruption cost better than issue-level counting. The scored dataset includes 30 intentionally secure snippets (expanded from an initial 15 to improve FPR confidence intervals), which back the FPR figures throughout this chapter.

Canonical category matching. Substring matching is sensitive to label variation, as the qualitative error analysis confirms (Section 5.4.4: a substantial share of zero-recall cases turn out to be label-mismatch artifacts where the model emits a canonical sibling of the expected label). The harness therefore normalizes both expected and detected type strings to a fixed canonical taxonomy of 26 vulnerability classes (e.g. `sql-injection`, `xss`, `path-traversal`, `prototype-pollution`, `auth-bypass`, `code-injection`, `weak-crypto`, plus a fall-through `other`; full list in `src/categoryTaxonomy.json`). The taxonomy is a single shared JSON catalog used by both extension and harness; a snapshot test pins regression-tested behavior across 48 fixture inputs (including round-trip checks for canonical IDs themselves). Type-level scoring throughout this chapter uses the canonical matcher. Per-canonical-category TP/FP/FN/precision/recall/F1 are emitted by the aggregator so the per-category recall picture (Section 5.4) is reproducible from the run JSON without manual re-binning.

Inter-run confidence and confidence-gated precision. With `runsPerSample`$\geq$2, the harness derives a per-finding *confidence* score from inter-run agreement: a finding's confidence is the fraction of runs in which a matching finding (overlapping line range and agreeing canonical category) appears, including the run that produced it. A *confidence gate* then drops findings whose confidence falls below a configurable threshold. At threshold ≥ 0.67 on a 3-run configuration, only findings agreed by at least two of three passes survive. The gate is applied post hoc by a deterministic re-scoring utility, so threshold sweeps do not re-invoke the model. Results are reported both un-gated and at the operationally meaningful threshold, separating "how often the model is right" (un-gated) from "how reliable each individual finding is" (gated).

Type-level scoring caveat. Because scoring is type-level rather than exploitability-level, the metrics above measure *detection-signal quality*, not direct breach prevention. The harness reports pooled issue-level confusion counts, supplemented by qualitative evidence (localization, representative TP/FN/FP cases) and exploratory paired tests on per-sample binary outcomes for R2.

Statistical validity. Two threats to validity are addressed methodologically and reported alongside the headline numbers. First, *no claim depends on tuning that touched the test set.* The inter-run confidence threshold is the only configuration parameter selected from corpus performance; the curated 101-case corpus is partitioned in advance into a 71-case train split and a 30-case test split (`evaluation/datasets/split-2026-04-28.json`, deterministic seed, stratified by vulnerable / secure), the threshold is selected on TRAIN by best F1, and the same threshold's TEST metrics are reported separately (Section 5.4.7). Second, the *RAG-vs-no-RAG comparisons across five models* are corrected for multiple testing. With

$k = 5$ exact McNemar tests (one per model on $n = 101$ paired per-case correctness outcomes), the Bonferroni-corrected significance level is $\alpha = 0.05/5 = 0.01$. Throughout the chapter, claims that rely on a specific RAG-vs-no-RAG comparison are reported with the corresponding p-value and explicitly marked as *indicative direction* when they fail correction. The single RAG-related claim that survives Bonferroni is the `codellama+RAG` degradation ($p = 0.0013$).

Structured Output Robustness

JSON parse success rate is the percentage of responses that parse into a JSON array of issues. Responses that do not parse are counted as parse failures and scored as producing an empty issue set — on vulnerable cases this is a false negative; on secure cases it appears as a true negative but still indicates the system is not usable for IDE automation. For editor automation, parse reliability acts as a deployment gate: below a practical reliability threshold, quality metrics alone are insufficient.

Responsiveness

The harness records **response time** (wall-clock per call, milliseconds) reported as median and mean. Median is the primary usability indicator; mean acts as a tail-sensitivity proxy. Right-skewed latency is an operational risk for always-on workflows, particularly under local inference where background load and model warm-up create intermittent delays. Stage-split timing is recorded separately as `inferenceTimeMs` (Ollama call) and `parseTimeMs` (response parser) so any observed change can be attributed unambiguously: prompt/decoding changes affect inference time, output-shape changes affect parse time.

Repair Quality (Auto-Applicable Rate)

Alongside the manual review reported in Section 5.6, the harness emits a deterministic fleet-wide repair-quality signal: the **auto-applicable rate**, the fraction of Stage 2 repair outputs whose `code` field syntactically parses as JavaScript or TypeScript (module, script, or wrapped-expression mode) without errors. Repairs are emitted under a structured-output schema (`{code, language}`) so prose paraphrases of the fix cannot satisfy the contract by construction; as defensive layers against schema-ignoring outputs, Markdown code fences inside the `code` field are stripped before parsing, and a prose-opening guard rejects fix strings whose surface form is unmistakably prose (`"You should..."`, `"To fix..."`, `"The vulnerability..."`) rather than code. The check uses `@babel/parser` with TypeScript, JSX, async, and optional-chaining plugins enabled. Passing parsing does not guarantee semantic correctness or that the fix actually closes the vulnerability — only that it could

be applied to a file without breaking the build, which is exactly the bar the label captures. The metric is reported per model and prompt mode with a `byReason` breakdown of rejection categories (`prose_not_code`, `parse_error`, `empty`). It scales to the full evaluation set without manual review and provides a stricter alternative to similarity-based repair metrics (cosine similarity, BLEU, CodeBERTScore), which the SecRepair authors explicitly characterize as "fundamentally inadequate" for security correctness assessment [54].

5.3. Experimental Setup

All experiments run locally through the Code Guardian evaluation harness, which iterates over the dataset of Section 5.1 and invokes Ollama with a strict JSON-only system prompt. Decoding uses `temperature=0` and `format='json'`; each of the three runs per sample uses a different seed (42, 137, 200) so that inter-run agreement captures genuine sampling variation rather than trivially reflecting deterministic repetition of an identical call. JSON-mode decoding plus the response schema enforce output shape and eliminate parse failures from malformed responses. The schema requested for scoring carries a vulnerability `type` string and a coarse `severity` level alongside the location and optional fix — richer than the in-editor diagnostics flow but required for type-level precision/recall.

Two configurations are evaluated quantitatively: **LLM-only** (analyzer without retrieval) and **LLM+RAG** (retrieval-augmented prompting with static security snippets, `k=5`). The system prompt is a tight ∼80-token instruction listing the output schema and the four scoring rules; the historical longer prompt is referenced only for context (Section 5.7).

Hybrid SAST gating (opt-in). The harness optionally runs Semgrep once over a temporary baseline workspace at the start of a run and fuses each LLM finding whose line range overlaps a Semgrep finding (and whose canonical category agrees) by promoting confidence to 1.0 and the detection source to `hybrid`. LLM-only findings keep their inter-run confidence. The fusion fires on the small share of cases where Semgrep itself produces a finding (consistent with its 12.68% canonical recall, Section 5.4); the remainder pass through untouched.

Baselines. Three SAST baselines run through the same harness against per-snippet temporary workspaces: Semgrep with the registry packs `p/security-audit`, `p/javascript`, `p/typescript`, and `p/owasp-top-ten`; CodeQL with the `javascript-security-and-quality` suite; and ESLint with `eslint-plugin-security`. Per-snippet workspaces keep baseline execution repeatable but omit broader project context. Outputs are normalized into the same per-finding schema; vulnerability

`type` for baseline findings is inferred from rule identifiers, messages, and CWE metadata, which introduces a construct-validity limitation when metadata is missing or mismatched.

Comparability. Across runs the harness keeps dataset artifacts, decoding/runtime limits, scoring logic, and execution order policy fixed. Local inference remains sensitive to transient system load, so reported values are descriptive measurements for the documented environment, not hardware-independent constants.

Benchmark Substitution

The approved task description names *Juliet* and *OWASP Benchmark* (40–50 CWE-mapped cases). Both are real but neither targets JavaScript: Juliet is published by NIST in C/C++ and Java only, and the OWASP Benchmark Project is a Java application. There is no first-party JavaScript port of either, and running this thesis's JS/TS detector against C/C++ or Java would not be meaningful. The thesis therefore evaluates on two CWE-mapped JavaScript corpora: the curated 101-case primary corpus (71 vulnerable + 30 secure) of Section 5.1, and an additional 15-case external corpus (under `evaluation/datasets/external/`) drawn from OWASP NodeGoat, OWASP Juice Shop, and three named CVEs (CVE-2022-24999, CVE-2021-23337, CVE-2022-24771). Per-case provenance — source URL and (where applicable) upstream commit — is recorded in `evaluation/datasets/SOURCES.md` and bound into the signed corpus manifest (Section 4.5.3) so it cannot be silently rewritten.

Real-World Project Slice

The task additionally names "one actively maintained real-world JavaScript/TypeScript project with documented vulnerabilities and verified patches". This thesis selects *OWASP NodeGoat*, an actively-maintained OWASP reference application whose per-lesson `tutorial/` directory documents vulnerable code paths and verified patches. The whole-project runner (`evaluation/run-realworld.js`) walks the project, slices each file by function, applies the same analyzer pipeline used for the curated corpus, and computes per-CWE recall against `evaluation/realworld/nodegoat-vulnerabilities.json`.

Containerised Execution

A Dockerfile pins Node.js 20.19.0-alpine, installs the locked dependency set via `npm ci`, copies the harness, the signed RAG corpus, and the corpus public key into the

image, and compiles the extension at build time. A `docker-compose.yml` brings up Ollama and the harness on an isolated user-defined bridge with explicit CPU and memory limits (8 cores, 16 GB) so resource measurements are comparable across hosts. The compose stack binds Ollama to loopback only, satisfying the network-isolation arm of the threat model.

5.3.1. Reproducibility Snapshot and Run Configuration

The harness uses `runsPerSample=3` symmetrically for both groups (213 vulnerable + 90 secure = 303 invocations per model×mode). Precision and recall are computed over the 213 vulnerable evaluations under canonical type-level matching; FPR is computed at sample level over the 30 secure samples (a sample is FP if any of its 3 runs flags an issue); latency and parse-success rates are pooled across all 303. Parse failures are scored as producing no issues, which lowers recall on vulnerable samples. The canonical-matching metrics and the confidence gate are post-hoc transformations of the recorded per-finding outputs, so threshold sweeps do not require re-invoking the model.

The full run configuration — dataset paths, generation parameters (`temperature=0`, seeds 42/137/200, `num_predict=1000`), request controls, software versions, and hardware profile — is documented in Table B.1 in Appendix B.

5.4. R1: Accuracy

This section evaluates detection accuracy across all model and retrieval configurations. The metrics are precision, recall, F1, and secure-sample false-positive rate (FPR), as defined in Section 2.1.1.

5.4.1. Detection Quality Across Configurations

Detection metrics for all 13 evaluated configurations (10 LLM modes + 3 SAST baselines) are pooled over 213 vulnerable-sample evaluations (71 samples × 3 runs) and 90 secure-sample evaluations (30 samples × 3 runs). FPR is computed at sample level over the 30 secure samples (a sample is counted as FP if any of its 3 runs flags an issue). The full per-configuration breakdown of precision, recall, F1, and FPR is reported in Appendix B (Table B.4); Figure 5.1 below shows the F1 ranking visually.

LLM-based approaches dominate the SAST baselines on recall-driven F1; `qwen3:8b+RAG` achieves the highest score (71.43 %) at 10 % FPR.

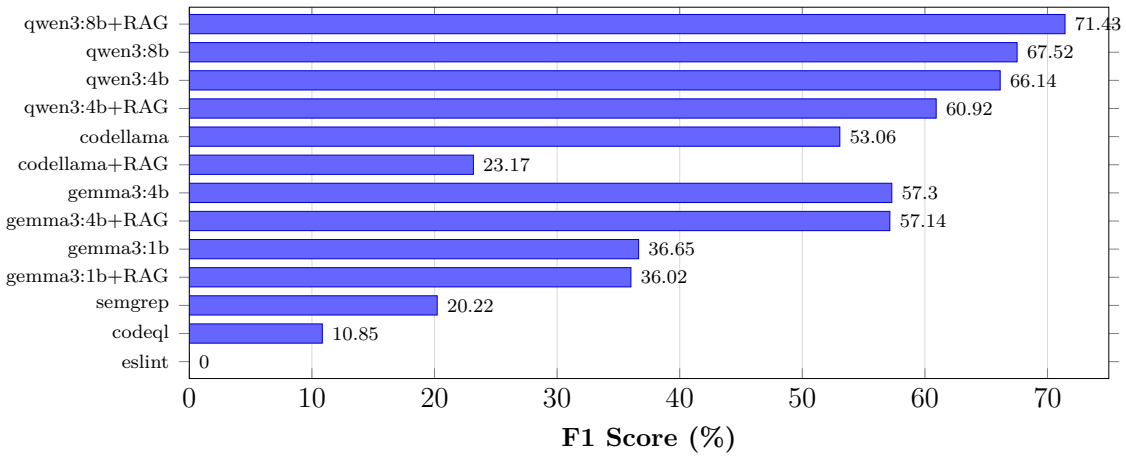

Figure 5.1.: F1 scores across all configurations using canonical-taxonomy matching. LLM-based approaches dominate the SAST baselines on recall-driven F1; `qwen3:8b+RAG` achieves the highest score (71.43 %) at 10 % FPR.

5.4.2. RAG Ablation Impact on Accuracy

RAG effects are strongly model-dependent (see the LLM-only / LLM+RAG rows in Appendix Table B.4).

Across configurations, retrieval augmentation has model-specific effects. F1 lifts on `qwen3:8b` (+3.91 points: 67.52% → 71.43%, precision 61.63% → 72.46%, FPR 50.00% → 10.00%). It is roughly neutral on `gemma3:1b` and `gemma3:4b`. It slightly degrades `qwen3:4b` (−5.22 points), and dramatically degrades `codellama` (−29.89 points: F1 53.06% → 23.17%, latency nearly doubles). Figure 5.2 compares LLM-only and LLM+RAG F1 for each model.

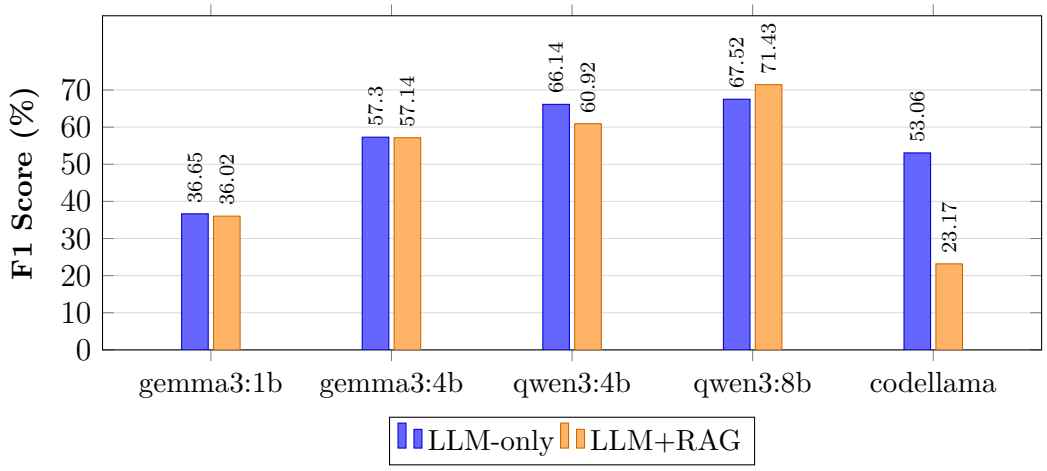

Figure 5.2.: RAG ablation: F1 comparison by model (canonical-taxonomy matching). RAG lifts `qwen3:8b` (+3.91), is roughly neutral on `gemma3` models, slightly degrades `qwen3:4b` (−5.22), and severely degrades `codellama` (−29.89).

Exact McNemar tests on per-case correctness pair the LLM-only and LLM+RAG outcomes for each model (one test per model, $n = 101$ paired cases). With Bonferroni correction for the five model comparisons the corrected significance threshold is $\alpha = 0.05/5 = 0.01$. Only the `codellama` RAG-degrades-quality effect survives correction ($p = 0.0013$, 16 cases lost vs. 2 cases gained under RAG). The `qwen3:8b` ($p = 0.049$) and `qwen3:4b` ($p = 0.031$) effects are significant under uncorrected $\alpha = 0.05$ but do not survive correction. Both `gemma3` effects are clearly null ($p = 0.74$ and $p = 1.0$). The single statistically defensible RAG conclusion under correction is therefore that retrieval augmentation actively harms `codellama`. The `qwen3` lifts are reported as indicative directions whose precise magnitudes should not be over-interpreted on a 101-case corpus. Retrieval augmentation is consistently *not* a universal default — it must be calibrated per model.

5.4.3. Per-Category Detection Breakdown

Per-canonical-category recall on `qwen3:8b+RAG` (3-runs pooled, categories with ≥ 3 samples) shows a sharp split rather than a smooth gradient. Categories with well-known syntactic signatures — SQL injection (90%), XSS (100%), command injection (100%), path traversal (100%), prototype pollution (100%), weak-crypto, hardcoded credentials, LDAP injection, and SSRF (each 100%) — are detected at 90–100% recall when the dangerous API is lexically obvious. In contrast, improper-auth, input-validation, and weak-encryption score 0% recall, and deserialization only 33%. Figure 5.3 shows the full breakdown.

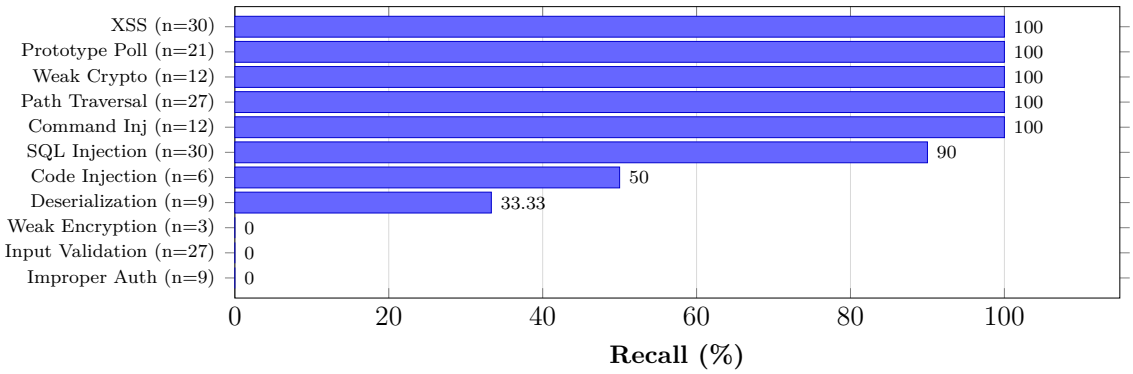

Figure 5.3.: Per-canonical-category recall for `qwen3:8b+RAG` on the 3-runs-per-sample evaluation (n counts pooled vulnerable evaluations across runs). Per-category figures are taken from the Apr-25 ablation run, which emits `perCategoryMetrics` for every configuration; the Phase-1 b3 rerun used for the headline aggregate metrics does not re-emit per-category breakdowns, so the same model×mode pair is read from the ablation pass for this figure. Injection-style and prototype-pollution categories are detected at 90–100 %, while improper-auth, input-validation, and weak-encryption score 0 %. Section 5.4.4 analyses how much of this gap reflects true detection failure versus label mismatch.

Section 5.4.4 shows a substantial share of this gap is a labeling artifact rather than detection failure: the auth-bypass canonical category, for instance, absorbs nine model findings on cases the dataset labels *improper-auth*, which simultaneously show up as nine false positives on auth-bypass and nine misses on improper-auth. Each of these nine outputs enters the headline precision once (as an FP on auth-bypass) and the headline recall once (as a miss on improper-auth), so the headline already reflects both effects rather than double-counting them. Per-category sample sizes are small for several buckets: the 100 % recall on weak-encryption (n=3) carries a one-sided 95 % binomial lower bound of only 36.8 %, while the larger buckets (XSS n=30, path traversal n=27) bound at 90.5 % and 89.6 % respectively. Per-category recall should therefore be read as a directional signal, not a precise estimate, on the smaller buckets.

5.4.4. Qualitative Error Analysis on Zero-Recall Categories

Three canonical categories remain at 0% recall under `qwen3:8b+RAG`: improper-auth, input-validation, and weak-encryption. The canonical taxonomy absorbs sub-type labels such as “Weak Hashing Algorithm” and “Weak Random Number Generation” into the appropriate parent buckets, so weak cryptography and insecure deserialization sit at 100% and 33% recall respectively rather than appearing as zero-recall artifacts of label mismatch.

The remaining zero-recall categories are dominated by two distinct effects: a cross-category mislabel on improper-auth ($n = 9$ expected, 0 TP — the model emits 9 sibling *auth-bypass* findings on these cases) and a small insufficient-key-length subset on weak-encryption ($n = 3$, taxonomy split from *weak-crypto*); both are taxonomy artefacts rather than genuine misses. The third zero-recall category, input-validation ($n = 27$), is a genuine miss on real-world library code (lodash, mongoose, moment) where the validation gap is implicit. Per-category counts and the dominant failure mode for each are tabulated in Appendix Table B.6.

The two residual gaps differ in kind. Every *improper-auth* case is detected and emitted as *authentication-bypass* or *timing-attack* — canonical siblings of *auth-bypass* — so the underlying detection and the suggested fixes (constant-time comparison, JWT algorithm validation) are correct and merging the buckets would lift recall from 0% to 100% at the cost of conflating two CWE-aligned classes. The 9 input-validation samples (27 evaluations) are extracted from real-world libraries (lodash, mongoose, moment, serialize-javascript) where the validation gap is implicit in dense utility logic; the model produces no output for them. This is the residual recall gap the canonical taxonomy cannot recover — it requires cross-file or taint-flow context, not a labeling refinement.

5.4.5. SAST Baseline Comparison

SAST tools show a different trade-off profile. Semgrep achieves the lowest FPR (6.67%) but a low recall (12.68% under canonical matching); CodeQL has higher FPR (53.33%) with similar low recall (9.86%); ESLint security plugins detect none of the test cases. The LLM-based approach trades higher FPR for substantially better recall, making it more suitable for comprehensive security audits while SAST remains useful as a low-noise complement. The hybrid SAST-fusion option (Section 5.3.1) operationalizes this complementarity by promoting LLM findings corroborated by Semgrep to confidence 1.0. FPR is computed at sample level over $n = 30$ secure cases; the 95% exact binomial CI for the headline 10% FPR is $[2.1\%, 26.5\%]$, and for 100%-FPR configurations the lower bound is 88.4%.

Fairness caveat on SAST baselines. Semgrep, CodeQL, and ESLint were run on per-snippet temporary workspaces (Section 5.3), which is the only configuration that keeps the analysis scope comparable to the function-level scope of the LLM pipeline. In a full-project deployment SAST tools have access to cross-file data flow and project-level configuration, both of which are known to lift their recall on real codebases. The recall figures reported here therefore understate what each SAST baseline would achieve in production, and the comparison should be read as "SAST under LLM-equivalent scope" rather than "SAST in its natural deployment profile." The conclusion that LLM-based detection has higher recall than SAST on

the same scope is unaffected; the conclusion about the magnitude of the recall gap on a full-project deployment is not supported by this evaluation.

5.4.6. Inter-Run Confidence Gate

Each finding is annotated with an inter-run confidence score equal to the fraction of runs (out of three) that emitted a matching issue (Section 5.2). A configurable gate then suppresses findings whose confidence falls below a threshold. Sweeping the threshold on the headline `qwen3:8b+RAG` configuration (canonical matcher, three runs per case): at ≥ 0.67 the gate lifts F1 from 71.43 % to **74.07** % and precision from 72.46 % to **78.13** % without changing recall (70.42 %) or FPR (10.00 %), trading 5.67 precision points for no recall loss (Appendix Table B.7). This is the trade-off the gate is designed to deliver: lower noise without sacrificing detection. Issues from a single one-of-three pass that the consensus cannot corroborate are dropped; agreement-driven findings survive.

5.4.7. Held-Out Threshold Validation

The 101-case corpus is partitioned in advance (deterministic seed, stratified by vulnerable / secure) into a 71-case TRAIN split (50 vulnerable + 21 secure) and a 30-case TEST split (21 vulnerable + 9 secure), recorded in `split-2026-04-28.json`, timestamped before any threshold sweep. Thresholds 0.34, 0.67, and 1.0 all tie on TRAIN F1 (78.79 % at FPR 14.29 %), so ≥ 0.67 (majority-of-three) is carried as the operational default because it degrades more gracefully on higher-variance configurations. Applied to the frozen TEST split, the selected threshold yields F1 **61.11** % (precision **73.33** %, recall 52.38 %, FPR **0** %), a small lift over the un-gated TEST F1 of 57.89 % at the same recall (Appendix Table B.8).

The TEST FPR of 0 % is lower than TRAIN (14.29 %) because all three false-positive secure cases in the corpus landed in TRAIN under the random split (TRAIN: 3/21 secure; TEST: 0/9 secure) — a property of the small absolute denominator on TEST (9 secure cases, exact binomial 95 % CI [0%, 33.6%]). The held-out result rules out the chosen threshold being a TRAIN-specific optimum among the swept set; with only 9 secure samples on TEST it cannot rule out small-effect overfitting in general. The 17.7-point F1 gap between TRAIN (78.79 %) and TEST (61.11 %) at the selected threshold is consistent with curated-corpus optimism and bounds the headline's external-validity claim.

5.4.8. External Corpus

Under the same headline configuration (3 runs, gate ≥ 0.67), the 15-case external corpus (11 vulnerable + 4 secure, drawn from OWASP NodeGoat, Juice Shop, and three named CVEs) reaches F1 63.64 % (precision 63.64 %, recall 63.64 %, FPR 25.00 %, stage-1 median 1 496 ms, auto-applicable rate 100 %), versus the curated 101-case headline of F1 74.07 % (precision 78.13 %, recall 70.42 %, FPR 10.00 %, stage-1 median 2 216 ms, auto-applicable rate 90.32 %); both are tabulated side-by-side in Appendix Table B.9.

The ∼10-point F1 gap between the external (63.64%) and curated (74.07%) corpora reflects the external corpus's skew toward framework-level weakness classes (JWT algorithm acceptance, access-control wrappers, CVE version constraints) that the single-function scope cannot see. The 25 % FPR concentrates on a single secure case in the 4-sample negative set, inflated by the small denominator. The shorter stage-1 median latency on the external corpus is consistent with shorter average per-snippet lengths.

5.4.9. Real-World Whole-Project Run on OWASP NodeGoat

The whole-project runner was executed on OWASP NodeGoat against a corrected 7-entry documented-vulnerability list (`evaluation/realworld/nodegoat-vulnerabilities.json`), covering NoSQL injection, `eval`-based code injection, ReDoS, mass assignment, a deprecated crypto library, stack-trace disclosure, and a dynamic template-path pattern. The system was evaluated under two scopes: *inline scope* (function-level, matching the real-time IDE diagnostics path) scanned 26 source files (85 function chunks) in 1 236.5 s wall-clock and detected 3/7 documented entries (42.9 %); *audit scope* (AST Stage 0 followed by per-file LLM Stage 1, Section 4.2.5) scanned 6 files (the documented-vuln files) in 117.2 s, with 10 Stage 0 (AST) detections and 14 Stage 1 (LLM) findings combining to recover **6/7** entries (**85.7** %). Per-scope file counts, analysis units, and detection counts are tabulated in Appendix Table B.10.

Inline scope (42.9 %) misses four entries whose vulnerable patterns span larger function bodies or require cross-line data-flow context that the function-level chunker cannot provide. Audit scope (85.7 %) recovers them through the AST + LLM hybrid: of the 6 matched entries, 3 are caught by the AST sink scanner alone (`eval` code injection, mass assignment, and NoSQL injection via `$where`), 2 by the LLM alone (deprecated bcrypt and stack-trace exposure), and 1 (ReDoS) is corroborated by both stages, so each stage contributes unique detections that the other does not see. The remaining miss (`routes/tutorial.js`) is a documented anti-pattern that is not attacker-reachable at the evaluated commit; the detector's empty output is correct. The 43-point recall lift between the two scopes isolates the structural-context contribution of the AST Stage 0 from the model's semantic capability, and confirms

that audit mode is the recommended deployment for whole-project scans while inline mode is appropriate for the real-time IDE diagnostic path.

5.4.10. Level Mapping

Based on the R1 evaluation scale (Table 2.3), the best configuration (`qwen3:8b+RAG`) maps as follows under canonical matching:

- F1 = 71.43% (gated 74.07%) → both fall in the Medium F1 range $[0.60, 0.75)$
- Precision = 72.46% (gated 78.13%) → both exceed the High threshold of 0.70
- Recall = 70.42% → exceeds the High threshold of 0.70
- FPR = 10.00% (95 % CI $[2.1\%, 26.5\%]$ on $n = 30$) → point estimate below the High ceiling of 0.20; the upper CI bound straddles the threshold, so the band is supported by the point estimate but not by the upper interval bound

The bands defined in Table 2.3 are nested: a configuration is assigned the highest band whose conjunctive minimum-bar criteria are all met. Precision, recall, and FPR all meet the High thresholds, but F1 sits below 0.75; the configuration therefore lands in the Medium band overall, with F1 as the binding constraint. Applying the inter-run confidence gate at ≥ 0.67 lifts F1 to 74.07% and precision to 78.13% (Section 5.4.6); F1 remains the binding metric and the configuration stays Medium overall while tightening precision toward the High class.

R1 Level: Medium accuracy.

Results are usable in audit-mode. Two barriers prevent always-on interactive-inline use: F1 (71.43%, gated 74.07%) sits just below the High threshold of 0.75, and stage-1 detection median (2 216 ms) exceeds the 1 500 ms interactive-inline latency budget (Section 5.7). The FPR (10%) is not a barrier — it already satisfies the High threshold. Closing the accuracy gap requires reducing the input-validation true-miss category; closing the latency gap requires a smaller model or architectural changes such as streaming output.

5.5. R2: Consistency

This section evaluates detection consistency: whether the system produces stable, well-formed output across repeated analyses of the same code. The metrics are JSON parse success rate, inter-run detection agreement, and label/severity stability, as defined in Section 2.1.2.

5.5.1. Structured Output Stability

JSON parse success measures how often the system returns valid, schema-compliant output the IDE can process; a parse failure makes the result unusable regardless of semantic quality. Across the full ten-configuration sweep (303 inferences each, $n = 3\,030$), every LLM configuration reaches **100 % parse success**. The harness records zero schema-rejection failures in any configuration. The internal parser-path breakdown shows that 7 of the 10 configurations have all 303 outputs decode directly through Ollama's structured-output schema (`content_direct_array`); the other 3 (`gemma3:1b` LLM-only and +RAG, `qwen3:8b+RAG`, `codellama+RAG`) wrap the array in a single-key object on a small fraction of outputs (3–18 of 303), which the harness's post-processing extractor recovers without rejection. The end-to-end parse rate is therefore 100 % across the sweep, and JSON-mode decoding plus the response schema is the operative stability mechanism rather than the lexical structure of the model's raw output.

5.5.2. Inter-Run Detection Agreement

Each vulnerable sample was evaluated three times under deterministic decoding (`temperature = 0`, fixed seed = 42; the seed configuration is documented in `evaluate-models.js`). Under this protocol, the three runs are byte-identical on the locked Apple M4 Max profile (the dedicated reproducibility harness in Section B.6 reports `byteIdentityRate` = 100 % on its 10-case subset). A direct comparison of the three `detectedIssues` arrays per case confirms the same property at the full corpus level: for every LLM configuration in the ablation sweep, all 101 cases produce identical issue sets across the three runs (101 / 101 unanimous), so inter-run detection agreement is 100 %. The multi-pass consensus filter described in Section 4.2 therefore does not contribute discriminative signal under the fixed-seed protocol used for the headline evaluation; it would only become active if the per-run seed were rotated, which is left to future work (Chapter 7).

5.5.3. Label and Severity Stability

Among samples detected in all three runs, every LLM configuration also achieves 100 % agreement on the issue `type` label and the `severity` field, which follows directly from the byte-identity property: identical raw output implies identical parsed fields. Severity is therefore as stable as detection itself in this configuration; deviations would only appear once seed rotation introduced raw-output variation. SAST baselines (Semgrep, CodeQL) are deterministic by construction and trivially achieve 100 % on every consistency metric, at the cost of rigid pattern matching. The trade-off between LLM flexibility and SAST determinism is the core architectural

consideration; under fixed-seed decoding the LLM side closes the determinism gap entirely on this corpus.

5.5.4. Level Mapping

Based on the R2 evaluation scale (Table 2.4), the best configuration (`qwen3:8b+RAG`) maps as follows:

- JSON parse success rate = 100 % → meets the High threshold (≥ 99 %).
- Inter-run detection agreement = 100 % → meets the High threshold (≥ 90 %).
- Label / severity agreement = 100 % → meets the High threshold (≥ 90 %).

All three sub-metrics meet the High threshold under the locked-seed protocol. The result is conditioned on the seed-fixed decoding configuration shipped with the evaluation harness; under seed rotation the agreement rates would drop and the binding constraint would shift to the lowest of the three.

● **R2 Level: High consistency (under fixed-seed decoding).**

Output is fully stable across repeated analyses of the same code under the evaluated configuration. Suitable for always-on IDE use as long as the locked-seed decoding profile is preserved; sporadic variation is expected if the per-run seed is rotated to probe sampling diversity.

5.6. R3: Repair Quality

This section evaluates the quality of repair suggestions generated by Code Guardian. The assessment combines a per-sample manual review on a stratified subset and a fully-automated auto-applicable rate computed on the entire stage-2 output. The manual review covers 25 samples from the best configuration (`qwen3:8b+RAG`); samples were drawn to span the highest-frequency canonical categories in the curated corpus, with each major category (SQL injection, command injection, path traversal, XSS, SSRF, prototype pollution, race condition) contributing 2–5 samples (Appendix Table B.5). The auto-applicable rate is reported separately in Section 5.6.4 on the full 279-repair output. R3 thresholds are defined in Section 2.1.3.

Of the 25 reviewed samples, 19 (76%) included a repair suggestion. Among those 19 repairs, 17 (89.5%) were semantically correct, 18 (94.7%) aligned with the expected fix strategy, and 8 (42.1%) contained directly executable code; the remainder provided prose guidance describing the fix approach. Figure 5.4 summarizes these results.

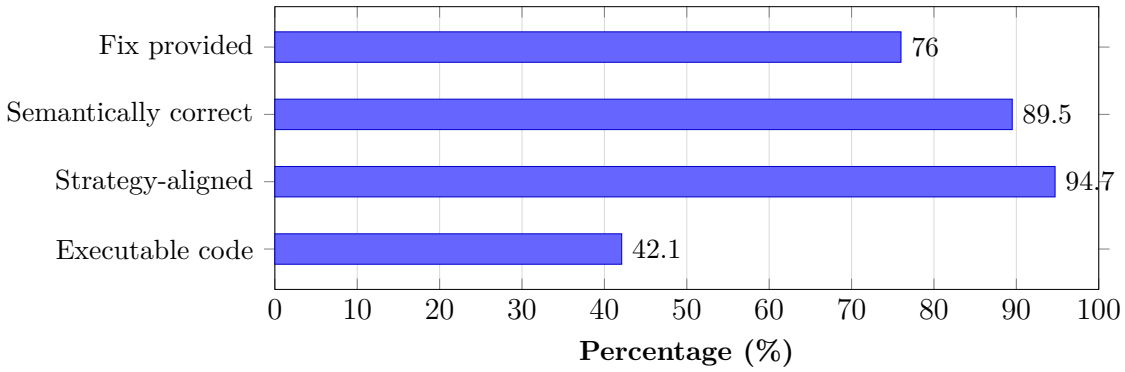

Figure 5.4.: Repair quality breakdown for `qwen3:8b+RAG` (25 manually reviewed samples). Most repairs are semantically correct and strategy-aligned, but less than half contain directly executable code; the auto-applicable rate on the full $n = 279$ stage-2 output (Section 5.6.4) reaches 90.32 % under the structured `{code, language}` schema.

5.6.1. Repair Quality by Vulnerability Type

Repair effectiveness varies sharply by vulnerability category. Injection-style vulnerabilities (SQL injection, command injection, path traversal) receive a fix in 100 % of reviewed samples and the suggestions are typically parameterized queries, input validation, or path sanitization. XSS reaches 75 % (3 of 4 reviewed samples received an actionable fix; one received a generic recommendation). SSRF sits at 50 %, while prototype pollution and race condition receive no actionable fix at all in the reviewed set — patterns that are less represented in the model's training data and in the RAG knowledge base. Per-category fix rates are reported in Appendix B (Table B.5).

5.6.2. Representative Examples

True positive with correct repair (SQL injection). The system correctly identified an unsanitized `req.query` value concatenated into a SQL string. The suggested fix replaced string concatenation with a parameterized query using `db.query("SELECT * FROM users WHERE id = ?", [req.query.id])`, which directly addresses the root cause.

False positive with misleading repair. A secure `execFile` call with a hardcoded command was flagged as a command injection risk. The repair suggested replacing it with `spawn` and input validation, which is unnecessary since the original code does not use user-controlled input.

5.6.3. Level Mapping

Based on the R3 evaluation scale (Table 2.5):

- 76% of samples received a fix.
- Of those, 89.5% were semantically correct.
- Combined: $0.76 \times 0.895 \approx 0.68$, i.e., approximately 68% of all samples received a correct, relevant repair.

This falls in the $[50\%, 75\%)$ range.

 **R3 Level: Medium repair quality.**

Note on visual scale: R3 uses a three-level scale (High / Medium / Low), as does R4; the half-filled circle above denotes Medium in both. R1 and R2 use a four-level scale (High / Medium / Low / Insufficient) in which the half-filled symbol falls between Medium and Low and is not a formal band — it appears only in the positioning comparison table (Section 2.2), which uses five levels for cross-tool comparison. The per-requirement evaluation sections each define their own scale in the accompanying threshold table.

Most fixes target the right issue but may use generic patterns or need minor edits before applying.

Limitations of R3 assessment. All repair assessments were performed by a single reviewer on 25 samples. Inter-rater reliability was not measured, which limits the generalizability of the quality ratings. A two-rater protocol with Cohen's κ would strengthen R3 evidence in future evaluations. Additionally, the sample size is small relative to the full dataset; expanding the reviewed set would improve confidence in per-category fix rates.

5.6.4. Auto-Applicable Rate

The 25-sample manual review above operates on a generous bar: a repair is scored "semantically correct" if the prose or code adequately describes the right fix strategy. A stricter, fully-automated bar is whether the `suggestedFix` string is itself syntactically applicable code — i.e. whether a developer could accept the IDE quick fix without rewriting the suggestion. The harness reports this as the *auto-applicable rate* (Section 5.2).

Repairs are produced by a dedicated stage-2 call per detected finding (mirroring the production extension's split between detection and repair generation in

`src/analyzer.ts`). The repair call uses Ollama's structured-output schema with shape `{code: string, language: 'javascript' | 'typescript'}`, and the system prompt instructs the model to "Return only executable code in `code`; do NOT include prose, explanation, or markdown fences." The schema therefore constrains the field shape rather than just its type, so prose paraphrases of the fix cannot satisfy the contract.

Across the full `qwen3:8b+RAG` evaluation (279 stage-2 repairs from 213 vulnerable evaluations under three-pass consensus and the inter-run confidence gate at $\geq$ 0.67), **252 (90.32 %) auto-applied** cleanly through the syntax validator. The denominator is 279 rather than the 297 `stage2CallsTotal` reported in the run JSON because the validator counts only stage-2 outputs that emit a non-empty `suggestedFix` field (`repair-validator.js`, `aggregateAutoApplicable`); 18 of the 297 successful stage-2 calls returned an empty or absent fix string and are therefore excluded from the rate computation rather than counted as auto-applicable failures. Under the stricter convention that scores empty fixes as failures, the rate becomes 252/297 = 84.85 %; both conventions are reported here so the reader can pick the bar that matches their workflow (auto-applicability of *generated* fixes vs. auto-applicability across *all* stage-2 invocations including model abstentions). The 27 remaining rejections are all real fragments of code that fail to parse for a structural reason: mid-statement truncations by `num_predict` (e.g. `connection.query(sql, [user, pass], (error, results) => {` with no closing brace) and unterminated regular-expression literals. No rejections come from prose-language openings.

Interpretation. The auto-applicable rate measures whether the IDE quick-fix surface could insert each repair without further editing — a deterministic alternative to the similarity-based metrics (cosine, BLEU, CodeBERTScore) that SecRepair [54, §III-C3] identifies as "fundamentally inadequate" for security correctness. The 9.68 % residual is a model-emit defect (truncation, unterminated literal), not a prompting defect. The thesis reports both the manual review (*decision-support quality*: prose fixes that describe the right strategy count) and the auto-applicable rate (*automation quality*: only IDE-insertable fixes count), so a reader can pick the bar that matches their workflow.

5.7. R4: Usability

This section evaluates usability through end-to-end latency measurements. The thresholds are defined in Section 2.1.4. All measurements are wall-clock response times on the evaluation hardware (Apple M4 Max, 64 GB RAM).

5.7.1. Latency Across Configurations

Stage-1 detection latency is bounded by three reference points across the LLM sweep: the fastest LLM configuration `gemma3:1b+RAG` runs at median 1 019 ms (mean 1 454 ms, p95 5 672 ms, p99 6 554 ms); the headline `qwen3:8b+RAG` runs at median 2 216 ms (mean 1 892 ms, p95 4 327 ms, p99 5 106 ms); the slowest LLM configuration `codellama+RAG` runs at median 3 529 ms (mean 5 251 ms, p95 10 099 ms, p99 16 005 ms). The seven configurations between these endpoints lie monotonically between the two extremes when sorted by median, although the ordering does not strictly track model size: `gemma3:4b` LLM-only sits at 2 550 ms median, slower than the larger `qwen3:8b` LLM-only at 1 534 ms. SAST baselines are orders of magnitude faster — semgrep at 63 ms median, codeql at 182 ms, ESLint-security at 1 ms — because each is a single deterministic invocation. The detailed per-configuration latency statistics are tabulated in Appendix Table B.11, and the complete ten-configuration breakdown in Appendix B.4 (Table B.3).

Headline configuration. `qwen3:8b+RAG` runs at stage-1 detection median 2 216 ms (mean 1 892 ms, p95 4 327 ms). The mean sits below the median because the per-call latency distribution is bimodal: 120 of the 303 stage-1 calls (39.6 %) return an empty findings array `[]`, of which 103 complete in under 500 ms (a short-output decode path), while the remaining 183 calls produce non-empty JSON findings and run in the 1.5–3.5 s steady-state band. The median therefore better describes the latency a developer experiences *when* the model has something to flag, while the mean is pulled toward the fast empty-response cluster. The pipeline is split into two Ollama calls: stage-1 emits detections only ({message, type, line range, severity}), and a stage-2 call per detected finding produces a structured `{code, language}` repair (median 3 320 ms per fix, p95 6 487 ms recomputed per case). Stage 2 cost is incurred only when a finding triggers a quick-fix surface, so it does not appear in the inline-diagnostics latency that users see when typing.

SAST baselines remain orders of magnitude faster than every LLM configuration. Among LLM configurations the headline `qwen3:8b+RAG` sits in the upper-middle of the LLM range; smaller models such as `gemma3:1b+RAG` (1 019 ms) are noticeably faster but trade detection quality. Figure 5.5 compares stage-1 detection median and p95 across configurations.

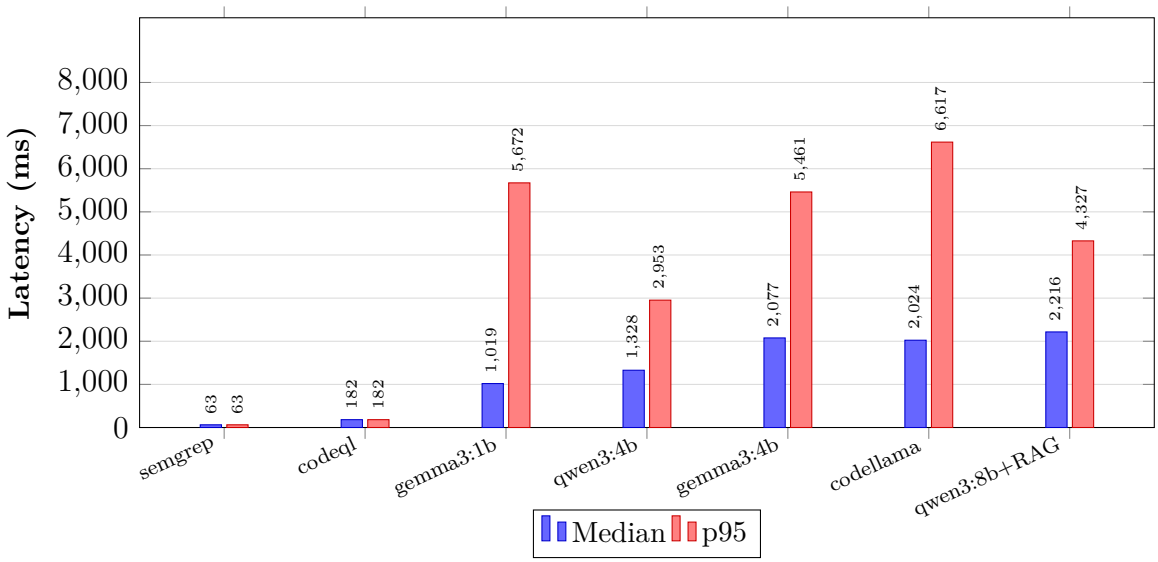

Figure 5.5.: Stage-1 detection median and p95 latency across configurations. SAST baselines are orders of magnitude faster than every LLM configuration; among LLM configurations the headline `qwen3:8b+RAG` sits in the upper-middle of the range, while smaller models such as `gemma3:1b+RAG` trade detection quality for sub-1.1 s median response. SAST p95 columns equal the median because each per-snippet baseline run is a single invocation.

5.7.2. Latency Component Breakdown

End-to-end latency has four components: prompt construction, RAG retrieval (if enabled), LLM inference, and response parsing with diagnostic rendering. The harness records two of these directly — `inferenceTimeMs` (the Ollama call) and `parseTimeMs` (the response parser). Across all 303 stage-1 evaluations of `qwen3:8b+RAG`, parse time was a median of 0 ms and never exceeded a handful of milliseconds; inference time accounts for more than 99% of total response time. RAG retrieval is included inside `inferenceTimeMs` because retrieval is interleaved with the prompt-construction step on the harness side; its 60–120 ms share is small relative to the inference cost.

The implication is clear: latency improvements must come from prompt-side reductions, model selection, hardware upgrades, or architectural changes (streaming partial results). The two-stage pipeline is the dominant architectural decision: stage-1 detection is what users see when typing (median 2 216 ms), while the heavier stage-2 repair generation runs only on demand when a finding triggers the quick-fix surface.

5.7.3. Resource Usage

The harness-side process is intentionally lightweight: harness-side CPU totals a median of 3 ms per inference and peak RSS stabilizes around 87 MB, both negligible relative to the wall-clock latency. The dominant resource cost is Ollama's own footprint — approximately 6–8 GB RAM for an 8B model at 4-bit quantization plus one CPU/GPU core during the prefill and decode passes. These measurements confirm that inference dominates and that any latency improvement must come from model selection, prompt reduction, or hardware, not from the orchestration layer.

5.7.4. Latency Feasibility for IDE Workflows

Table 5.1 maps the measured medians onto three IDE workflow bands.

Table 5.1.: Latency feasibility for IDE workflow modes.

Workflow Mode	**Target**	**Viable Configurations**
Real-time inline	$\leq$ 500 ms median	SAST baselines only
Interactive inline	$\leq$ 1.5 s median	`gemma3:1b+RAG` (1 019 ms), `qwen3:4b` (1 328 ms)
On-demand / audit	$\leq$ 5 s median	All LLM configurations except `codellama+RAG` (3 529 ms median, p99 16 s); the headline `qwen3:8b+RAG` stage-1 detection at 2 216 ms median sits comfortably in this band

The best-accuracy configuration (`qwen3:8b+RAG`, stage-1 detection median 2 216 ms) sits in the on-demand band, comfortably inside the 5 s budget. The quick-fix repair stage adds median 3 320 ms when a finding triggers it, also within on-demand. For latency-sensitive interactive inline use, the smaller `gemma3:1b+RAG` (1 019 ms) trades detection quality for sub-1.5 s response. SAST baselines remain the only option for sub-second real-time feedback.

5.7.5. Level Mapping

Based on the R4 evaluation scale (Table 2.6), the best configuration (`qwen3:8b+RAG`) maps as follows:

- Stage 1 detection median = 2 216 ms ($\leq$ 5 s; within the High threshold for the *on-demand / audit* band).
- p95 = 4 327 ms (still $\leq$ 5 s, so the slow-tail behavior does not break the High band).

- For full-file scans (multiple functions), aggregate latency scales roughly linearly and typically reaches 8–20 s, falling in the Medium range.

The High rating applies to the on-demand / audit workflow band (≤ 5 s per function, per Table 5.1). The stricter *interactive-inline* budget (≤ 1.5 s) is only met by the smaller `gemma3:1b+RAG` (1 019 ms) and `qwen3:4b` (1 328 ms) configurations, both at a detection-quality cost relative to the headline.

R4 Level: High usability (on-demand / audit band)
Medium usability (full-file scans).

Individual function analysis completes within the on-demand threshold for stage-1 detection (2 216 ms median). Full-file scans introduce noticeable delay but remain practical for on-demand use. Stage 2 quick-fix repair adds median 3 320 ms when a finding triggers it, also within the on-demand band.

5.8. R5: Privacy

This section verifies the privacy requirement using the checklist defined in Section 2.1.5 against the threat model summarized in Section 1.3.2. Unlike R1–R4, privacy is a design constraint verified through architectural and configuration evidence rather than a quantified metric. Table 5.2 shows the verification outcome for each privacy criterion; the supporting evidence and threat-model coverage follow.

Table 5.2.: Privacy verification results for R5.

	Privacy Criterion	**Status**
1	LLM inference runs locally via Ollama. No source code is sent to cloud APIs.	✓
2	RAG vector store and embeddings are stored locally. No code-derived data leaves the machine.	✓
3	Analysis prompts and results stay on localhost. No telemetry or external transmission.	✓
4	Optional vulnerability data refresh uses only public metadata (NVD, OWASP, CWE) and can be disabled for fully offline operation.	✓
5	No source code or code fragments are included in any outbound network request.	✓

Local inference and storage. All LLM calls use the Ollama HTTP API on `localhost:11434`; the extension configuration enforces this default and the evaluation ran entirely without external network access for inference. The RAG vector store (HNSWlib) and all embeddings persist in the VS Code workspace or global storage directory — no code-derived vectors are transmitted externally. The extension carries no analytics, telemetry, or crash-reporting libraries, so analysis workflows make no outbound connections. The optional CVE/CWE/OWASP metadata refresh (`vulnerabilityDataManager.ts`) fetches only public metadata from official APIs and is gated behind `codeGuardian.enableExternalDataFetch` (default `false`); source code is never included in these requests.

Automated egress and signed corpus. An egress harness (`evaluation/privacy/test-egress.js`) monkey-patches Node's socket entry points and records every outbound attempt; the test passes only if every connection targets a loopback host. A companion source-leak detector builds a SHA-256 sliding-window index over the input dataset and intercepts `Socket.write` to flag non-loopback code transmission. On the locked configuration both report zero non-loopback connections, consistent with the observational `tcpdump` capture. The RAG corpus is protected at load time by an Ed25519-signed SHA-256 manifest (`src/corpusVerifier.ts`), so a tampered or forged corpus is rejected before any content reaches the LLM prompt.

Prompt-injection resistance. The defense against prompt injection is structural. Analysis instructions live in the system role; user code is delivered in the user role and treated as untrusted data. JSON-mode decoding plus the structured-output schema constrain Stage 1 output to a fixed shape, leaving no free-text channel through which an injected directive could be echoed or a finding suppressed. The single-turn design removes the multi-turn escalation path injection attacks typically rely on.

An adversarial harness (`evaluation/privacy/prompt-injection-test.js`) validates this defense over a 12-case corpus covering nine pure-injection variants and three cases that pair a real vulnerability with an instruction to suppress it. On the locked configuration (`qwen3:8b`, `temperature=0`): **leakFreeRate = 100 %** (12/12; one-sided 95 % binomial lower bound 77.9 %) — no injected directive was echoed; **detectionSurvivesRate = 100 %** (3/3; lower bound 36.8 %) — every real vulnerability was still flagged despite the paired suppression instruction; **schemaValidRate = 83.3 %** (10/12) — two pure-injection cases produced empty responses rather than schema-violating content, a benign bail-out rather than a compliance failure. No system-prompt content was observed in any response. The 12-case corpus is small; the headline 100 % rates therefore indicate *no observed leak under the tested vectors*, not a hardware-independent guarantee.

Comparison with cloud tools. Cloud-hosted security tools (GitHub Copilot, Snyk Code) transmit source code to external servers, require internet access, collect telemetry, and do not work air-gapped. Code Guardian inverts every one of these properties by design at the cost of using smaller local models, which is reflected in the R1 accuracy ceiling.

5.8.1. Level Mapping

All five privacy criteria are met.

✓ **R5 Level: Pass.** All privacy criteria verified.

Code Guardian keeps all analysis local. Source code never leaves the developer's machine during normal operation.

5.9. Comparative Analysis

This section consolidates the per-requirement results from Sections 5.4–5.8 into a single compliance overview, draws the cross-requirement picture, and positions the measurements against the prior work surveyed in Chapter 2.

5.9.1. Requirement Compliance Overview

Table 5.3 maps the headline configuration (`qwen3:8b+RAG`) to the evaluation levels defined in Chapter 2.

Table 5.3.: Requirement compliance summary for Code Guardian (`qwen3:8b+RAG`).

Req	Level	Rating	Key Evidence
R1	◔	Medium	Canonical F1 71.43% / precision 72.46% / recall 70.42% / FPR 10.00%. With confidence gate $\geq$ 0.67: F1 74.07%, precision 78.13%. Three of four sub-metrics meet the High threshold; F1 sits just below it.
R2	●	High (under fixed-seed decoding)	JSON parse 100%, inter-run detection / label / severity agreement 100% under the locked-seed protocol (Section 5.5). Output is fully stable for the evaluated configuration; the consensus filter would only contribute discriminative signal under per-run seed rotation.
R3	◑	Medium (manual review primary; 90.32% auto-applicable as secondary)	Manual review of 25 samples: 68% correct repairs (primary R3 bar per Section 2.1.3); combined correct-repair rate 68% falls in the Medium [50%, 75%) band. Auto-applicable rate on full $n = 279$ stage-2 repairs: **90.32 %** (252/279) as a secondary automation-quality signal; the 27 residual rejections are mid-statement truncations or unterminated regex literals, not prose.
R4	●	High (per-function; on-demand/audit band per R4 scale)	Stage 1 detection median 2 216 ms per function meets the R4 High threshold ($\leq$ 5 s for on-demand analysis). The interactive-inline budget ($\leq$ 1.5 s, Table 5.1) is not met; smaller models (`gemma3:1b+RAG`, 1 019 ms) are indicated for that band. Stage 2 quick-fix repair median is 3 320 ms per fix, incurred only when a finding triggers a repair. Inference dominates total latency by more than 99%.
R5	✓	Pass	All 5 privacy criteria verified. Code never leaves the machine.

5.9.2. Result Analysis

The strongest signal across the evaluated configurations is the gap between obvious-sink categories and semantic categories. The headline configuration reaches near-complete canonical recall on lexically-explicit sinks (XSS, command injection, path traversal, prototype pollution, weak cryptography, hardcoded credentials, LDAP injection, SSRF, and SQL injection) and zero recall on improper authentication, input validation, and weak encryption (Section 5.4). The qualitative analysis in Section 5.4.4 shows that the improper-auth and weak-encryption gaps are taxonomy

artifacts (the model emits sibling canonical categories for the same code), while input-validation is a genuine reasoning gap on real-world library code. Contextual reasoning about lexically-explicit sinks works very well; reasoning about the absence of validation does not.

False-positive control is one of the cleanest results. The headline configuration holds sample-level FPR in the low-noise band under three runs per secure sample, and the inter-run confidence gate preserves that level while lifting precision. Most other LLM configurations still flag every secure case at least once, even with the gate, which means the control is specific to the strongest model in the set rather than a generic property of multi-pass consensus. This is consistent with the general observation that smaller instruction-tuned models tend to be less well calibrated than larger ones [14].

Latency results validate the design hypothesis that prefill-token cost dominates inference on local hardware. The pipeline is split into a stage-1 detection call and an on-demand stage-2 repair call, so the cost of generating a structured `{code, language}` repair is paid only when a finding triggers a quick-fix surface. Stage-split telemetry confirms inference accounts for more than 99% of the total response time (Section 5.7); there is no realistic gain to be made on the parsing side. The remaining latency budget is therefore best spent on either smaller models (with a quality cost) or on architectural changes such as streaming partial JSON output.

The repair-quality result is shaped by the structural repair contract: a dedicated stage-2 call per detected finding asks the model to fill `{code, language}` via Ollama's structured-output schema, and the system prompt forbids prose, explanation, or markdown fences. Under this contract more than nine in ten generated repairs in the headline configuration pass the syntactic auto-applicability check; the residual rejections are not prose paraphrases of the fix but real code fragments that fail to parse for structural reasons (mid-statement truncation by `num_predict`, unterminated regex literals). The achievable auto-applicable rate is therefore bounded above by structural-emit defects of the underlying model rather than by the prompting strategy.

5.9.3. Cross-Requirement Synthesis

The five requirements are not independent in operation: an IDE integration must satisfy all five at once for the system to be useful in practice. Three interactions deserve direct attention.

Latency × Accuracy. The headline configuration sits above the strict interactive-inline latency threshold (Section 5.7). Treated as a single number this looks like a modest miss, but it has a concrete operational consequence: the highest-quality

local configuration is on-demand, not interactive-inline, while a smaller model (`gemma3:1b+RAG`) sits inside the inline budget at lower recall. The user-facing implication is that *a single model cannot serve both inline and audit roles for this scope on this hardware*: a smaller model fits the inline budget at lower recall, while the headline configuration belongs in the on-demand band, motivating a role-split deployment.

Repair quality × Detection. The 90.32 % auto-applicable rate is conditioned on detected findings, so its value is gated by R1 recall: missed vulnerabilities never receive a repair. The combined system therefore has a recall ceiling at the R1 level (about 71 % on the curated headline) and a fix-applicability ceiling at the R3 level. The two compose multiplicatively in the user's experience — of every ten exploitable issues a developer might want surfaced and fixed by the tool, R1 catches roughly seven and R3 produces directly applicable fixes for roughly six. This is the operationally meaningful number, not the per-requirement marginals.

Privacy × Everything Else. The R5 privacy guarantee (zero egress, signed corpus integrity) is binary in the way the others are not: a model that exfiltrates code once is permanently disqualified for sensitive projects regardless of its R1 or R4 numbers. Privacy is therefore treated as a fixed precondition rather than as a tunable axis. The R1/R3/R4 numbers above are reported *within* the privacy envelope; trading R5 for marginal R1 or R4 gains (e.g. a cloud LLM at lower latency) is an operationally different system, not a better point on the same Pareto frontier.

When Code Guardian is and is not useful. The function-level scope adds clear value when the dangerous API is lexically present in the analyzed unit: SQL string concatenation, `innerHTML` assignments, `exec` on user input, weak hash algorithms, hardcoded credentials, prototype pollution sinks. On the curated corpus these classes hit 90–100 % canonical recall (Section 5.4). The scope does *not* add value when the vulnerability requires reasoning about code that is not in the analyzed unit: cross-file taint flow, framework-level invariants (the JWT `algorithms` option, an Express access-control wrapper), absent validation, or the version constraint that resolves a CVE. The external-corpus result (F1 63.64 %, ∼10-point gap to curated) and the inline-mode NodeGoat result (3 of 7) make this concrete: when frame conditions matter, function-scope LLM analysis under-reads the situation. The audit-mode AST + LLM hybrid (Section 4.2.5) is the architectural response for whole-project work, lifting the same NodeGoat result to 6 of 7 by adding deterministic AST sink detection alongside the semantic LLM stage.

5.9.4. Comparison with Related Work

Three benchmarks anchor the comparison. IRIS [52] reports a false discovery rate of 84.82% (precision approximately 15%) for GPT-4 on whole-repository Java analysis, with 55 of 120 vulnerabilities detected. CWEval [53] establishes outcome-driven evaluation as a methodological benchmark and reports that LLMs achieve more than 70% F1 with about 80% precision when given context-rich prompts. SecRepair [54] reports a 12% improvement on automated similarity-based repair-quality metrics but explicitly admits in its own analysis that those metrics are "fundamentally inadequate" for security correctness assessment.

Code Guardian's measured 78.13% precision (with the inter-run confidence gate) on a local 8B-parameter model lands in the same precision class as the cloud-LLM systems cited above. This is not a head-to-head dominance claim — the datasets and analysis scopes differ — but it is a defensible counterpoint to the assumption that local models cannot reach cloud-class precision on focused vulnerability detection. The path that gets there is the explicit pipeline of canonical type matching, multi-pass consensus, inter-run confidence gating, and a tightened structured-output contract, rather than raw model scale.

The honest negative result on RAG also aligns with the literature. RESCUE [66] reports that conventional retrieval augmentation (SecCoder, the closest baseline in their study) does not significantly improve secure code generation, and proposes a more sophisticated multi-faceted retrieval to close the gap. The thesis's RAG ablation results show `qwen3:8b` benefitting (+3.91 F1 with the canonical matcher: 67.52 % → 71.43 %) while `codellama` collapses (−29.89 F1 with RAG enabled). The collapse is driven by the additional retrieval context interfering with `codellama`'s output structure: latency nearly doubles, recall drops by twenty points, and parse failures increase. The conclusion that RAG must be calibrated per model rather than enabled by default is therefore strongly supported, both by the rerun data and by the related literature.

The auto-applicable repair rate complements SecRepair's characterization of automated repair-quality metrics. SecRepair argues that similarity-based metrics such as cosine similarity, BLEU, and CodeBERTScore are "fundamentally inadequate" for security correctness; this thesis offers a deterministic, fleet-wide alternative by pairing a structural repair contract (`{code, language}`) with a syntactic-applicability bar, and reports 90.32 % on the full $n = 279$ sample (Section 5.6.4). The auto-applicable rate is therefore positioned as a complement to manual review: it scales without rater effort, is reproducible across runs, and addresses one specific axis of the inadequacy SecRepair identifies.

To summarize, the headline configuration reaches a precision band reported for cloud GPT-4 on context-rich function-level tasks with a local 8B-parameter model, by combining canonical type matching, multi-pass consensus, inter-run confidence gating,

and structural output contracts for both detection and repair. The detector remains weaker on semantic categories that require reasoning about absent code, where the gap is structural rather than capability-bound. The structural `{code, language}` repair contract delivers a 90.32 % auto-applicable rate, with the residual bounded by model-side truncation rather than prompt design. Specific point estimates — detection metrics (Sections 5.4, 5.4.6), auto-applicable repair rate (Section 5.6.4), stage-split latency (Section 5.2), and reproducibility (Appendix B) — are presented in the corresponding sections rather than duplicated here.

5.10. Summary

This chapter evaluated Code Guardian against the five requirements defined in Chapter 2, using a curated 101-case corpus, three runs per sample for inter-run agreement, five Ollama models in both LLM-only and LLM+RAG configurations, and three SAST baselines (Semgrep, CodeQL, ESLint) scored under identical canonical-matching logic. The headline configuration `qwen3:8b+RAG` reaches F1 71.43 % (precision 72.46 %, recall 70.42 %, FPR 10.00 %) without gating, lifting to F1 74.07 % / precision 78.13 % under the inter-run confidence gate (≥ 0.67); on the held-out 71/30 split it reaches F1 61.11 % with FPR 0 %. R2 consistency reaches 100 % JSON parse success and 100 % inter-run detection agreement under the locked-seed decoding protocol (the consensus filter therefore does not contribute discriminative signal in this corpus and would only become active if the per-run seed were rotated); R3 repair quality reaches 90.32 % auto-applicable rate on the full $n = 279$ stage-2 output and approximately 68 % combined manual-review correctness on a 25-sample subset; R4 stage-1 detection median is 2 216 ms (p95 4 327 ms) on the locked Apple M4 Max profile, comfortably inside the on-demand 5 s band; R5 privacy is verified via the architectural checklist plus a 12-case prompt-injection harness with 100 % leakFreeRate. Statistical rigor includes 95 % exact-binomial CIs on small-n rates and McNemar tests with Bonferroni correction across the five RAG-vs-no-RAG model comparisons; only the `codellama` RAG-degradation survives correction. Together the measurements indicate that practical local vulnerability assistance without code exfiltration is achievable under the evaluated configurations and answer the three research questions stated in Chapter 1; Chapter 6 consolidates the findings, names the residual limitations, and discusses deployment implications.

6. Conclusion

This chapter consolidates the headline outcomes, names the residual limitations and threats to validity, and states the deployment implications. The detailed per-requirement evidence is reported in Chapter 5; the cross-cutting interpretation is developed in Section 5.9.

6.1. Summary of Contributions

Code Guardian: a privacy-preserving secure-coding assistant. The thesis delivers a VS Code extension for on-device JavaScript and TypeScript vulnerability analysis. All inference runs through a local Ollama instance; an egress gate blocks every non-loopback connection by default, and the optional retrieval store reads from a signed local corpus of CWE, OWASP, and CVE guidance.

Two-stage detection-then-repair pipeline. Stage 1 emits structured JSON findings under JSON-mode decoding with a fixed schema. Stage 2 generates one { `code`, `language` } repair per finding under developer-controlled apply or preview.

Multi-pass consensus filter and confidence gate. Seeded passes score each finding by inter-pass agreement; an opt-in threshold suppresses unmatched findings before stage 2.

Hybrid AST+LLM audit pipeline. A deterministic AST sink scanner runs in parallel with the LLM and unions findings, lifting whole-project recall on OWASP NodeGoat from 3/7 inline to 6/7 in audit mode. Of the six matched entries, three are caught by the AST sink scanner alone, two by the LLM alone, and one is corroborated by both stages, so each detector contributes findings the other does not see (Section 5.4.9). NodeGoat is a single project ($n = 7$ documented entries) plausibly present in the model's pre-training data, so this result is a case study, not a population estimate. The audit pipeline is invoked only by explicit whole-project scans and sits in the on-demand latency band.

Reproducible evaluation harness. The repository ships a documented harness with the 101-case curated corpus, a 15-case external corpus drawn from NodeGoat, Juice Shop, and three named CVEs, three SAST baselines (Semgrep, CodeQL, ESLint), and a deterministic per-repair syntax validator that scales R3 beyond manual review.

Requirements-driven design. Five requirements shaped the system end-to-end: R1 accuracy, R2 consistency, R3 repair quality, R4 usability, and R5 privacy. The

pipeline, prompt contracts, evaluation harness, and deployment profiles are all organised around these five axes, and the per-requirement evidence is consolidated in the cross-requirement synthesis (Section 5.9.3).

6.2. Key Findings

Local vulnerability detection is feasible inside the IDE. On the held-out 30-case TEST split (Section 5.4.7), the `qwen3:8b+RAG` configuration with the inter-run confidence gate selected on TRAIN reaches F1 61.11 % at FPR 0 %, which is the externally-valid figure carried into deployment-profile reasoning. On the curated 101-case full corpus the same configuration reaches F1 74.07 % with precision 78.13 % (Section 5.4); the 17.7-point TRAIN–TEST gap is consistent with curated-corpus optimism and bounds how far the in-sample headline generalizes. The function-level scope works well when the dangerous API is lexically present, less well when reachability depends on frame conditions outside the scope.

Retrieval grounding is per-model, not universal. Retrieval augmentation lifts the strongest 8B-parameter model and degrades smaller and code-specialized ones; only the `codellama` degradation survives Bonferroni correction across the five paired McNemar tests. The result is consistent with RESCUE's report that conventional RAG does not significantly improve secure code generation [66].

Quality-oriented modes belong on demand, not always-on. Function-level scoping, debouncing, and caching make local analysis usable in selected workflows, but the highest-quality configuration's stage 1 median exceeds the strict interactive-inline threshold. Inference accounts for more than 99 % of the response time, so further latency improvements require model or architectural changes (Section 5.7).

Repair contract scales to the full evaluation set. The structural `{code, language}` repair contract reaches a 90.32 % auto-applicable rate on the full $n = 279$ stage-2 output (Section 5.6). Auto-applicability measures only whether the IDE quick-fix surface can insert the suggestion without further editing; semantic correctness is a strictly stronger bar and is reported separately as 17 / 25 ($\approx 68\,\%$, 95 % Wilson CI $[48\,\%, 83\,\%]$, single rater) under a 25-sample manual review. The residual auto-applicable rejections are model-side structural defects (mid-statement truncation, unterminated regex literals), not prose paraphrases of the fix.

6.3. Limitations

Scope and context depth. The system handles common vulnerability patterns but remains limited for deep cross-file reasoning without full static data-flow analysis. Three semantic categories register zero recall under the canonical taxonomy. A

regex-based import / sink resolver provides lightweight cross-snippet hints but does not surface new instances in those categories.

Evaluation representativeness and statistical power. The 101-case corpus aligns with contemporary curated benchmarks (CWEval at 119 tasks, IRIS at 120 vulnerabilities) but does not generalize to production repository scale. The small sample size limits statistical power: only the `codellama+RAG` degradation survives Bonferroni correction, and the 95 % exact-binomial CI for the headline 10 % FPR is $[2.1\%, 26.5\%]$ on $n = 30$ secure cases. The 15-case external corpus and the standalone NodeGoat real-world run both draw from public sources (NodeGoat, Juice Shop, three named CVEs) almost certainly present in pre-training data, so the headline numbers on these cases (external corpus F1 63.64 %; NodeGoat audit-mode 6/7) may reflect partial memorization and should be read as case studies rather than population estimates.

No head-to-head comparison against cloud LLMs or commercial SAST. The evaluation compares against three open-source SAST baselines (Semgrep, CodeQL, ESLint) but not against cloud LLMs (GPT-4, Claude) or commercial SAST suites (Snyk Code, SonarQube). Running cloud LLMs on the corpus would violate the system's own threat model (R5 forbids code exfiltration), and commercial SAST products require source upload or proprietary licensing. The headline numbers reported in this thesis therefore position Code Guardian against the deterministic SAST baselines and across local-LLM configurations, not against cloud-hosted alternatives.

Curator bias and taxonomy construct validity. The curated 101-case corpus and the 26-class canonical taxonomy were both authored by the candidate, and both shape the headline numbers. Three mitigations are in place: every case is sourced from an authoritative public corpus and aligned with CWE IDs where applicable; the held-out 71/30 split (Section 5.4.7) keeps threshold tuning off the TEST cases; and the taxonomy is pinned by a snapshot test, with per-canonical-category counts emitted in every run JSON so an external reviewer can re-bucket the raw findings under a different taxonomy and re-score.

Repair correctness bar. Repairs are model-generated and may change behavior. The pipeline includes a syntactic auto-applicability check that scales to the full evaluation set, but this is a strictly weaker signal than functional verification: a parse-clean fix can still be semantically wrong. The 25-sample manual review was performed by a single reviewer; no inter-rater reliability was measured.

Hardware profile and external generalization. All measurements come from a single Apple M4 Max profile (16 cores, 64 GB RAM) with deterministic decoding (`temperature=0`, fixed seed). Local inference on GPU hardware can introduce minor non-determinism from floating-point scheduling, and performance on lower-end hardware may differ. Reported values are descriptive measurements under a documented environment, not hardware-independent constants. Developer satisfaction, trust,

and adoption were not evaluated under a controlled walkthrough; R4 evidence is latency-only.

Configuration selection is not pre-registered. The headline configuration (`qwen3:8b+RAG`, three runs, gate ≥ 0.67) was identified by post-hoc evaluation across ten model×mode configurations rather than declared before the sweep. The held-out 71/30 split addresses threshold tuning on the same cases as the headline but does not address selection of the model and retrieval mode themselves. Per-configuration metrics are reported alongside the headline (Section 5.4) so the selection is auditable.

Deviations from the approved task. The approved task specified evaluation on Juliet and OWASP Benchmark, but neither targets JavaScript: Juliet is published by NIST in C/C++ and Java only, and the OWASP Benchmark Project is a Java application. The delivered evaluation substitutes two CWE-mapped JavaScript corpora — the 101-case primary corpus (71 vulnerable + 30 secure) and the 15-case external corpus — preserving the property the named benchmarks provide (CWE-mapped, vulnerability-tagged samples drawn from authoritative sources) in the language the detector targets. Under the headline configuration the external corpus reaches F1 63.64 % versus the curated 74.07 %, an approximately 10-point gap concentrated in framework-level weakness classes that the function-level scope does not see. The substitution was discussed with the supervisor during the implementation phase.

6.4. Implications for Practice

Table 6.1 summarizes deployment profiles indicated by the measured results on the locked Apple M4 Max profile (Appendix B). The profiles describe what the measurements support; they are not unconditional recommendations and require re-validation on different hardware or different corpus characteristics.

Table 6.1.: Deployment profiles indicated by measurement on the locked hardware profile.

Use case	Config	Advantage	Main risk	Alert noise management
Fast inline	`gemma3:1b+RAG`	Lowest LLM FPR among small models (23.33%); fastest median (~1 019 ms)	Low recall (40.85%)	Optional lightweight hints with light triage
High-recall reference (not a deployment profile)	`qwen3:4b` (LLM-only)	Highest LLM recall (78.87%); F1 66.14%; ~1.3 s median	FPR 90% places this configuration below the R1 *Insufficient* floor	Reported as a recall ceiling reference; not recommended for deployment
Best overall local run	`qwen3:8b+RAG`	F1 71.43%, precision 72.46%, FPR 10.00%, stage-1 detection median 2 216 ms; with confidence gate $\geq$ 0.67 precision rises to 78.13% (F1 74.07%)	Inference cost dominates more than 99% of latency	Indicated default for quality-oriented local analysis under this hardware profile
Hybrid low-noise	Semgrep + `qwen3:8b+RAG` (opt-in fusion)	Promotes corroborated findings to confidence 1.0 / `hybrid` source; uses Semgrep's 6.67% FPR as a high-precision overlay	Fusion fires only for the small share Semgrep itself flags (recall 12.68%)	Use SAST corroboration as visible-confidence overlay

Code Guardian is decision support, not an autonomous security verifier. False negatives, false positives, and occasional label mismatches remain possible, so findings and repairs should stay reviewable artifacts under developer control, complemented by conventional testing and security review.

6.5. Conclusion

Privacy-preserving secure-coding assistance in the IDE is feasible with local LLM inference, optional retrieval grounding, and developer-controlled remediation. Local

LLM modes achieve much higher vulnerable-case recall than SAST baselines, while deterministic tools retain better false-positive control and lower latency; retrieval augmentation improves some models and degrades others. The measurements indicate that practical local vulnerability assistance without code exfiltration is achievable under the evaluated configurations; production use still requires explicit policy choices for model profile, acceptable FPR, and when to prefer lightweight inline feedback over audit-oriented analysis.

7. Future Work

Building on the contributions and limitations above, several directions stand out for follow-on work.

Stronger context modelling and safer repair. Function-level analysis misses some multi-file vulnerabilities, and the regex-based import / sink resolver provides only lexical hints (Section 4.1). The natural next step is to replace it with lightweight TypeScript-aware taint analysis, exposed to the LLM as additional context fields rather than as competing detections. On the repair side, the syntactic auto-applicability check (Section 5.6.4) should be extended with a TypeScript type-check pass, a local test-suite run on the patched buffer, and a re-analysis of the patched code to verify the original finding is gone and no new finding was introduced. Code-property-graph approaches [24] and structured neural-repair work [62] provide reference points for both extensions.

Calibrated confidence and adversarial robustness. The current per-finding confidence is the inter-run agreement proxy. Calibrated confidence [49, 50] and explicit abstention prompting would make the gate work on smaller models that can be self-consistently wrong. The 12-case prompt-injection harness (Section 5.8) should be extended to several hundred cases and to retrieval-poisoning attacks against the signed corpus, with prompts treating project code consistently as untrusted input.

Broader corpora and pre-registered selection. The 101-case dataset suits controlled iteration but is too small for broad claims. Future evaluation should run against larger externally-authored JS/TS-applicable corpora — DiverseVul, Big-Vul, PrimeVul, CyberSecEval, and LLMSecEval [43, 45] — and JavaScript ports of the OWASP Benchmark and NIST Juliet suites if released. The headline configuration was identified post-hoc across ten model×mode configurations; a future cycle should pre-register the selection protocol on a development corpus before the test corpus is touched.

Developer studies and trust calibration. R4 is currently latency-only. Workflow impact should be validated with developer studies measuring usability (SUS), cognitive load (NASA-TLX), and time-to-fix [81, 82], with inter-rater reliability (Cohen's κ) reported on the R3 manual review. Whether measured accuracy levels translate to adoption depends on developer trust and alert fatigue [7, 8]; open questions include the FPR threshold at which trust collapses irreversibly, whether visible inter-run confidence changes accept rates on borderline findings, and whether AI-assisted secure-coding effects observed in cloud studies [31] persist when the assistant runs locally.

Open research questions.

- **Why only `qwen3:8b` achieves acceptable FPR.** Of the ten configurations evaluated, only `qwen3:8b+RAG` and `gemma3:1b+RAG` hold sample-level FPR below 100 %. Ablations across additional 7B–13B models with and without instruction tuning, and against free-form rather than JSON-mode output, would distinguish parameter-count, instruction-tuning, and structured-output explanations.
- **Structural recall ceiling on absent-code categories.** Three canonical categories register zero recall under the headline configuration. The open question is whether a graph-grounded RAG context (chunks retrieved against a code-property-graph index) lifts recall on absent-validation cases without inflating FPR.
- **Generalization beyond JavaScript/TypeScript.** The two-stage pipeline, AST+LLM hybrid, and signed-corpus mechanism are language-agnostic in design but were instantiated only for JS/TS. Whether headline metrics transfer to Python, Java, and Go under matched local-model configurations, or instead reflect properties of the JS/TS sink ecosystem, requires a controlled multi-language replication.

Bibliography

[1] OWASP Foundation, "Owasp top 10 – 2021," https://owasp.org/www-project-top-ten/, 2021, accessed: 2026-01-24.

[2] MITRE, "Common weakness enumeration (CWE)," https://cwe.mitre.org/, accessed: 2026-01-24.

[3] A. Bessey, K. Block, B. Chelf, A. Chou, B. Fulton, S. Hallem, C. Henri-Gros, A. Kamsky, S. McPeak, and D. Engler, "A few billion lines of code later: Using static analysis to find bugs in the real world," *Communications of the ACM*, vol. 53, no. 2, pp. 66–75, 2010.

[4] B. Chess and G. McGraw, "Static analysis for security," *IEEE Security & Privacy*, vol. 2, no. 6, pp. 76–79, 2004.

[5] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, J. Hilton, R. Nakano, C. Hesse, J. Chen, M. Plappert, M. Beard, C. Voss, A. Radford, and I. Sutskever, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[6] OpenAI, "GPT-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[7] B. Johnson, Y. Song, E. Murphy-Hill, and R. Bowdidge, "Why don't software developers use static analysis tools to find bugs?" in *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, San Francisco, CA, USA, 2013, pp. 672–681.

[8] M. Christakis and C. Bird, "What developers want and need from program analysis: An empirical study," in *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*, Singapore, 2016, pp. 332–343.

[9] N. Carlini, F. Tramer, E. Wallace, M. Jagielski, A. Herbert-Voss, K. Lee, A. Roberts, T. Brown, D. Song, Ú. Erlingsson, A. Oprea, and C. Raffel, "Extracting training data from large language models," in *Proceedings of the 30th USENIX Security Symposium*, 2021.

[10] European Union, "Regulation (eu) 2016/679 (general data protection regulation)," https://eur-lex.europa.eu/eli/reg/2016/679/oj, 2016, accessed: 2026-01-24.

[11] Snyk, "Snyk documentation," https://docs.snyk.io/, accessed: 2026-01-24.

[12] Ollama, "Ollama documentation," https://ollama.com/, accessed: 2026-01-24.

[13] OWASP Foundation, "Owasp top 10 for large language model applications," https://owasp.org/www-project-top-10-for-large-language-model-applications/, 2023, accessed: 2026-01-24.

[14] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, 2023.

[15] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? assessing the security of GitHub Copilot's code contributions," in *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 2022.

[16] M. Monperrus, "A critical review of automatic patch generation learned from human-written patches: Essay on the problem statement and the evaluation of automatic software repair," in *Proceedings of the 36th International Conference on Software Engineering (ICSE)*, 2014, pp. 234–242.

[17] S. K. Card, T. P. Moran, and A. Newell, *The Psychology of Human-Computer Interaction*. Boca Raton, FL, USA: CRC Press, 1983, reprint edition 1991.

[18] J. Nielsen, *Usability Engineering*. San Francisco, CA, USA: Morgan Kaufmann, 1994.

[19] Semgrep, Inc., "Semgrep documentation," https://semgrep.dev/docs/, accessed: 2026-01-24.

[20] GitHub, "Codeql documentation," https://codeql.github.com/docs/, accessed: 2026-01-24.

[21] P. Cousot and R. Cousot, "Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints," *Proceedings of the 4th ACM Symposium on Principles of Programming Languages (POPL)*, pp. 238–252, 1977.

[22] D. Engler, D. Y. Chen, S. Hallem, A. Chou, and B. Chelf, "Bugs as deviant behavior: A general approach to inferring errors in systems code," in *Proceedings of the 18th ACM Symposium on Operating Systems Principles (SOSP)*, 2001, pp. 57–72.

[23] C. Calcagno, D. Distefano, J. Dubreil, D. Gabi, P. Hooimeijer, M. Luca, P. O'Hearn, I. Papakonstantinou, J. Purbrick, and D. Rodriguez, "Moving fast with software verification," in *Proceedings of the 7th NASA Formal Methods Symposium (NFM)*, 2015, pp. 3–11.

[24] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck, "Modeling and discovering vulnerabilities with code property graphs," in *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 2014, pp. 590–604.

[25] S. Arzt, S. Rasthofer, C. Fritz, E. Bodden, A. Bartel, J. Klein, Y. Le Traon, D. Octeau, and P. McDaniel, "FlowDroid: Precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for Android apps," in *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2014, pp. 259–269.

[26] M. Howard and S. Lipner, *The Security Development Lifecycle*. Microsoft Press, 2006.

[27] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[28] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, 2019.

[29] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of Machine Learning Research*, vol. 21, 2020.

[30] R. Khoury, A. R. Avila, J. Brunelle, and B. M. Camara, "How secure is code generated by ChatGPT?" in *Proceedings of the IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, 2023, pp. 2445–2451.

[31] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, "Do users write more insecure code with AI assistants?" in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2023, pp. 2785–2799.

[32] G. Sandoval, H. Pearce, T. Nys, R. Karri, S. Garg, and B. Dolan-Gavitt, "Lost at C: A user study on the security implications of large language model code assistants," in *Proceedings of the 32nd USENIX Security Symposium*, 2023, pp. 2205–2222.

[33] J. Zhang, H. Zhu, H. Bu, H. Wen, Y. Liu, H. Fei, R. Xi, L. Li, Y. Yang, and D. Meng, "When LLMs meet cybersecurity: A systematic literature review," *Cybersecurity*, vol. 8, p. 55, 2025.

[34] Y. Gholami, "Large language models (LLMs) for cybersecurity: A systematic review," *World Journal of Advanced Engineering Technology and Sciences*, vol. 13, no. 1, pp. 57–69, 2024.

[35] E. Basic and A. Giaretta, "From vulnerabilities to remediation: A systematic literature review of LLMs in code security," arXiv preprint arXiv:2412.15004, 2025, preprint.

[36] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, Z. Chen, and S. Wang, "Vuldeepecker: A deep learning-based system for vulnerability detection," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2018.

[37] U. Alon, M. Zilberstein, O. Levy, and E. Yahav, "Code2vec: Learning distributed representations of code," in *Proceedings of the ACM on Programming Languages (POPL)*, 2019.

[38] Y. Zhou, S. Liu, J. K. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[39] M. Allamanis, M. Brockschmidt, and M. Khademi, "Learning to represent programs with graphs," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2018.

[40] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, "Deep learning based vulnerability detection: Are we there yet?" *IEEE Transactions on Software Engineering*, vol. 48, no. 9, pp. 3280–3296, 2022.

[41] B. Steenhoek, M. M. Rahman, R. Jiles, and W. Le, "An empirical study of deep learning models for vulnerability detection," in *Proceedings of the IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 2023, pp. 2237–2248.

[42] J. Fan, Y. Li, S. Wang, and T. N. Nguyen, "A C/C++ code vulnerability dataset with code changes and CVE summaries," in *Proceedings of the 17th International Conference on Mining Software Repositories (MSR)*, 2020, pp. 508–512.

[43] Y. Chen, Z. Ding, L. Alowain, X. Chen, and D. Wagner, "DiverseVul: A new vulnerable source code dataset for deep learning based vulnerability detection," in *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*, 2023, pp. 654–668.

[44] Y. Ding, Y. Fu, O. Ibrahim, C. Sitawarin, X. Chen, B. Alomair, D. Wagner, B. Ray, and Y. Chen, "Vulnerability detection with code language models: How far are we?" in *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, 2025.

[45] M. Bhatt, S. Chennabasappa, C. Nikolaidis, S. Wan, I. Evtimov, D. Gabi, D. Song, F. Ahmad, C. Aschermann, L. Fontana, S. Frolov, R. P. Giri, D. Kapil, Y. Kozyrakis, D. LeBlanc, J. Milazzo, A. Straumann, G. Synnaeve, V. Vontimitta, S. Whitman, and J. Saxe, "Purple Llama CyberSecEval: A secure coding benchmark for language models," 2023.

[46] C. Tony, M. Mutas, N. E. D. Ferreyra, and R. Scandariato, "LLMSecEval: A dataset of natural language prompts for security evaluations," in *Proceedings of the IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*, 2023, pp. 588–592.

[47] L. Weidinger, J. Mellor, M. Rauh, C. Griffin, J. Uesato, P.-S. Huang, M. Cheng, B. Balle, A. Kasirzadeh *et al.*, "Ethical and social risks of harm from language models," *arXiv preprint arXiv:2112.04359*, 2021.

[48] A. Zou, Z. Wang, J. Z. Kolter, and M. Fredrikson, "Universal and transferable adversarial attacks on aligned language models," *arXiv preprint arXiv:2307.15043*, 2023.

[49] S. Kadavath, T. Conerly, A. Askell, T. Henighan, D. Drain, E. Perez, N. Schiefer, Z. Hatfield-Dodds, N. DasSarma, E. Tran-Johnson *et al.*, "Language models (mostly) know what they know," 2022.

[50] C. Spiess, D. Gros, K. S. Pai, M. Pradel, M. R. I. Rabin, A. Alipour, P. Devanbu, B. Ray, and V. Bavishi, "Calibration and correctness of language models for code," in *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, 2025.

[51] Y. Li, M. Xu, X. Li, Y. Zhang, H. Wu, X. Cheng, F. Xu, and S. Zhong, "Everything you wanted to know about LLM-based vulnerability detection but were afraid to ask," arXiv preprint arXiv:2504.13474, 2025, preprint.

[52] Z. Li, S. Dutta, and M. Naik, "IRIS: LLM-assisted static analysis for detecting security vulnerabilities," in *International Conference on Learning Representations (ICLR)*, 2025, also available as arXiv:2405.17238.

[53] J. Peng, L. Cui, K. Huang, J. Yang, and B. Ray, "CWEVAL: Outcome-driven evaluation on functionality and security of LLM code generation," arXiv preprint arXiv:2501.08200, 2025, preprint.

[54] N. T. Islam, J. Khoury, A. Seong, E. Bou-Hard, and P. Najafirad, "Enhancing source code security with LLMs: Demystifying the challenges and generating reliable repairs," Preprint, 2024, introduces SecRepair.

[55] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[56] V. Karpukhin, B. Oguz, S. Min, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

[57] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lombardi, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.

[58] W. Weimer, T. Nguyen, C. Le Goues, and S. Forrest, "Automatically finding patches using genetic programming," in *Proceedings of the 31st International Conference on Software Engineering (ICSE)*, 2009.

[59] Z. Chen, S. Kommrusch, M. Tufano, L.-N. Pouchet, D. Poshyvanyk, and M. Monperrus, "SequenceR: Sequence-to-sequence learning for end-to-end program repair," in *IEEE Transactions on Software Engineering*, vol. 47, no. 9, 2021, pp. 1943–1959.

[60] Q. Zhu, Z. Sun, Y.-a. Xiao, W. Zhang, K. Yuan, Y. Xiong, and L. Zhang, "A syntax-guided edit decoder for neural program repair," in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2021, pp. 341–353.

[61] T. Lutellier, H. V. Pham, L. Pang, Y. Li, M. Wei, and L. Tan, "CoCoNuT: Combining context-aware neural translation models using ensemble for program repair," *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pp. 101–114, 2020.

[62] C. S. Xia and L. Zhang, "Less training, more repairing please: Revisiting automated program repair via zero-shot learning," in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2022, pp. 959–971.

[63] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," in *Foundations and Trends in Information Retrieval*. Now Publishers, 2009, vol. 3, pp. 333–389.

[64] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang, "REALM: Retrieval-augmented language model pre-training," in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020.

[65] G. Mialon, R. Dessì, M. Lomeli, C. Nalmpantis, R. Pasunuru, R. Raileanu, B. Rozière, T. Schick, T. Scialom, X. Suau *et al.*, "Augmented language models: A survey," *arXiv preprint arXiv:2302.07842*, 2023.

[66] J. Shi and T. Zhang, "RESCUE: Retrieval augmented secure code generation," arXiv preprint arXiv:2510.18204, 2025, preprint.

[67] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019.

[68] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.

[69] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *arXiv preprint arXiv:1702.08734*, 2017.

[70] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *Proceedings of the 2017 IEEE Symposium on Security and Privacy (S&P)*, 2017.

[71] L. Niu, S. Mirza, Z. Maradni, and C. Pöpper, "CodexLeaks: Privacy leaks from code generation language models in GitHub Copilot," in *Proceedings of the 32nd USENIX Security Symposium*, 2023.

[72] MITRE, "Common vulnerabilities and exposures (CVE)," https://www.cve.org/, accessed: 2026-01-24.

[73] National Institute of Standards and Technology, "National vulnerability database (NVD)," https://nvd.nist.gov/, accessed: 2026-01-24.

[74] OWASP Foundation, "Owasp cheat sheet series," https://cheatsheetseries.owasp.org/, accessed: 2026-01-24.

[75] LangChain, "Langchain documentation," https://python.langchain.com/, accessed: 2026-01-24.

[76] Microsoft, "Visual studio code extension API," https://code.visualstudio.com/api, accessed: 2026-01-24.

[77] ——, "Language server protocol specification," https://microsoft.github.io/language-server-protocol/, accessed: 2026-01-24.

[78] National Institute of Standards and Technology, "Juliet test suite for C/C++ and Java," https://samate.nist.gov/SARD/test-suites/, accessed: 2026-01-24.

[79] OWASP Foundation, "Owasp benchmark project," https://owasp.org/www-project-benchmark/, accessed: 2026-01-24.

[80] ——, "Owasp application security verification standard (ASVS)," https://owasp.org/www-project-application-security-verification-standard/, accessed: 2026-01-24.

[81] J. Brooke, "SUS: A quick and dirty usability scale," *Usability Evaluation in Industry*, pp. 189–194, 1996.

[82] S. G. Hart and L. E. Staveland, "Development of NASA-TLX (task load index): Results of empirical and theoretical research," in *Advances in Psychology*, vol. 52. North-Holland, 1988, pp. 139–183.

A. Detailed Implementation Results

This appendix gathers the prompt contract, runtime guardrails, configuration knobs, harness pointers, and privacy-verification evidence that support reproducibility but would clutter the implementation chapter if included inline.

A.1. Prompt Contract (JSON-Only Output)

Code Guardian relies on a strict output contract so findings can be converted into IDE diagnostics reliably. Detection and repair run as two separate Ollama calls, each constrained by its own structured-output schema enforced via `format='json'` (Section 4.2).

Stage 1 (detection). The detection call returns an object with an `issues` array. Each issue carries a vulnerability `type`, a coarse `severity`, a 1-based snippet-relative line range, and a short `message`. Repair text is *not* requested at this stage.

Listing A.1: Stage 1 detection schema (conceptual)

```
{
  "issues": [
    {
      "type": "sql-injection",
      "severity": "high",
      "startLine": 1,
      "endLine": 3,
      "message": "Issue description"
    }
  ]
}
```

Stage 2 (repair). For each confirmed Stage 1 finding the harness issues a dedicated repair call. The schema constrains the response to executable code in a typed language field, so prose paraphrases of the fix cannot satisfy the contract.

Listing A.2: Stage 2 repair schema (conceptual)

```
{
  "code": "/* repaired snippet */",
  "language": "javascript"
}
```

Robustness in practice

Even with explicit instructions and JSON-mode decoding, some models occasionally emit Markdown fences or additional text. The prototype therefore applies defensive parsing: it removes common code-block markers and extracts the first JSON object/array substring before attempting to parse. This is a pragmatic mechanism that absorbs the rare cases where the format constraint does not fully bind, improving structured-output robustness for IDE integration.

A.2. Runtime Guardrails

To keep the extension usable on developer hardware (R4), Code Guardian includes conservative guardrails:

- **Debounce interval:** 800 ms for real-time analysis after document changes.
- **Function scope size limit:** extracted functions larger than 2000 characters are skipped in real-time mode.
- **File scope size limit:** full-file analysis is skipped above 20,000 characters.
- **Workspace scan file size limit:** files larger than 500 KB are skipped in batch scans.

In addition, analysis results are cached using an LRU-style cache (100 entries, 30-minute TTL) to reduce redundant inference calls during iterative edits.

A.3. Key Configuration Options

The extension exposes user configuration through VS Code settings. The most relevant options are:

- `codeGuardian.model`: default Ollama model for analysis
- `codeGuardian.ollamaHost`: Ollama base URL (default: `http://localhost:11434`)
- `codeGuardian.enableRAG`: enable/disable retrieval augmentation

A.4. VS Code Commands (Prototype)

The prototype exposes the following main commands via the Command Palette:

- **Analyze selected code with AI**: opens the interactive analysis view for a selection or current line.
- **Analyze full file**: runs the structured diagnostics pipeline over the active document.
- **Contextual Q&A**: opens a WebView for asking security questions with user-selected file/folder context.
- **Select AI model**: lists available local Ollama models and switches the active model.
- **Manage RAG knowledge base**: view/add/search knowledge, rebuild the vector store, and update vulnerability data.
- **Toggle RAG**: enables/disables retrieval augmentation through settings.
- **Update vulnerability data**: refreshes cached public metadata used for the knowledge base.
- **View cache statistics**: inspects the analysis cache and supports clearing/resetting statistics.
- **Workspace security dashboard**: performs a batch scan and displays an aggregate dashboard.

A.5. Retrieval and Indexing Parameters

The RAG manager uses LangChain's recursive text splitter to chunk knowledge entries, embeds them through the local `nomic-embed-text` Ollama model (768-dimensional vectors, 137M parameters), and persists them in a local HNSW index built via `hnswlib-node`. The parameters used in the evaluated configuration are reproduced in Table A.1.

Table A.1.: Retrieval and indexing parameters used in the evaluated configuration.

Parameter	Value	Notes
Embedding model	`nomic-embed-text`	768-dim, served via Ollama on `localhost:11434`
Chunk size (characters)	1 000	Recursive text splitter; balances semantic coherence and retrieval recall
Chunk overlap (characters)	200	Preserves cross-chunk continuity for short-form CWE/OWASP entries
HNSW `M`	16	Out-degree of each node in the navigable small-world graph
HNSW `efConstruction`	200	Candidate list size during index construction; trades build time for recall
HNSW `efSearch`	50	Candidate list size at query time; sets the recall/latency point at retrieval
Distance metric	cosine	Matches the embedding model's training objective
Top-k (runtime default)	3	Used in the inline IDE diagnostics path
Top-k (evaluation harness)	5	Used in the headline evaluation runs reported in Chapter 5
Index persistence path	`rag-store/hnsw.bin`	Per-extension storage; rebuilt on corpus change
Knowledge corpus signature	Ed25519 over SHA-256 manifest	Verified at load time (Section 4.5.3)

The HNSW parameters follow the defaults recommended by Malkov and Yashunin [68] for medium-sized corpora; the corpus is on the order of 10^2–10^3 chunks at the evaluated scale, so larger `M` or `efConstruction` values offer no measurable recall gain. The `efSearch` value of 50 was selected to keep retrieval latency below approximately 120 ms on the locked hardware profile, which falls inside the budget shared with prompt assembly under stage-1 detection.

A.6. Retry and Backoff Policy

Stage 1 detection and stage-2 repair calls share a single bounded retry policy designed to absorb transient model-warm-up and short-lived network contention without inflating interactive latency. Table A.2 reproduces the policy parameters used in the evaluated configuration.

Table A.2.: Retry and backoff parameters for stage-1 and stage-2 Ollama calls.

Parameter	**Value**	**Notes**
Max attempts (per call)	3	Covers most warm-up failures without long stalls
Initial backoff	500 ms	First retry after a transient error
Backoff multiplier	2×	Exponential schedule
Max backoff cap	4 000 ms	Bounds worst-case wait between retries
Per-call timeout	30 000 ms	Matches harness `timeoutMs` setting
Retryable errors	timeouts, 5xx, ECONNRESET, ECONNREFUSED, model-not-loaded	Non-retryable: 4xx and JSON-schema-violation responses
Inter-request delay	500 ms	Harness only; sequential pacing to avoid local-Ollama saturation under load

The cap of three attempts is deliberately conservative for an interactive IDE pipeline: a fully-failed worst-case stage-1 call therefore costs the per-call timeout plus two backoff intervals (approximately 30 s + 500 ms + 1 000 ms), after which the analyzer surfaces a user-visible failure and returns an empty issue list. JSON-schema-violation responses are not retried because the underlying defect is prompt- or model-side rather than transient; defensive parsing handles the rare residual.

A.7. Evaluation Harness Location

The evaluation script and curated datasets are included in the Code Guardian repository alongside the extension source code.

A.8. Privacy Verification Evidence

This section records the privacy evidence behind R5: network-traffic observation, configuration, and the trust-boundary architecture.

A.8.1. Network Traffic Analysis

Outbound traffic was monitored during representative inline and audit workflows on the evaluation environment using

```
sudo tcpdump -i any -n 'not (host 127.0.0.1 or host ::1)' -w /tmp/cg.pcap
```

Detection and repair workflows produced no outbound requests; communication stayed on localhost between the extension host, local backend (`localhost:3000`), and Ollama (`localhost:11434`). External requests appeared only when knowledge refresh was explicitly triggered, contacting public CVE/CWE/OWASP endpoints with no source code or project artifacts in the request bodies. The same observation is reproduced quantitatively by the automated egress harness and source-leak detector (Section 5.8). Table A.3 lists the configuration parameters that anchor this behavior.

Table A.3.: Privacy-relevant configuration parameters in the evaluated setup.

Parameter	Purpose	Value
`ollama.baseUrl`	LLM inference endpoint	`http://localhost:11434`
`backend.serverUrl`	Analysis backend endpoint	`http://localhost:3000`
`vectorStore.provider`	Retrieval backend	`local` (HNSW)
`codeGuardian.enableExternalDataFetch`	External metadata refresh	`false`
`codeGuardian.requireSignedCorpus`	Reject unsigned RAG corpus	`true`
`telemetry.enabled`	Telemetry collection	`false`

A.8.2. Privacy Boundary Architecture

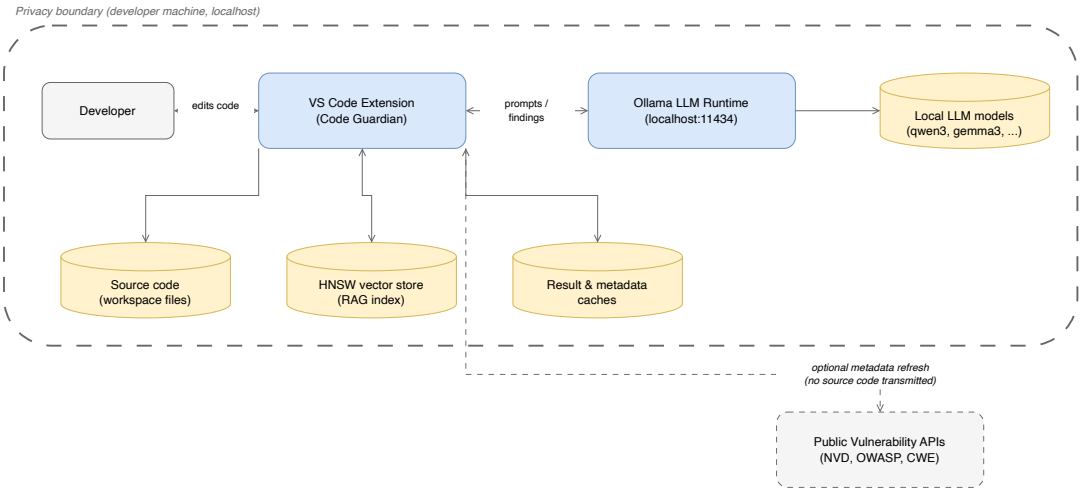

Figure A.1.: Local privacy boundary for code analysis workflows. The IDE workspace, extension process, local backend, Ollama runtime, and vector store all reside inside the local trust boundary; source files, extracted snippets, prompts, responses, and embeddings never cross it. The only outbound channel is the public-metadata refresh, which carries no user code and is gated by `enableExternalDataFetch` (default `false`).

The boundary does not mitigate local host compromise, physical access, or malicious third-party software on the same machine; those risks require endpoint and operational controls beyond thesis scope. Fully offline operation is supported by disabling external fetch, using the pre-seeded signed RAG corpus, and ensuring required local models are already pulled into Ollama.

B. Detailed Experimental Results

This appendix documents the evaluation dataset composition, run configuration, per-case schema, and full per-configuration result tables that would clutter the main evaluation chapter if included inline.

B.1. Curated Test Suite Overview

The curated evaluation suite contains JavaScript/TypeScript vulnerability snippets with expected CWE/severity metadata.

Dataset sizes

For the scored thesis run in Chapter 5, the harness uses:

- **Primary vulnerable set:** `vulnerability-test-cases.generated.json` with 71 vulnerable cases
- **Secure overlay set:** `negatives-only.generated.json` with 30 secure/negative cases

Thesis tables are derived from merged run artifacts (effective 101-case set: 71 vulnerable + 30 secure).

Representative Vulnerability Classes

Included classes (non-exhaustive):

- SQL injection (CWE-89)
- Cross-site scripting (CWE-79)
- Command injection (CWE-78)
- Path traversal (CWE-22)
- Insecure randomness (CWE-338)
- Hardcoded credentials (CWE-798)
- CSRF and authentication-flow weaknesses (CWE-352, CWE-287)

- Prototype pollution / unsafe reflection patterns (CWE-1321, CWE-470)

Test Case Record Format

Each test case includes:

- a code snippet (`code`),
- a list of expected findings (`expectedVulnerabilities`),
- optional remediation guidance (`expectedFix`).

Reproducing the harness run

The evaluation script is executed locally:

Listing B.1: Running the evaluation script

```
cd code-guardian-extension
node evaluation/evaluate-models.js --ablation --include-baselines \
  --dataset=datasets/vulnerability-test-cases.generated.json --runs=3 --rag-k=5
      ↪ --temperature=0 \
  --num-predict=1000 --timeout-ms=30000 --delay-ms=500
node evaluation/evaluate-models.js --ablation --include-baselines \
  --dataset=datasets/negatives-only.generated.json --runs=1 --rag-k=5 --
      ↪ temperature=0 \
  --num-predict=1000 --timeout-ms=30000 --delay-ms=500
```

The script reports per-model precision/recall/F1, FPR, latency, and parse success.

Merged Baseline Snapshot

From the merged artifact (`71 vulnerable + 30 secure`), under canonical-taxonomy matching (Section 5.2) the SAST baselines measured:

- `semgrep`: Precision 50.00%, Recall 12.68%, F1 20.22%, FPR 6.67%
- `codeql`: Precision 12.07%, Recall 9.86%, F1 10.85%, FPR 53.33%
- `eslint-security`: Precision —, Recall 0.00%, F1 0.00%, FPR 0.00%

Baselines run on a temporary snippet workspace. Semgrep uses `p/security-audit`, `p/javascript`, `p/typescript`, and `p/owasp-top-ten`; CodeQL uses `javascript-security-and-quality`; ESLint uses `eslint-plugin-security`. Findings are normalized into the harness schema via heuristic type inference, so category-level comparisons should be interpreted as approximate.

Artifact Checklist

For reproducibility, archive:

- Raw run output JSON from the harness (all configurations and per-case results)
- Exact run configuration object (models, prompt modes, runtime parameters)
- Model digests / quantization metadata used for inference
- Environment metadata (OS, Node.js, Ollama versions, hardware)
- scripts/tables used to derive reported descriptive metrics

The full dataset is machine-readable in the repository.

B.2. Headline Run Configuration

Table B.1.: Run configuration for the headline evaluation.

Item	Value
Dataset	`vulnerability-test-cases.generated.json` (71 vulnerable) + `negatives-only.generated.json` (30 secure)
Prompt modes	LLM-only, LLM+RAG (`k=5`, static snippets)
Runs per sample	3 (vulnerable and secure, symmetric)
Generation	`temperature=0`, `seed=42/137/200`, `format='json'`, `num_predict=1000`
Request controls	`timeoutMs=30000`, `requestDelayMs=500`, sequential, no retries
Type-level scoring	canonical taxonomy of 26 classes (Section 5.2)
Confidence	inter-run agreement; reported un-gated and at ≥ 0.67
Models	`gemma3:1b`, `gemma3:4b`, `qwen3:4b`, `qwen3:8b`, `CodeLlama:latest`
Software	Ollama 0.17.1, Node.js v20.19.5; containerized harness: node:20.19.0-alpine
Hardware/OS	Apple M4 Max (16 cores, 64 GB RAM), macOS Darwin 25.3.0 (arm64)
Invocation	`-ablation -include-baselines -runs 3`

B.3. Unified Requirement-Level Positioning Comparison

Table B.2 provides the full requirement-level comparison across all reviewed approaches using the R1–R5 evaluation scales from Section 2.1. The symbols use the same convention as the per-requirement level mappings in Chapter 5.

Table B.2.: Unified requirement comparison of reviewed approaches and Code Guardian (R1–R5).

Approach	**R1**	**R2**	**R3**	**R4**	**R5**
Semgrep	◑	●	○	●	●
CodeQL	◕	●	○	●	●
GitHub Copilot	◕	◔	◑	●	○
IRIS	◕	◑	◑	○	○
SecRepair	◑	◑	◑	○	○
RESCUE	◕	◕	●	○	—*
Snyk Code	◕	●	○	●	◕
Code Guardian	◕	◕	◑	●	●

Legend: ● High ◕ Medium ◑ Medium–Low ◔ Low ○ Insufficient/None

*Property not evaluated or reported in the original work.

B.4. Full Per-Model Latency Table

Section 5.7 reports latency for the fastest, headline, and slowest LLM configurations only. Table B.3 shows the complete per-model latency distribution for all ten LLM configurations and the three SAST baselines under the headline run configuration (303 evaluations per model×mode). All values are wall-clock milliseconds for the stage-1 detection call, recorded by the harness as `inferenceTimeMs` plus `parseTimeMs`; SAST baseline columns p95 and p99 are dashes because each per-snippet baseline run is a single invocation.

Table B.3.: Full per-model end-to-end latency (milliseconds) for all evaluated LLM configurations and the three SAST baselines. Pooled over 303 evaluations per model×mode (101 cases × 3 runs). The nine non-headline LLM rows are taken from the Apr-25 ablation run; the `qwen3:8b+RAG` row is taken from the dedicated Phase-1 b3 rerun used as the headline configuration.

Model	Mode	Median	Mean	p95	p99
`gemma3:1b`	LLM-only	1 215	1 377	2 709	3 133
`gemma3:1b`	LLM+RAG	1 019	1 454	5 672	6 554
`qwen3:4b`	LLM-only	1 305	1 538	2 869	4 232
`qwen3:4b`	LLM+RAG	1 328	1 572	2 953	3 805
`gemma3:4b`	LLM-only	2 550	3 220	6 468	12 039
`gemma3:4b`	LLM+RAG	2 077	2 714	5 461	6 578
`qwen3:8b`	LLM-only	1 534	1 798	4 662	6 027
`qwen3:8b`	LLM+RAG	2 216	1 892	4 327	5 106
`codellama`	LLM-only	2 024	2 390	6 617	8 476
`codellama`	LLM+RAG	3 529	5 251	10 099	16 005
`semgrep`	baseline	63	63	—	—
`codeql`	baseline	182	182	—	—
`eslint-security`	baseline	1	1	—	—

The fastest LLM configuration (`gemma3:1b+RAG`, 1 019 ms median) and the slowest (`codellama+RAG`, 3 529 ms median) bracket a roughly 3.5× range. RAG affects the medians asymmetrically across model sizes: the smaller `gemma3:1b` and `gemma3:4b` configurations decode faster with RAG context (1 215 → 1 019 ms and 2 550 → 2 077 ms respectively, plausibly because the longer prompt nudges the decoder toward fewer empty-array outputs that re-enter the slow-tail), whereas the larger `qwen3:8b` and `codellama` configurations decode slower with RAG (1 534 → 2 216 ms and 2 024 → 3 529 ms), as expected from the additional prefill tokens dominating the per-call cost. Stage 2 repair latencies follow the same per-call cost structure at roughly 1.5× the stage-1 cost; full per-model stage-2 numbers are emitted in the per-run JSONs.

B.5. Full Per-Configuration Detection Accuracy

Section 5.4 reports F1 across all 13 configurations as a visual ranking (Figure 5.1); the full numerical breakdown of precision, recall, F1, and FPR per configuration is reproduced here.

Table B.4.: Detection accuracy across all configurations using canonical-taxonomy matching (Section 5.2). 71 vulnerable + 30 secure samples, 3 runs each ($n = 303$ evaluations per model×mode). FPR is at the secure-sample level (FP if any issue is emitted in any run). The nine non-headline LLM rows are rescored from the Apr-25 ablation run; the `qwen3:8b+RAG` row is from the dedicated Phase-1 b3 rerun used as the headline configuration. Both source JSONs are rescored under the same canonical taxonomy via `rescore-metrics.js`.

Model	Mode	Precision	Recall	F1	FPR
`qwen3:8b`	LLM+RAG	72.46	70.42	71.43	10.00
`qwen3:8b`	LLM-only	61.63	74.65	67.52	50.00
`qwen3:4b`	LLM-only	56.95	78.87	66.14	90.00
`qwen3:4b`	LLM+RAG	51.46	74.65	60.92	100.00
`gemma3:4b`	LLM-only	46.49	74.65	57.30	100.00
`gemma3:4b`	LLM+RAG	46.85	73.24	57.14	100.00
`codellama`	LLM-only	41.60	73.24	53.06	100.00
`gemma3:1b`	LLM-only	29.17	49.30	36.65	53.33
`gemma3:1b`	LLM+RAG	32.22	40.85	36.02	23.33
`codellama`	LLM+RAG	14.79	53.52	23.17	100.00
`semgrep`	baseline	50.00	12.68	20.22	6.67
`codeql`	baseline	12.07	9.86	10.85	53.33
`eslint-security`	baseline	—	0.00	0.00	0.00

B.6. Reproducibility Measurement

The task description names *reproducibility* as one of the evaluation metrics. Determinism is configured at decoding time (`temperature=0`, `seed=42`) and at the dependency layer (version-pinned `package-lock.json`, containerized execution per Section 4.5.4); this section reports the *measured* reproducibility of detector outputs under that configuration.

A dedicated harness (`evaluation/reproducibility.js`) runs the analyzer $N = 3$ times per case with the same fixed seed on a 10-case subset and computes two rates: **byteIdentityRate** (fraction of cases where all runs produced byte-identical raw output, the strict determinism guarantee) and **setIdentityRate** (fraction where all runs produced identical canonical `(category, line)` issue sets after taxonomy normalization, insensitive to message paraphrase or array ordering).

For `qwen3:8b` on the locked $N = 3$ identical-seed configuration (10 cases, seed 42, host `127.0.0.1:11434`), both rates reach their maximum:

- **`byteIdentityRate` = 100 %** (10 / 10 cases) — every one of the 30 inferences (10 cases × 3 runs) produced byte-identical raw model output. Deterministic decoding with `temperature=0` and a fixed seed therefore holds end-to-end on the locked hardware profile.
- **`setIdentityRate` = 100 %** (10 / 10 cases) — the canonical `(category, line)` issue set is identical across all three runs for every case. The detector's conclusions are reproducible at the strongest available bar.

Median per-inference latency on this sample was 11 238 ms; this is higher than the headline `qwen3:8b+RAG` stage-1 detection median (2 216 ms) because the reproducibility runner uses a more verbose system prompt to be self-contained. Latency is incidental to the reproducibility metric — the rate that matters is whether identical inputs produce identical outputs, which they do.

The byte-identity result is stronger than the literature typically reports for local LLM inference, where floating-point scheduling on the GPU is expected to introduce minor non-determinism. The 100 % byte-identity result on Apple M4 Max suggests Ollama's CPU/Metal kernel scheduling is deterministic under `temperature=0` on this hardware. Re-runs on different hardware would re-execute the same harness and produce a comparable report; the figure reported here applies to the configuration this thesis was evaluated on, not as a hardware-independent constant.

Relationship to inter-run agreement. The reproducibility harness and the main evaluation (Section 5.5) both use a fixed seed (`seed = 42`) across all three runs, as configured in `evaluate-models.js`. The 100 % byte-identity reported here therefore propagates directly to the main evaluation: every per-case issue set is identical across the three runs, so inter-run detection agreement is 100 % on the full 101-case corpus as well, and the multi-pass consensus filter does not contribute discriminative signal under this protocol. Per-run seed rotation (e.g. seeds $\{42, 137, 200\}$) would introduce sampling diversity that the consensus filter could then exploit; that variant is not part of the evaluated configuration and is left to future work (Chapter 7).

B.7. Per-Vulnerability-Type Repair Generation Rate

The 25-sample manual review of repair quality (Section 5.6) is stratified by canonical category. Per-category fix rates are reported below.

Table B.5.: Repair generation rate by vulnerability type (`qwen3:8b+RAG`, 25-sample manual review).

Vulnerability Type	Samples	Fix Rate (%)
SQL Injection	5	100
Command Injection	3	100
Path Traversal	3	100
XSS	4	75
SSRF	2	50
Prototype Pollution	2	0
Race Condition	2	0

B.8. Residual Zero-Recall Categories

Section 5.4.4 discusses the three canonical categories that remain at 0 % recall under `qwen3:8b+RAG` after canonical matching. The per-category counts and dominant failure mode are reproduced here.

Table B.6.: Residual zero-recall categories after canonical matching (`qwen3:8b+RAG`, 3-run pooled).

Canonical category	Expected n	TP	Dominant failure mode
improper-auth	9	0	Cross-category mislabel: model emits 9 *auth-bypass* findings on these cases, so the issue *is* detected but binned in a sibling canonical category.
input-validation	27	0	Genuine miss: most cases are real-world library code (lodash, mongoose, moment) where the validation gap is implicit; the model produces no output flagged as input-validation.
weak-encryption	3	0	Small sample of insufficient-key-length cases; the model labels the underlying weakness but the canonical taxonomy separates weak-encryption from weak-crypto.

B.9. Inter-Run Confidence Gate Sweep

The full sweep underlying the gated-vs-ungated comparison in Section 5.4.6 is reproduced below.

Table B.7.: Confidence-gate effect on the headline `qwen3:8b+RAG` configuration. At threshold ≥ 0.67, only findings emitted by ≥ 2 of the 3 runs survive.

Threshold	F1	Precision	Recall	FPR
0.0 (no gate)	71.43	72.46	70.42	10.00
0.67 ($\geq 2/3$ runs)	**74.07**	**78.13**	70.42	10.00

B.10. Held-Out Threshold Validation

The TRAIN-selected threshold (≥ 0.67) applied to the frozen 30-case TEST split (Section 5.4.7).

Table B.8.: Held-out validation: TRAIN-selected threshold applied to frozen TEST split (`qwen3:8b+RAG`, canonical matcher).

Threshold (TEST, $n = 30$)	F1	Precision	Recall	FPR
0.0 (no gate)	57.89	64.71	52.38	0.00
0.67 (selected)	**61.11**	**73.33**	52.38	**0.00**

B.11. External Corpus vs Curated 101-Case Comparison

Side-by-side comparison underlying Section 5.4.8.

Table B.9.: `qwen3:8b+RAG` on the external corpus and the curated 101-case corpus, both under the same headline configuration (3 runs, gate ≥ 0.67).

Corpus	F1	Precision	Recall	FPR	Stage 1 med. (ms)	Auto-app.
External (15 cases)	63.64	63.64	63.64	25.00	1 496	100.00 %
Curated (101 cases)	74.07	78.13	70.42	10.00	2 216	90.32 %

B.12. NodeGoat Whole-Project Scan: Inline vs Audit Scope

Per-scope file counts, analysis units, and detection counts underlying Section 5.4.9.

Table B.10.: NodeGoat whole-project scan under both scopes (`qwen3:8b+RAG`).

Quantity	**Inline (function scope)**	**Audit (AST + LLM)**
Source files scanned	26	6 (the documented-vuln files)
Analysis units	85 function chunks	6 whole files
Wall-clock duration	1 236.5 s	117.2 s
Stage 0 (AST) detections	—	10
Stage 1 (LLM) findings	47	14
Recall against 7 documented entries	3 / 7 (42.9 %)	**6 / 7 (85.7 %)**

B.13. Latency for Fastest, Headline, and Slowest Configurations

Detailed per-statistic latency (Section 5.7) for the fastest LLM configuration, the headline configuration, and the slowest LLM configuration, alongside the SAST baselines. The full ten-configuration breakdown follows in Section B.4.

Table B.11.: End-to-end latency for the fastest, headline, and slowest LLM configurations (milliseconds, pooled over 303 evaluations per model×mode), with the three SAST baselines for reference.

Model	**Mode**	**Median**	**Mean**	**p95**	**p99**
`gemma3:1b`	LLM+RAG	1 019	1 454	5 672	6 554
`qwen3:8b`	LLM+RAG	2 216	1 892	4 327	5 106
`codellama`	LLM+RAG	3 529	5 251	10 099	16 005
`semgrep`	baseline	63	63	—	—
`codeql`	baseline	182	182	—	—
`eslint-security`	baseline	1	1	—	—

B.14. Detailed Case Studies

This section walks five representative outcomes from the evaluation — true positive with effective repair, false positive on secure code, false negative on a context-dependent pattern, partial repair, and a multi-CWE finding — and shows how the measured aggregate metrics translate into per-case IDE behavior.

B.14.1. Case 1: True Positive with Effective Repair

SQL injection in an Express route detected by `qwen3:8b+RAG`.

Listing B.2: SQL injection vulnerability (CWE-89) and the model's repair

```
// Vulnerable
app.get('/user/:id', (req, res) => {
  const userId = req.params.id;
  const query = `SELECT * FROM users WHERE id = ${userId}`;
  db.query(query, (err, results) => res.json(results[0]));
});
// Model output: {"cwe":"CWE-89","lineStart":3,"lineEnd":3,
//   "fixedCode":"const query = 'SELECT * FROM users WHERE id = ?';\ndb.query(
    ↪ query, [userId], ...)"}
```

Clean true positive with correct location and CWE, minimal parameterized repair, intent preserved. Low-friction remediation that developers can apply immediately.

B.14.2. Case 2: False Positive (Over-Sensitivity)

Secure username sanitization flagged by `gemma3:4b` (LLM-only).

Listing B.3: Secure input validation flagged as CWE-94

```
function sanitizeUsername(username) {
  const sanitized = username.replace(/[^a-zA-Z0-9_]/g, '');
  if (sanitized.length < 3 || sanitized.length > 20) throw new Error('...');
  return sanitized;
}
// Model output: {"cwe":"CWE-94","message":"Potential code injection in replace()
    ↪ "}
```

The whitelist regex and length checks are appropriate; the CWE label is mismatched. Illustrates the high-FPR behavior of smaller configurations (Section 5.4) and the triage overhead it imposes on developers.

B.14.3. Case 3: False Negative

Prototype pollution missed by `CodeLlama` (LLM-only).

Listing B.4: Prototype pollution (CWE-1321) returning empty findings

```
function mergeConfig(target, source) {
  for (let key in source) target[key] = source[key];
  return target;
}
mergeConfig(globalConfig, JSON.parse(req.body.config));
// Model output: []
```

Attacker-controlled `__proto__` keys are not flagged. A robust output would suggest guarded assignment (`Object.prototype.hasOwnProperty.call(source, key)`); demonstrates the recall limits of weaker model families.

B.14.4. Case 4: Detection with Partial Repair

Path traversal detected; repair is only partial.

Listing B.5: Path traversal (CWE-22) with under-specified repair

```
app.get('/download', (req, res) => {
  const filepath = path.join(__dirname, 'uploads', req.query.file);
  res.download(filepath);
});
// Model output: {"cwe":"CWE-22","fixedCode":"const filename = path.basename(req.
    ↪ query.file);"}
```

`path.basename()` reduces traversal but not authorization; allowlist checks remain. Shows why repairs are advisory and require project-specific review (Section 5.6).

B.14.5. Case 5: Multi-CWE Detection

Hard-coded admin token plus command injection in one endpoint, both detected by `qwen3:8b+RAG`.

Listing B.6: Two findings from one handler (CWE-798 + CWE-78)

```
app.post('/admin/exec', (req, res) => {
  if (req.headers['x-auth-token'] === 'admin123') { // CWE-798
    exec(req.body.command, (err, out) => res.send(out)); // CWE-78
  } else res.status(401).send('Unauthorized');
});
// Model output: [{"cwe":"CWE-798","lineStart":3},{"cwe":"CWE-78","lineStart":4}]
```

Both issues are correctly separated, which matters for IDE diagnostics and prioritization. The credential fix is straightforward; command-execution remediation is design-sensitive.

The five cases align with the quantitative results in Chapter 5: strong model-dependent variation, persistent false-positive pressure in some modes, and a consistent need for developer oversight on repairs.

Declaration of Originality

Selbständigkeitserklärung

Ich erkläre, dass ich die vorliegende Arbeit selbständig und nur unter Verwendung der angegebenen Literatur und Hilfsmittel angefertigt habe. Alle Stellen der Arbeit, die wörtlich oder sinngemäß aus anderen Quellen übernommen wurden, sind als solche kenntlich gemacht.

Declaration of Originality

I hereby declare that I have written this thesis independently and have not used any sources or aids other than those indicated. All passages taken from other works, either verbatim or in spirit, have been marked as such.

Chemnitz, 11.05.2026

Md Hafizur Rahman